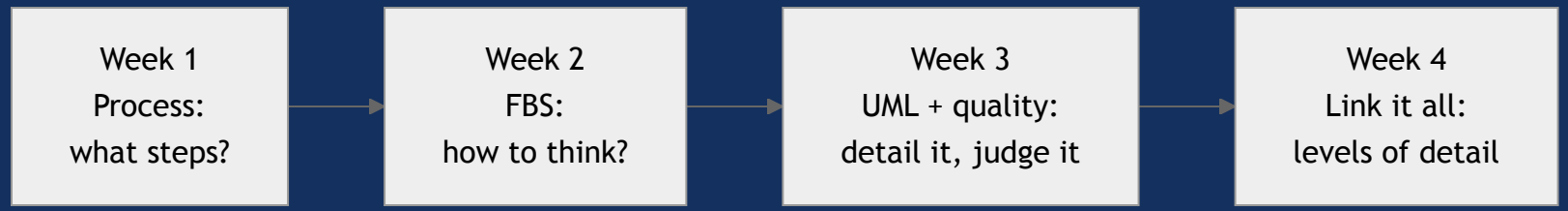
Software Conceptual Design: the story in 4 weeks

One story: **you must build a feature (Amazon Pay, then a mood-based music player). How do you go from idea to working software?**



Week 1: The process (what steps do we follow?)

1.1 Software = modules

Amazon is built from **modules** (separately addressable components, integrated to satisfy requirements).

Component	What it does
Inventory management	measures **amount, location, pricing, mix** of products; home page updates with **purchasing, seasonal trends, demand** (logistics + analytics); page is **personalised** by items viewed + shopping history
Payment gateway	authorises electronic payments (net banking, debit cards...); **middleman between bank and merchant site**. Flow: site sends **encrypted card info** → gateway **confirms with the bank** → money moves from user to merchant
Amazon Pay	a **mobile wallet** (digital cash): link card/bank, categories (recharge, bills, travel, rewards, gift vouchers), **add money**, **auto reload**

1.2 Five phases

Requirements. Goals the system must meet; they must cater to the clients. First step = understand the problem + study existing/similar systems (Paytm, PhonePe), **not** "learn a programming language". Think: **who** uses it, for **what**, in **what way**.

Client type	Example
External user	mobile banking app serves bank customers
Internal to the company	employee resource portal built by an internal team with HR
Another software	Razorpay gateway between shop site and bank

Design. Big-picture view; gives structure. Without it: developers have **different ideas** of how functionality works, change a function and **break a colleague's component**, delays, and **no overview to add new features**.

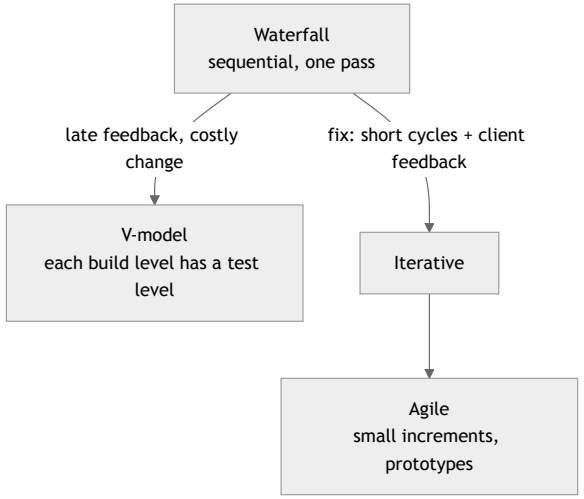
Development. Write code from requirements + design. **Distributed** (many places/time zones), tools like **Git/GitHub**, hence **documentation + precise interfaces**. - **Interface** = *what* is accepted (inputs, format) and *what* is returned, **not how**. Example: inputs W, X, Y → output Z. **Code behind an interface can change** if the interface stays.

Testing. Check the software **behaves per requirements**; bugs still exist. Cost of bugs to the US economy: **~$59.5 billion (2002) → $1.1 trillion (2016)**.

	Who	Where	Goal
Alpha	**internal employees**	**lab / staging**	catch as many issues as possible **before public release**
Beta	**actual users**	**real-life setting**	real-world feedback

Maintenance. After release: **monitor** how users use it, **change code** for upgrades/fixes (missed conditions, browser-specific UI bugs), **add features**, plan the next release from feedback.

1.3 Development models



- **Waterfall:** phases strictly in sequence. Story: 2+3+4+3 = **12 months**, client sees it first at release: "I don't like it!". **Drawbacks:** (1) later change raises **cost and time**, (2) **clients don't know what they need**, (3) **designers don't know** which design is feasible/usable.
- **V-model:** waterfall **bent upward** after implementation. Left arm: concept → requirements/architecture → detailed design → **implementation** (bottom). Right arm: test/verify **individual modules** against design → verify/validate **whole system** against architecture → operation & maintenance.
- **Iterative:** many iterations: plan/build key features → deploy → **client feedback** → refine next iteration → repeat until complete.
- **Agile:** an iterative model; **small increments**, **prototypes** of key features released quickly. Frameworks: **Scrum, Kanban**. Practices: **sprints, test-driven development**. An **umbrella term / philosophy**, more than frameworks.
- Teams **choose and adapt** a model to their process and software type.

1.4 Software vs building (and car)

	Building	Software
Parts	**tangible**	**logical, abstract**
Change	hard	**adaptable**, anyone on the team can change it
Over time	**static**	**keeps changing** (Winamp → Saavn/Gaana)
Similar	blueprint first, requirement gathering, site visits / understanding end users, scale models	

MCQ rules, Week 1

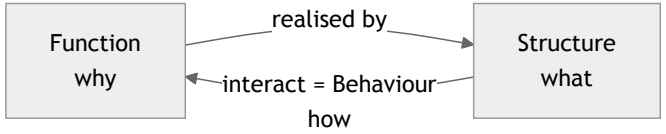
- "NOT a phase" → **UI design**. Stage before development → **Design**.
- Internal colleagues test → **alpha**; real users → **beta**.
- "Gather info/regulations" → requirements · "modules + interaction" → design · "feedback + next release" → maintenance · "release to select users, find errors" → testing.
- Lab builds a model for itself → **internal** client.
- Interface → what, not how; code **can** change. Waterfall's drawback → **changing requirements**.

Week 2: How to think about a design (FBS)

Phases and models don't say *how to think*. Experts use past systems as **reference points** and keep an **integrated view** asking **why, what, how**:

	Asks	Is	Mood-player example
Function (F)	**Why**	**Features**, deliberated from requirements	mood detection, playlist creation, user authentication
Structure (S)	**What**	**Physical + logical components** that realise functions	emotion camera, playlist algorithm, emotion tagging of songs
Behaviour (B)	**How**	**Interaction between structures** that achieves the function	camera recognises emotion → gives it to playlist generator → playlist by mood

Mood player: auto-detects mood, plays music by mood, secure authentication, remembers choices, recommends by mood + choice.



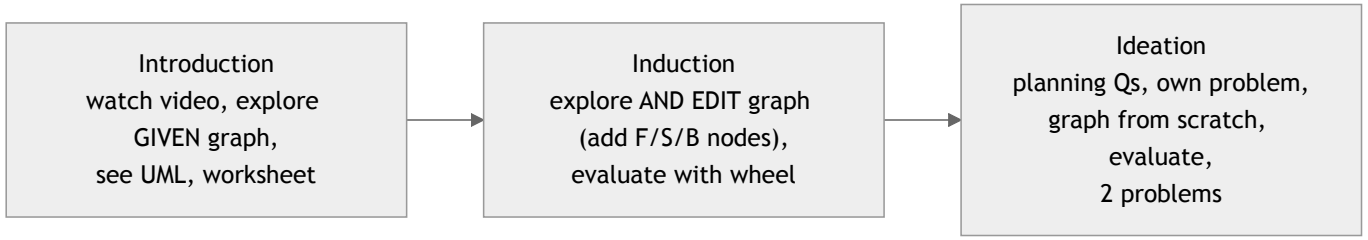
Why integrate F, B, S? - Not integrated → **incoherent/incomplete design, many solutions, undecidable** (mood detection alone: webcam vs voice vs history). - Integrated → (1) **unified view**, (2) **all requirements met**, (3) **team/stakeholder communication**, (4) **avoid getting stuck**.

How to integrate (5 steps): start from a **known solution** (e.g. existing mood cameras; check it fits) → think as **end user and system** to generate **behaviours** → **map** the structures working together in each behaviour → check how structures + behaviours **satisfy a requirement** → you now have F, B, S **identified and mapped**.

FBS graph: nodes = F/B/S (slides: F yellow, S blue, B red), links connect them, start from **any** node, **many valid graphs** (e.g. BSF, FSB), never one correct graph.

Think & Link (tool to build + evaluate FBS graphs)

Pedagogical agent (prompts), notes box, tabs unlock as tasks complete.



Introduction worksheet (6 tasks): list F/S/B nodes · true/false with reasons · describe F, S, B in own words · list **connector types** (F-S, S-B ...) · examples of **pairs and trios** · write abstract sentences like "Function is implemented by Structure used during Behaviour". **UML from FBS (component diagram):** one **component per structure node**; reflect **hierarchy** between structure nodes; check components that **implement no function**; every F-B-S link should have a matching interface.

Evaluation wheel (core: syntax, semantic, pragmatism). Each slice rated **Meets Expectation / Needs Improvement / Inadequate / Missing**, with a reason and planned changes. Can edit the graph and re-evaluate.

Criterion	Means
Complexity	**number of nodes** in each of F, B, S
Connectivity	F, S, B nodes **interconnected** (linking purpose, concept, component, mechanism)
Formal realisation	graph **converts to UML diagrams**
Level adjacency, Validity, Consistency, Levels	on the wheel; **not defined in lectures** (only names appear)

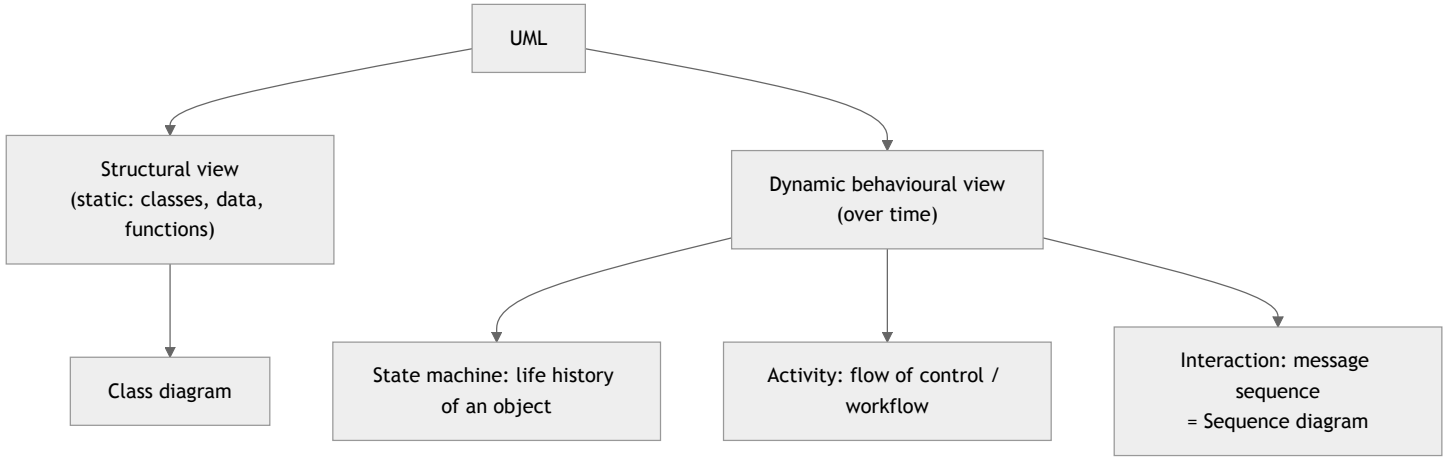
Summary (L13): conceptual design = a design description, **graphical, numerical or textual**; technique FBS, tool Think & Link.

MCQ rules, Week 2

- No integration → **incoherent/redundant, multiple solutions**. Integration advantages → **all four**.
- How many graphs → **multiple**. Reason to link → **integrated solution**.
- Explore **then edit** → **Induction**. Build from scratch → **Ideation**.
- "Camera → playlist generator → playlist" = **interaction/behaviour** (and implementation of the feature), **not a use case**.
- Camera, algorithm = **structure**; "mood detection" = **function**; "user places face near camera" / "system fetches songs" = **behaviour**.
- Complexity = **count**; connectivity = **links**; formal realisation = **UML**.

Week 3: Detail it in UML, then judge it

Modelling = creating an **external, explicit representation** of the system to be built. Benefits: plan what to code, **communicate to a larger team**, overview. FBS = **idea/concept**; UML = **more detail, multiple views**. (Example: one FBS function, mood detection, is expanded as a sequence diagram.)



Other UML diagrams exist (state chart, communication, use case, component, deployment...); a **subset** is used per design.

Class diagram (main building block of OO modelling): box with **3 compartments: name / attributes (data members) / operations (member functions)** + relationships. So it includes behaviour as operations. Mood player classes: User, User Voice Profile, Playlist, Song, User Mood (`moodName`, `detectMood()`). ATM: `balance` = data member, `setOption()` = member function.

Sequence diagram: objects from the class diagram on top, each with a **lifeline**; **horizontal arrows = messages in time order** (class-diagram functions such as `detectMood`, `generatePlaylist`). **One per requirement** (mood detection, login). Login has **alt/else**: valid → fetch profile/playlist → success; else → unsuccessful. For a check function returning false, read the **else message** ("Incorrect PIN").

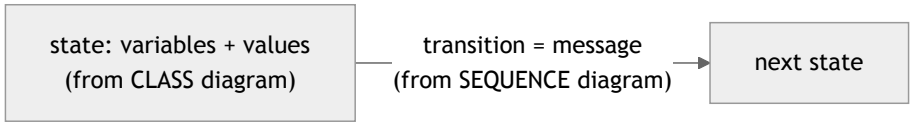
Understanding a design

Design comprehension = understanding a given design. Needed because developers work on **large existing systems** and add features (e.g. integrating the mood player into Amazon Music).

Student difficulties	Expert knowledge (4 kinds)
(1) **notations** (syntax)	**Problem domain** (how a music player works)
(2) mapping **requirements** ↔ **design**	**Design diagram** knowledge (syntax + semantics)
(3) **connecting diagrams** (sequence uses class-diagram objects)	**Main goals** (purpose of each diagram; made of **sub-goals**: input, success, failure)
	Dynamic behaviours (how class **data members change** as the sequence runs)

VeriSIM = Verifying designs by Simulating scenarios; gives an integrated understanding of **class + sequence** diagrams. Context: **automated door-locking system**: (1) if no passcode set, user **registers** one; (2) **lock** + correct passcode → locks, incorrect → **stays unlocked**; (3) **unlock** + correct → unlocks, incorrect → **stays locked**. Diagrams: class (Door, Controller, Passcode) + several sequence diagrams.

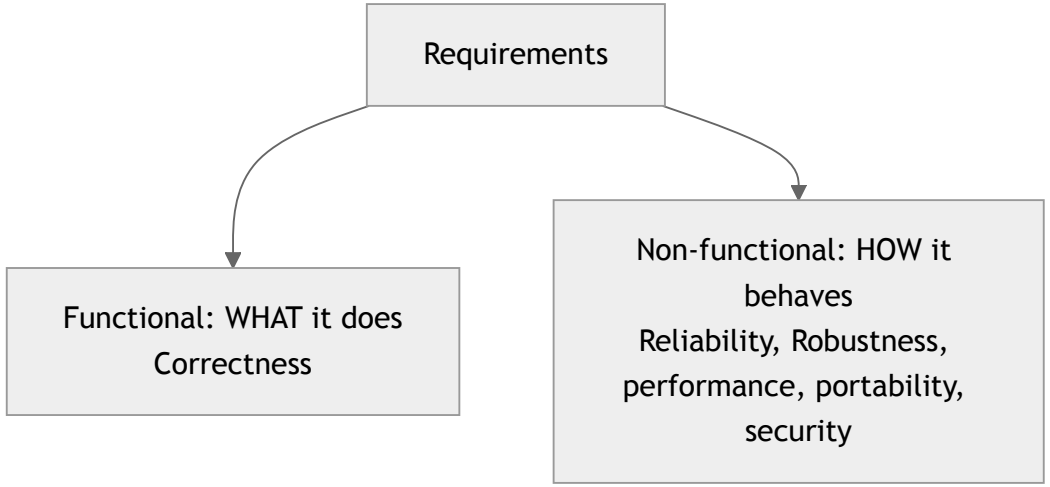
Design tracing = build a **state diagram for one scenario**:



e.g. start `lock=true`, E1 "set option unlock", E2 "enter correct passcode", E3 "door unlocks" → `lock=false`.

VeriSIM stages: Introduction (objectives) → **Problem understanding** (requirements + diagrams) → **Design tracing** (4 challenges, any order: **explore, correct, complete, construct the model**) → **Reflection**. Each activity has evaluation + reflection.

Judging a design: quality (tiffin-service analogy)



Attribute	Definition	Tiffin	App
Correctness	behaves **according to specification** (a **functional** requirement)	on time, no mix-up (breakfast for lunch)	all requirements satisfied (as in VeriSIM tracing)
Reliability	behaves the **same way over time** in the same environment	misses >3 meals a week	does **not crash often**; withdraw/deposit work
Robustness	**recovers from errors / unexpected input**	chef sick, replacement	**payment failure**, negative/large numbers, high traffic

Checking reliability/robustness checks **how it behaves**, not the requirements. Quality attributes are **implicit, not in diagrams**, so **write them down** with the decisions based on them.

MCQ rules, Week 3

- Two views → **structural + dynamic behavioural**. Design tracing → **state diagram of a scenario**; state variables ← **class**, transitions ← **sequence**.
- Functional = **features**; non-functional = **behaviour**.
- Handles failures/unexpected input → **robustness**; crashes often → **reliability**.
- "Class diagram only has data members" → **False**. `deposit()` returns → **True/False**.

Week 4: Link it all (levels of detail)

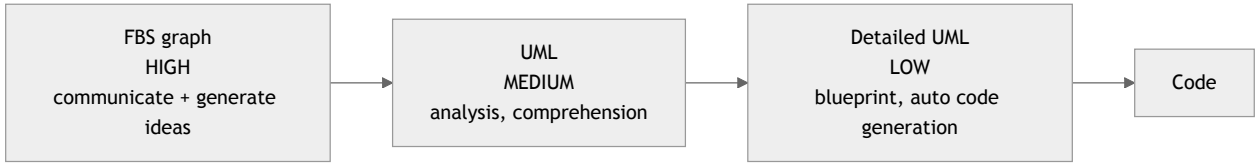
Recap: week 1 modules (**modularization**) and phases; weeks 2-3 models. Development is a **systematic problem-solving process**, phases are broad and done **iteratively in many ways**. Software design is harder than architecture because software is **abstract and intangible**, though design processes are similar across domains.

FBS definitions: F = functionalities/features, **S = physical + logical components**, B = how structures interact.

Mapping FBS to the process (table from the Rational Unified Process): linking two FBS elements triggers a process.

Link	Process initiated
Function ↔ Behaviour	**Requirement definition**
Structure ↔ Behaviour	**Design analysis**, testing, implementation, requirements change
Behaviour ↔ Behaviour, **Structure ↔ Structure**	**Evaluation, decomposition, refinement**
Function ↔ Structure	**Reusability and change in needs**

Abstraction = hiding details at each stage of design; **level of detail = level of abstraction**.



FBS → UML: **functions → use cases**, **structures → class diagrams**, **behaviours → sequence diagrams**. Detailed UML ↔ code: **each model class maps to one or more code classes**; tools **auto-generate code**.

Term	Meaning
Modularization	split into **independent co-working modules**, the system's **building blocks**
Abstraction	**hide details** at each design stage
Decomposition	**break down** implementation components

Experts spend a lot of time **early**: understanding the problem, building models, **checking models in different scenarios**, before implementing. That is **software conceptual design**.

MCQ rules, Week 4

- Structures in FBS → **physical and logical components**.
- Purposes of modelling (MSQ) → **communication/idea generation + analysis/comprehension + blueprint for implementation**; **not** "gather requirements from stakeholders".
- Communicate key ideas first → **FBS**; devs transform directly to code → **detailed UML**. FBS = **high-level overview**, UML = **more detailed**.
- Mapping questions: Function↔Behaviour = **requirements**; Structure↔Behaviour = **design analysis**; B↔B / S↔S = **evaluation/decomposition**; F↔S = **reusability**.
- Independent modules → modularization; hide details → abstraction; breaking components → decomposition.

Last 10 minutes

1. Requirements → Design → Development → Testing → Maintenance. Clients: external / internal / software.
2. Alpha = internal + lab; Beta = real users. Waterfall problem = change. Agile = Scrum, Kanban, sprints, TDD.
3. Interface = what not how; code can change.
4. F = why/features, S = what/components, B = how/interactions. No integration → incoherent, many solutions.
5. Many FBS graphs. Induction edits, Ideation creates. Complexity = nodes, Connectivity = links, Formal realisation = UML.
6. UML: structural (class: name/attributes/operations) + dynamic (sequence: lifelines, messages). VeriSIM tracing = state diagram of a scenario.
7. Correctness (functional) · Reliability (crashes) · Robustness (failures, odd input).
8. FBS (high) → UML (medium) → detailed UML (low) → code. Modularization / abstraction / decomposition.

SOFTWARE CONCEPTUAL DESIGN

Prof. Sridhar Iyer
Prof. Prajish Prasad
Prof. T. G. Lakshmi
Computer Science and Engineering
IIT Bombay

INDEX

S.NO	TOPICS	PAGE.NO
	Week 1	
1	Week 1 : Lecture 1	3
2	Week 1 : Lecture 2	4
3	Week 1 : Lecture 3	12
4	Week 1 : Lecture 4	20
5	Week 1 : Lecture 5	27
6	Week 1 : Lecture 6	31
7	Week 1 : Lecture 7	37
	Week 2	
8	Week 2 : Lecture 8	39
9	Week 2 : Lecture 9	40
10	Week 2 : Lecture 10	42
11	Week 2 : Lecture 11	47
12	Week 2 : Lecture 12	54
13	Week 2 : Lecture 13	71
	Week 3	
14	Week 3 : Lecture 14	72
15	Week 3 : Lecture 15	73
16	Week 3 : Lecture 16	77
17	Week 3 : Lecture 17	83
18	Week 3 : Lecture 18	90
19	Week 3 : Lecture 19	94
20	Week 3 : Lecture 20	100
	Week 4	

21	Week 4 : Lecture 21	101
22	Week 4 : Lecture 22	102
23	Week 4 : Lecture 23	105
24	Week 4 : Lecture 24	108
25	Week 4 : Lecture 25	114

Software Conceptual Design

Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering
Indian Institute of Technology, Bombay

Lecture - 01
Deconstructing a Software System

Welcome to week 1 of this Software Conceptual Design course. In this week we will deconstruct an e-commerce application to understand the software conceptual design process broadly. We will also have activities for you to do, as well as dialogues between the instructors to understand this process, what the various components of a large software system are and so on.

We will also be looking into the software design process, the various stages that are involved, such as requirements specification, detailed design, testing and maintenance. This will give you a big picture on how software systems are built.

Software Conceptual Design

Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering
Indian Institute of Technology, Bombay

Lecture - 02
Thinking of Software in terms of Components

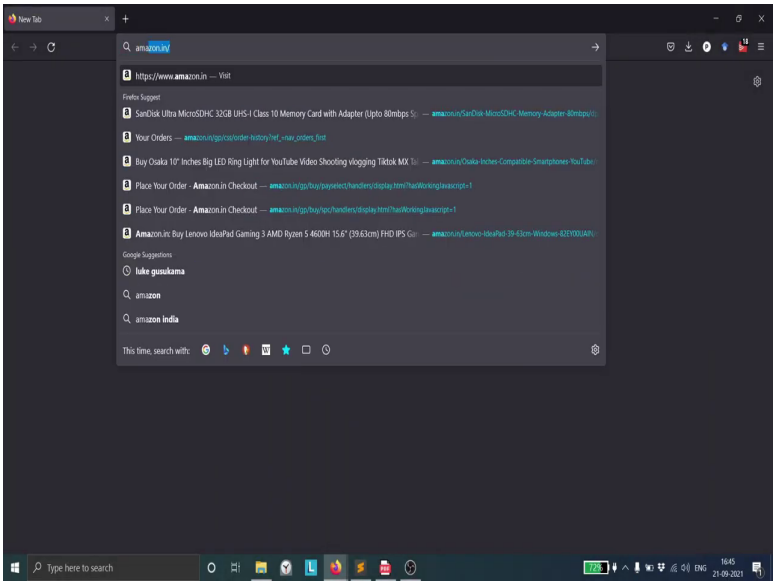
Sorry I did not notice you, I was busy buying something from Amazon. They seem to have everything on their site, and the process is so effortless. How do you think they built such a software system?

Well, I think one major component is their inventory management system, they keep track of their inventory in such an intelligent way.

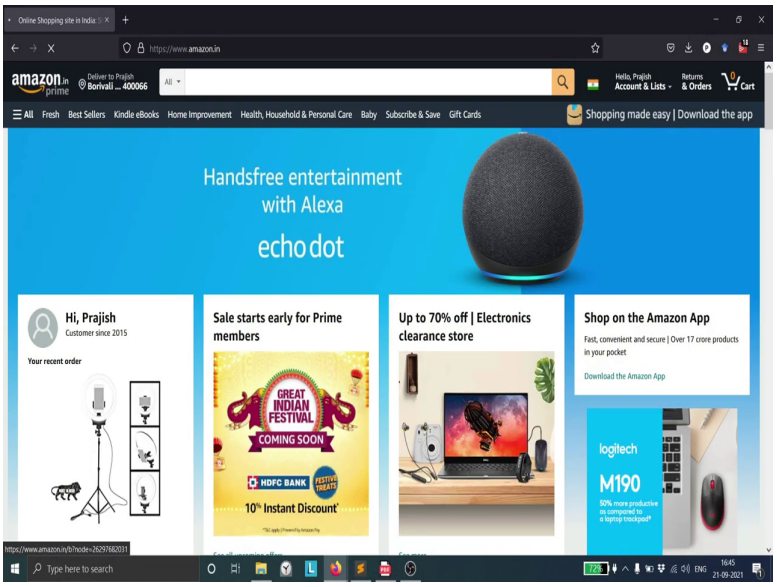
What do you mean by intelligent, can you elaborate?

So, inventory management is the act of measuring the amount, location, pricing and the mix of products available.

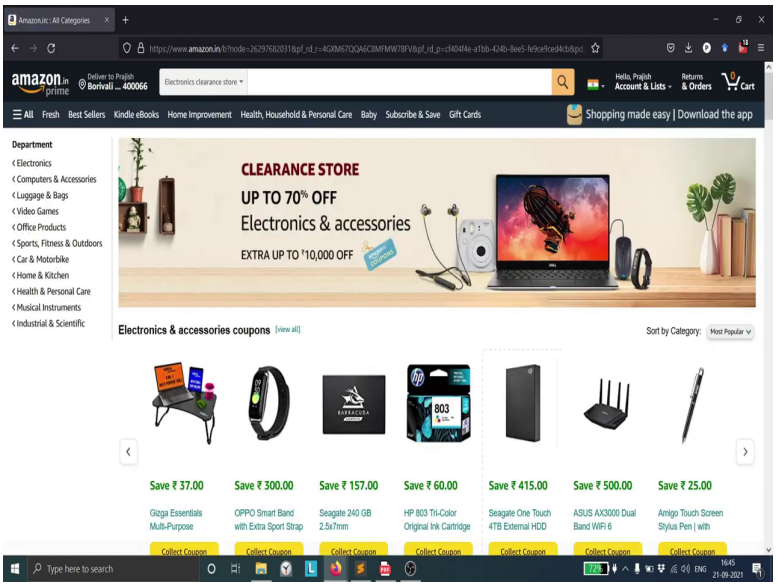
(Refer Slide Time: 00:43)



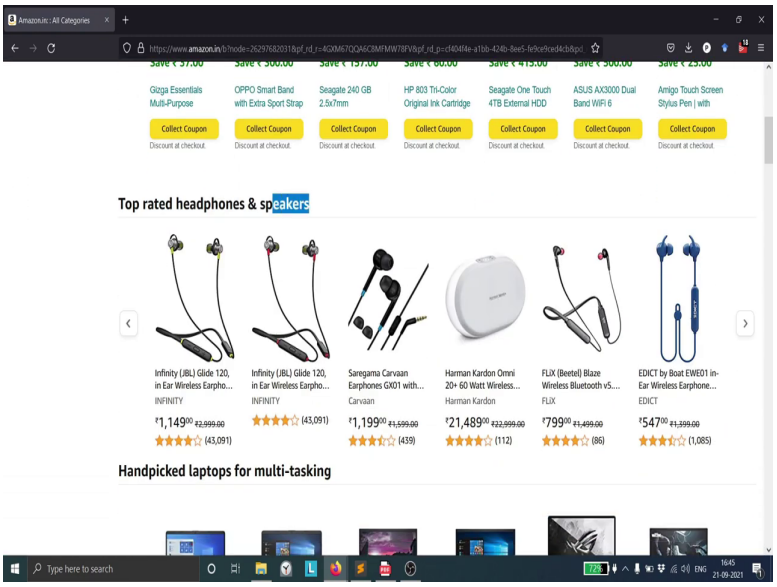
(Refer Slide Time: 00:44)



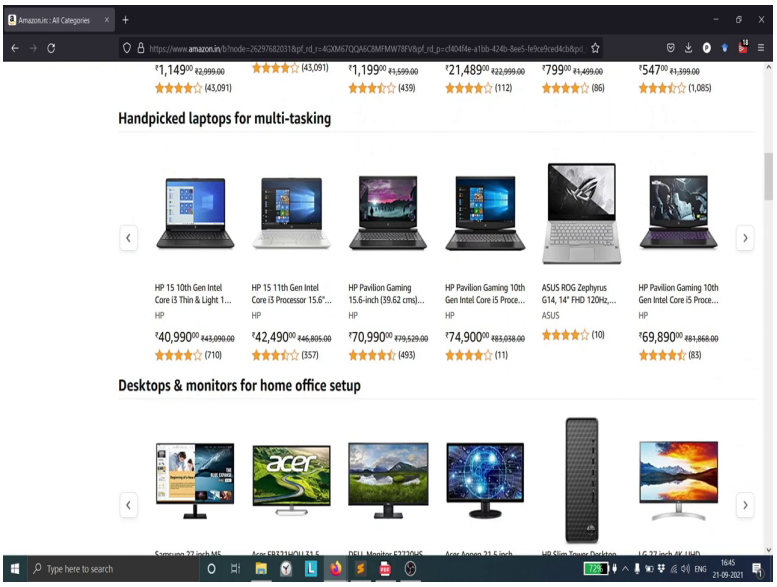
(Refer Slide Time: 00:50)



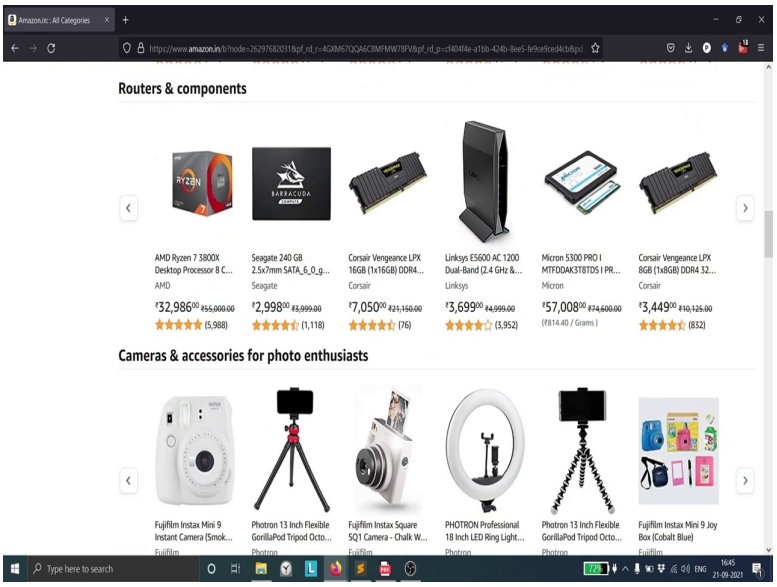
(Refer Slide Time: 00:53)



(Refer Slide Time: 00:56)

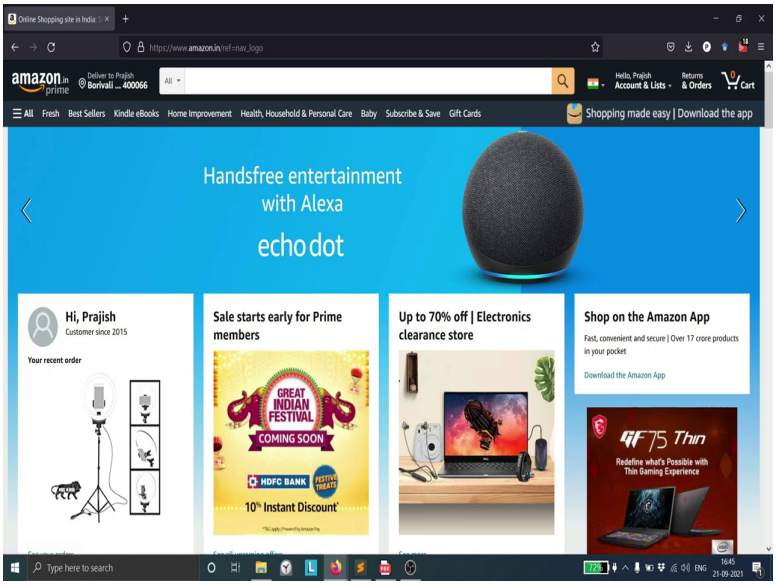


(Refer Slide Time: 00:59)

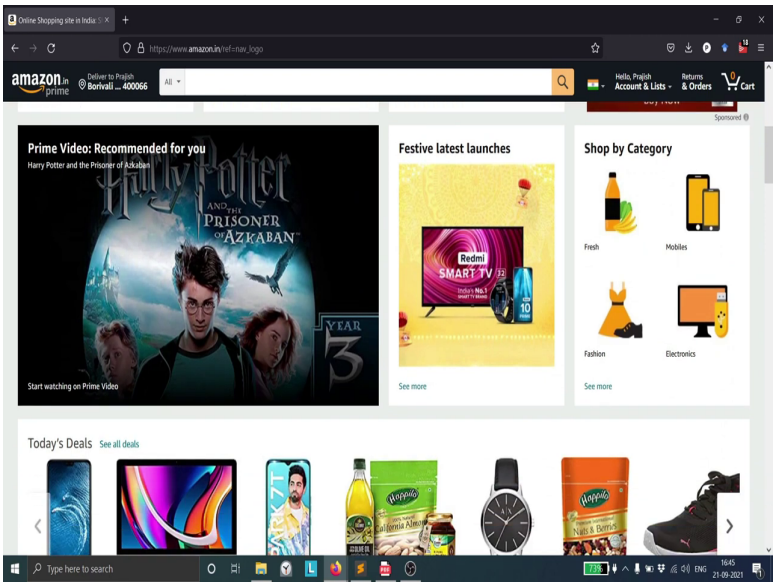


So, when we go to the Amazon home page, we see that their inventory gets updated based on current purchasing and seasonal trends and fluctuating customer demand. Hence, logistics and analytics play a large part in inventory management.

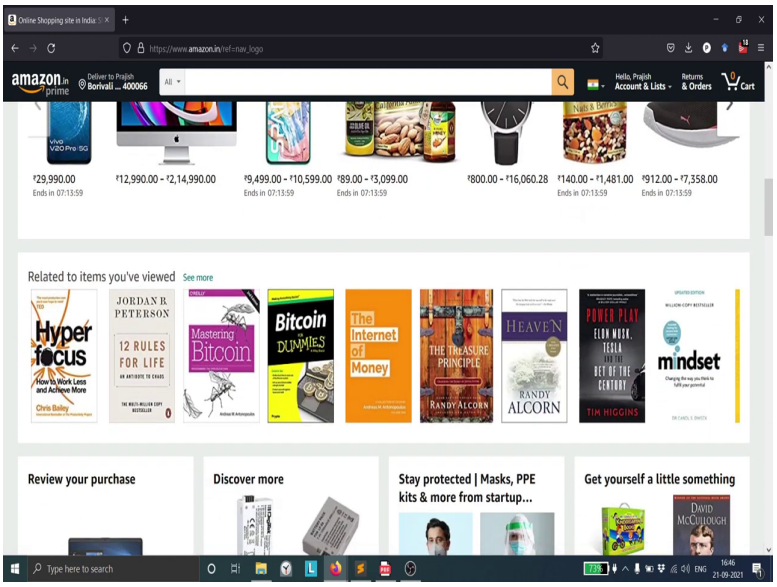
(Refer Slide Time: 01:11)



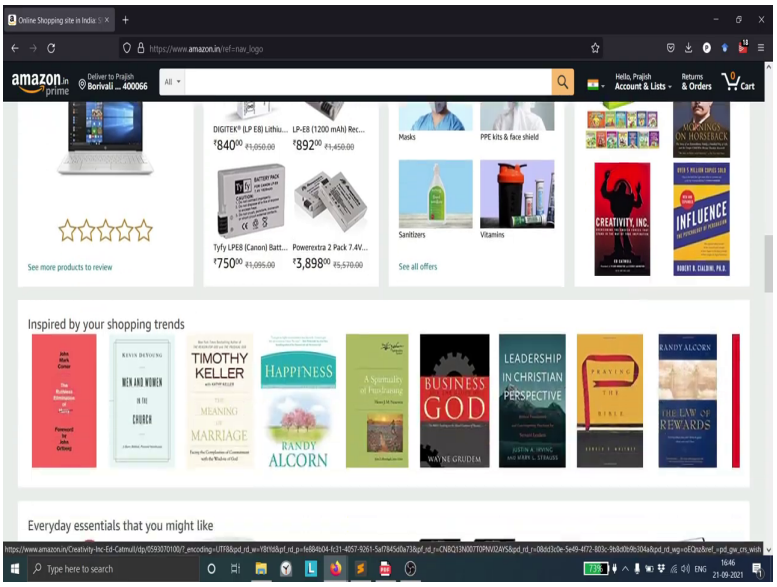
(Refer Slide Time: 01:17)



(Refer Slide Time: 01:24)



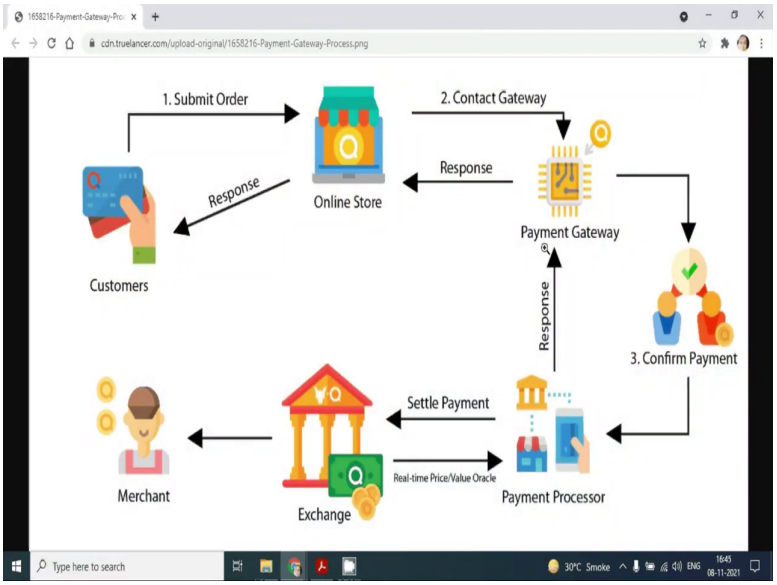
(Refer Slide Time: 01:26)



So, it is very interesting to see how my Amazon home page could be quite different from yours. Amazon customizes my home page based on the items I viewed as well as my shopping history.

Yes, that is true. My home page will be based on my shopping history, moreover, the ease of buying on the platform is also essential. The payment gateway is another module in the platform which offers ease of payment for both the buyer and the seller.

(Refer Slide Time: 01:50)



Payment gateway is a service that authorizes electronic payments like online banking, debit cards, cash cards, etcetera. Payment gateway acts as a middleman between the bank and the merchant's website or application. When a user wishes to make a payment, the Amazon website sends the encrypted card information to the payment gateway. Then the gateway confirms the validity of these details with the bank, and the required amount of money gets transferred from user's account to the Amazon account.

So, we looked at two components, the inventory management system and the payment gateway.

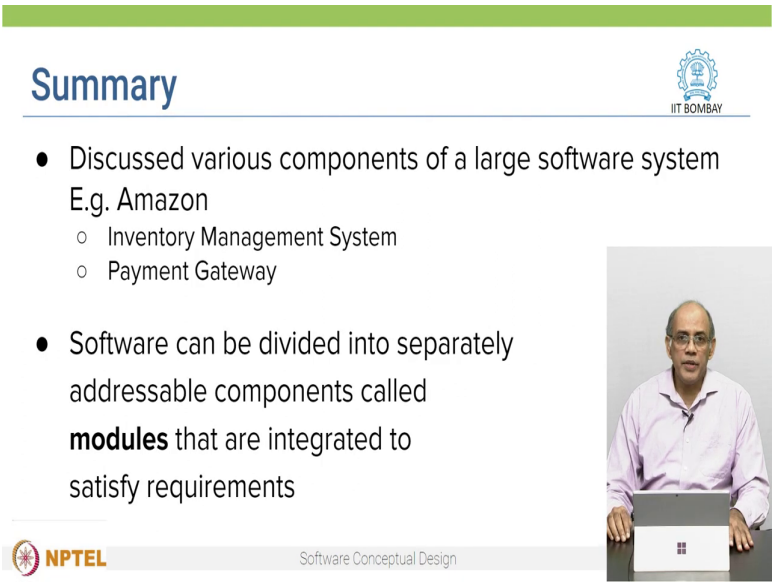
But I think there are many more components which make up the website.

Yes, that is true; so many components make up an e-commerce website such as Amazon.

(Refer Slide Time: 02:54)

So, here is the question for you, we discussed that an ecommerce website has several components, could you list a few of them? You can pause the video, write down your responses and then continue.

(Refer Slide Time: 03:17)



In this video, we discussed various components of a large software system for example, the inventory management system, the payment gateway and so on. So, it is important for us to think in terms of components. Software can typically be divided into several addressable components called modules, which are all integrated to satisfy the overall requirements.

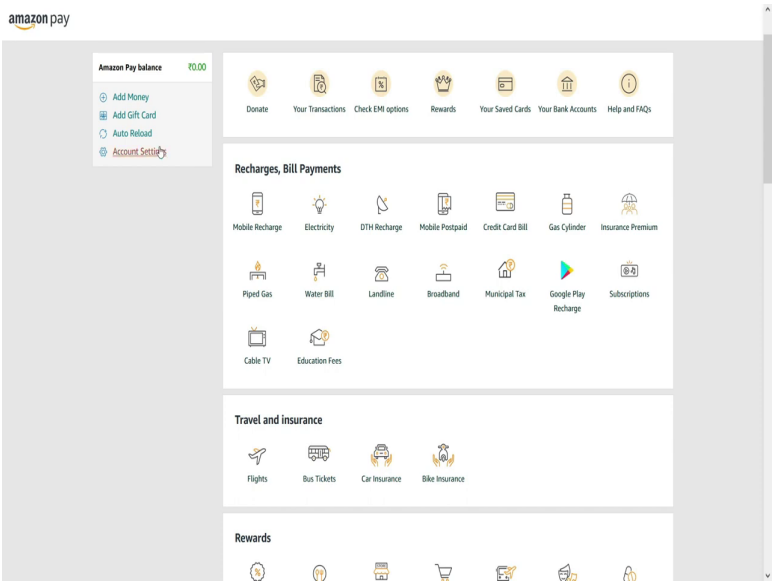
Software Conceptual Design
Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering
Indian Institute of Technology, Bombay

Lecture - 03
Requirement Specification

In the previous video, we explored the Amazon e-commerce web page and also identified some components.

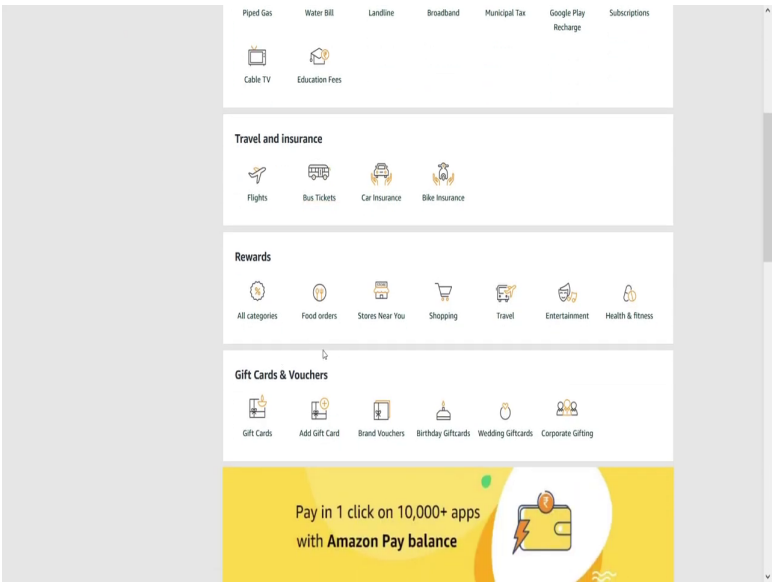
So now, I understood that there are different components to a software system, but it is also interesting to understand how Amazon implements each of these components. For example, how did they come up with the Amazon Pay feature?

(Refer Slide Time: 00:33)



Amazon Pay is a mobile wallet, a mobile wallet is a way to carry cash in digital format. You can link your credit card or debit card information, as well as your bank accounts, to a mobile wallet application, or you can transfer money online to your mobile wallet. Instead of using your debit or credit card to make purchases, you can pay with your smartphone which has this mobile wallet.

(Refer Slide Time: 01:12)



There are many categories like recharges, bill, payments, travel and insurance, rewards, gift vouchers and so on. A mobile wallet has features such as adding money and auto reload as well.

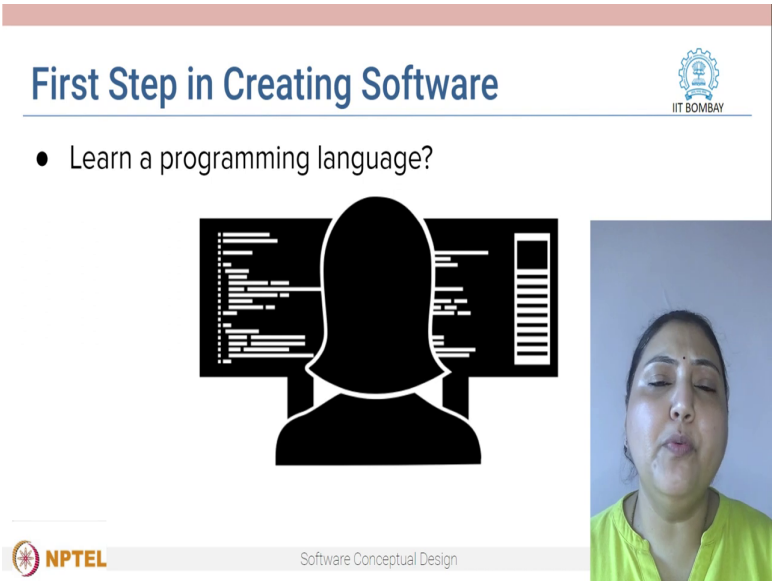
So, how did they start building the system? How did they come up with all these functionalities?

That is an important question. What do you think is a first step in creating a software?.

(Refer Slide Time: 01:41)

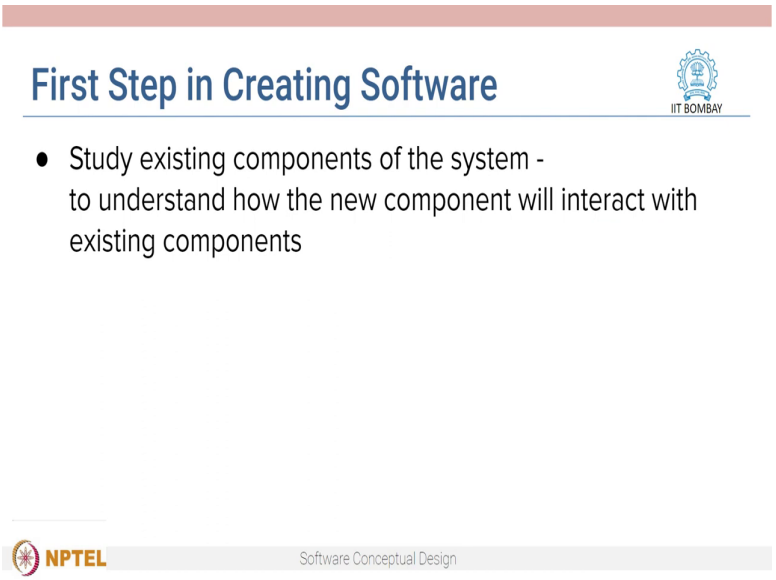
Pause and reflect on this question, write the answer in your notebook.

(Refer Slide Time: 01:57)



Some of you would have thought that we first need to learn a programming language. That is, in a way true, you need to write code for the software using a programming language, but even before you start thinking of the solution, you would need to think what is the problem that you have want to solve.

(Refer Slide Time: 02:21)



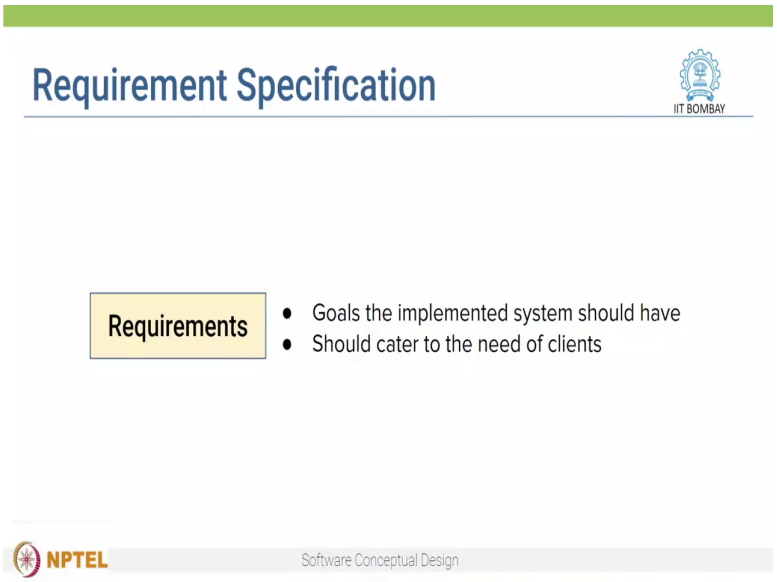
Another answer can be that we need to study existing systems and components for example, in Amazon, we saw the components inventory management and payment gateway. This can give us an idea of how a payment gateway will interact with the software.

(Refer Slide Time: 02:39)

That is right, maybe I can also look at other wallet systems like Paytm, PhonePe etcetera and see how they have implemented various features.

All these are valid answers. We need to first understand what is the problem that we want to solve, and based on the analysis of existing an similar systems, we need to come up with an explicit set of goals that the implementation should solve.

(Refer Slide Time: 03:10)



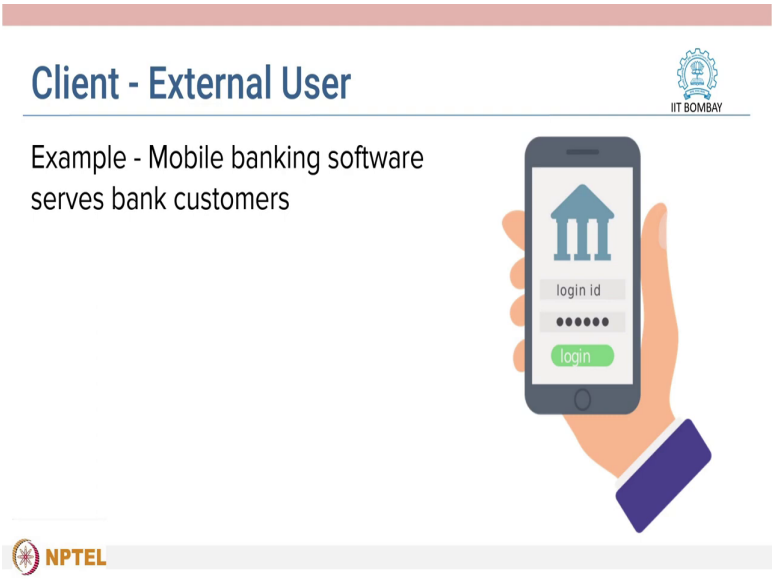
These goals are called the requirements of the software system, and this is usually the first step in the software development process.

Ultimately these software components will be used by someone, and these users are known as the clients of the software system. So, we need to ensure that these requirements cater to the needs of these users.

So, who is a client? Should they necessarily be end users of the system?

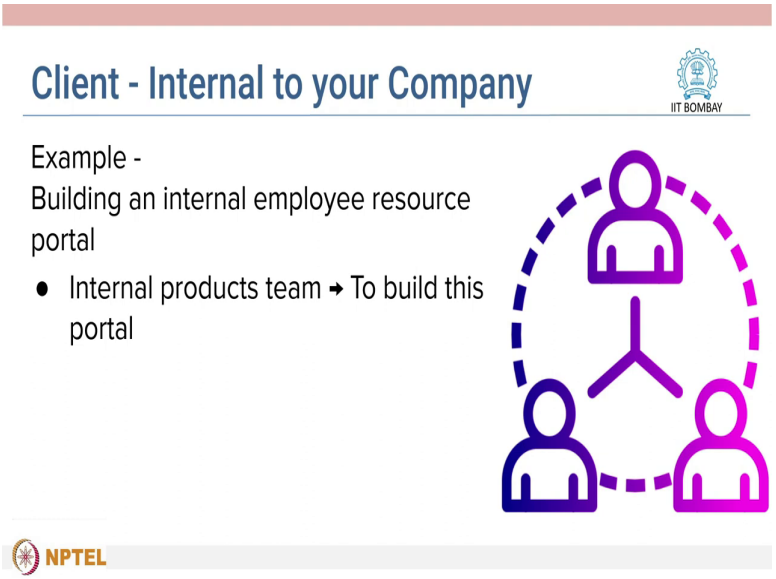
Well, not necessarily, they can be an external user, they can be internal to your company, and they can even be another software.

(Refer Slide Time: 03:56)



A client can be an external user. This is the most typical way to think about clients for example, a mobile banking software serves customers of the bank by providing various banking features such as checking the account balance and transferring money.

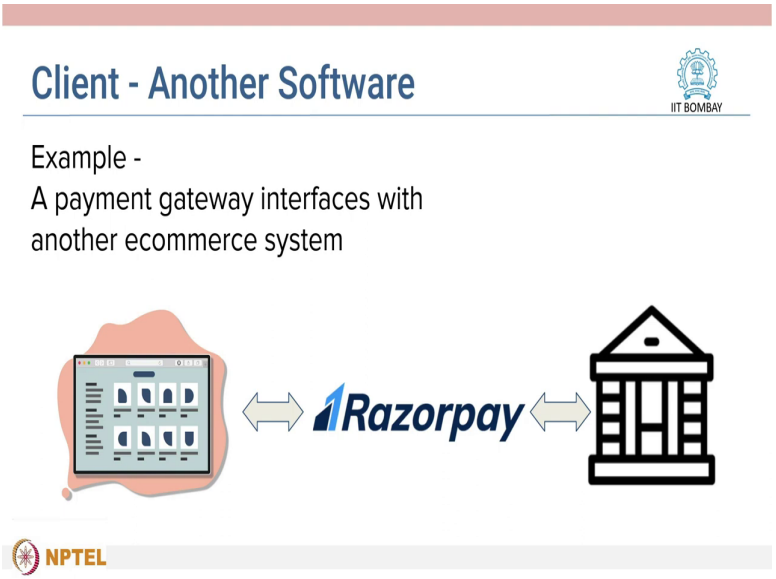
(Refer Slide Time: 04:18)



A client may be internal to your company: for example, a company might want to build an employee resource portal which contains information about various employees and teams in the company. To build such a system, the company might form an internal products team

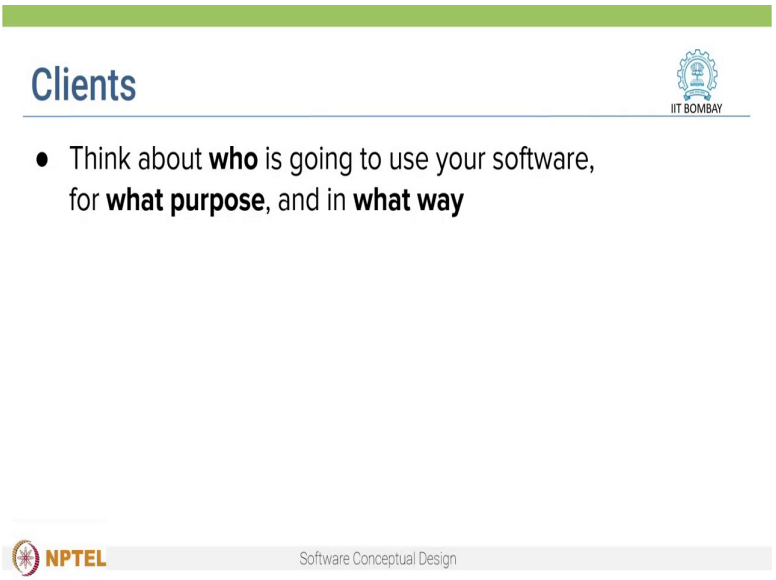
which communicates with other departments like human resources to understand the requirements.

(Refer Slide Time: 04:48)



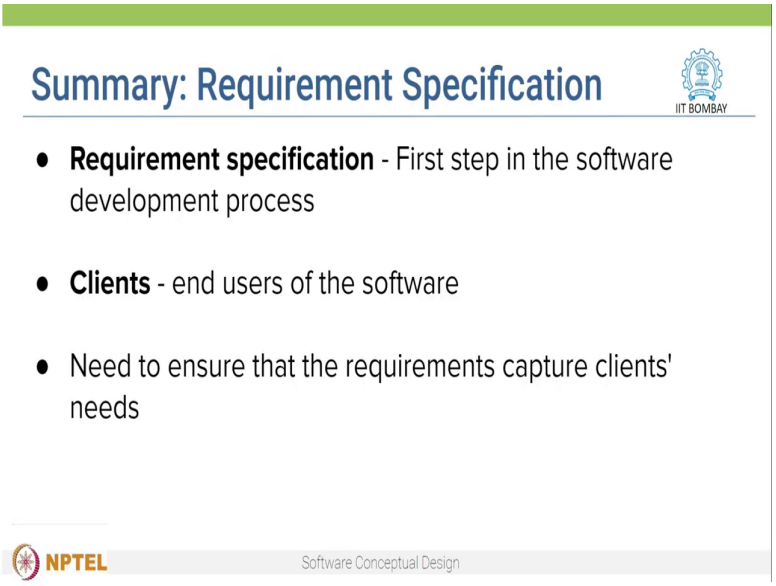
The client can be another software system as well: for example, a payment gateway like Razorpay interfaces with another e-commerce website or app to handle customer payments and refunds. Customer details are sent to the payment gateway by the e-commerce website. The payment gateway communicates with the customer's bank and sends the response back to the e-commerce website.

(Refer Slide Time: 05:22)



So, think about who is going to use your software, for what purpose and in what way. The persona of the intended user that is, the client, must be alive in your mind as you think about who you are going to create the software for.

(Refer Slide Time: 05:43)



So, to summarize the first step in software development process is to gather requirements. The requirements need to be gathered by the end users of the software, these users are usually called as clients, and we need to ensure that the requirements capture the needs of the client.

Software Conceptual Design
Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering
Indian Institute of Technology, Bombay

Lecture - 04
Software Design and Development

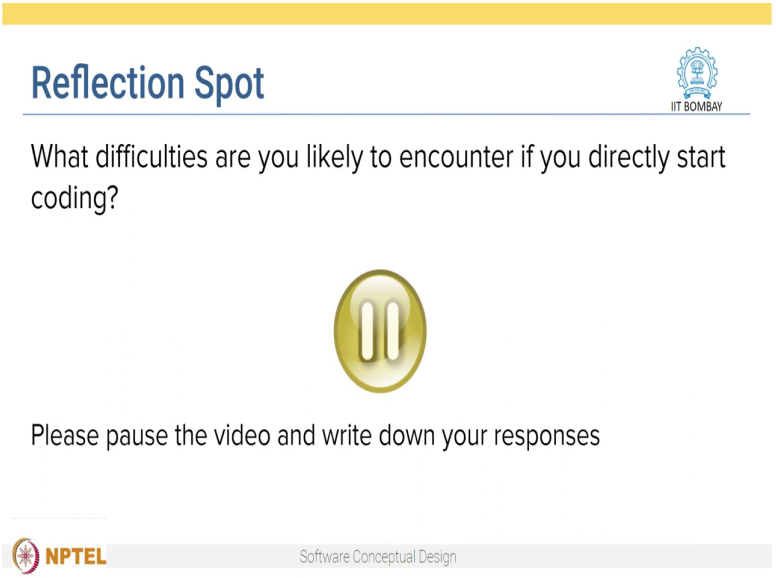
In the previous video, we talked about one of the first steps in the Software Development life cycle, that is identifying requirements from the clients Ok. So, now that we have come up with the requirements most of you would think that we can directly jump into writing code.

Yes, that is true, but there is an important step before that. Let me illustrate this with an example.

(Refer Slide Time: 00:31)

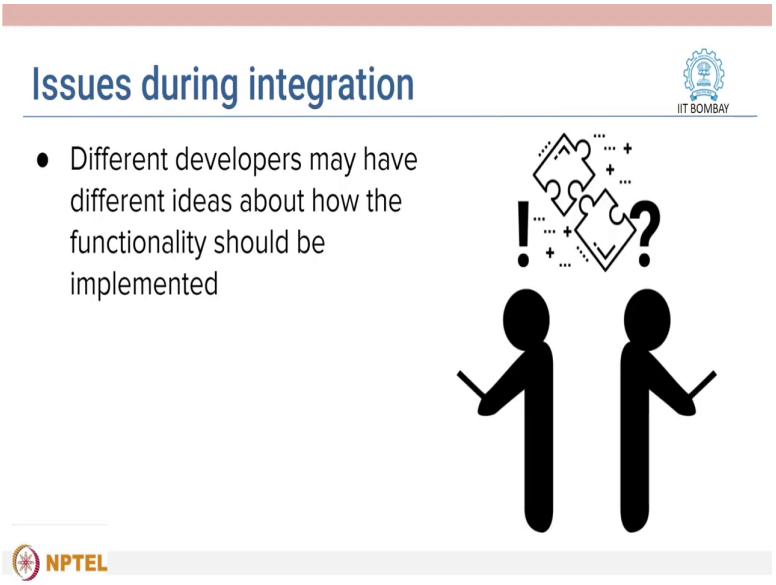
Imagine you are a small team going to implement the Amazon pay feature from the requirements. All of you are really excited to start implementing the feature and directly jump into writing the code. What difficulties or issues you are likely to encounter?

(Refer Slide Time: 00:52)



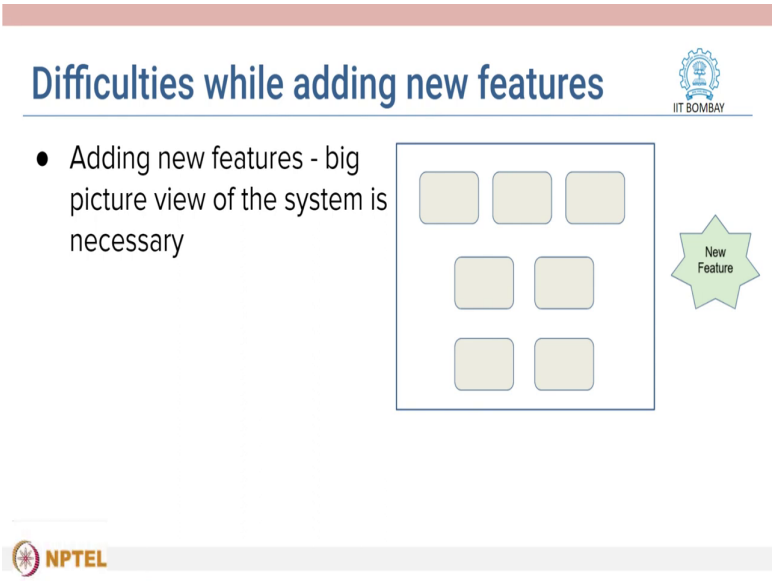
Pause the video and write your answer in the notebook.

(Refer Slide Time: 01:05)



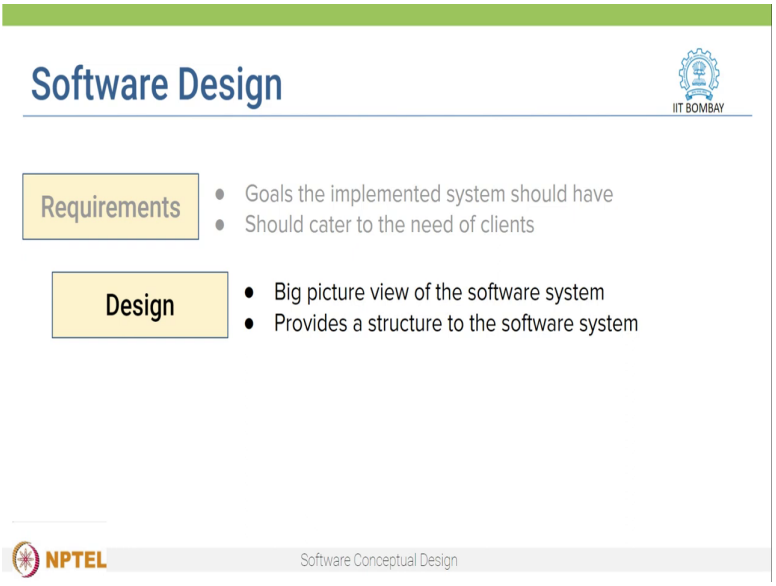
One issue can be that different developers have different ideas about how a functionality should be implemented. Even if you have discussed with your colleagues, you are most likely to make a change in your code. You might add an argument to an existing function, and your colleague who is using the same function to develop her components can face difficulties. This can be more problematic when different components created by different people have to interact with each other.

(Refer Slide Time: 01:37)



Oh, that can be a disaster. It will needlessly make it more complicated, and the product will not be delivered on time. Also, if I want to add a new feature later on it would help to have a big picture view of the system so that I can decide how this new feature can be correctly integrated into the existing system.

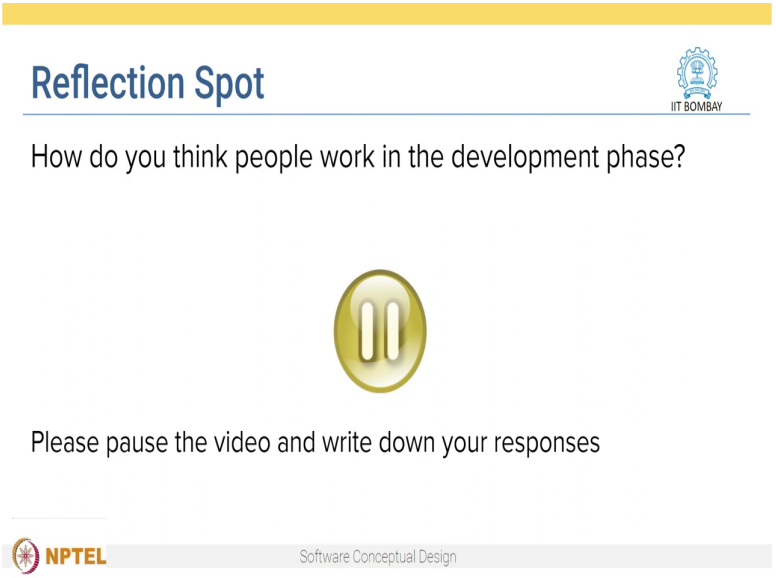
(Refer Slide Time: 02:02)



Yes, this big picture view of the system is created in the design phase of the development cycle. The design of a system is a way of organizing the code you will implement and

providing a structure to the software system. After coming up with the design, then we can go into implementing the system and write code.

(Refer Slide Time: 02:27)

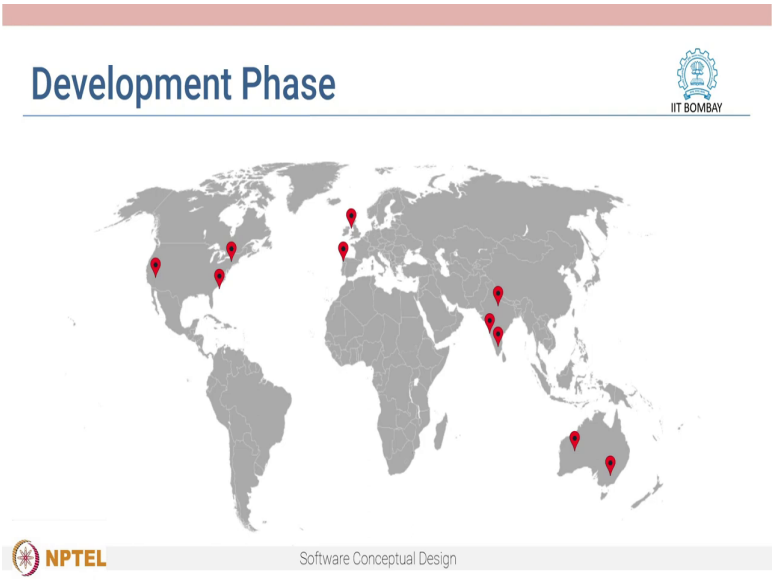


So, here is a reflection spot for you. What all comes to your mind when you see software developers writing code? How do you think people work in the development phase? You can pause this video and write your answers in your notebook.

(Refer Slide Time: 02:53)

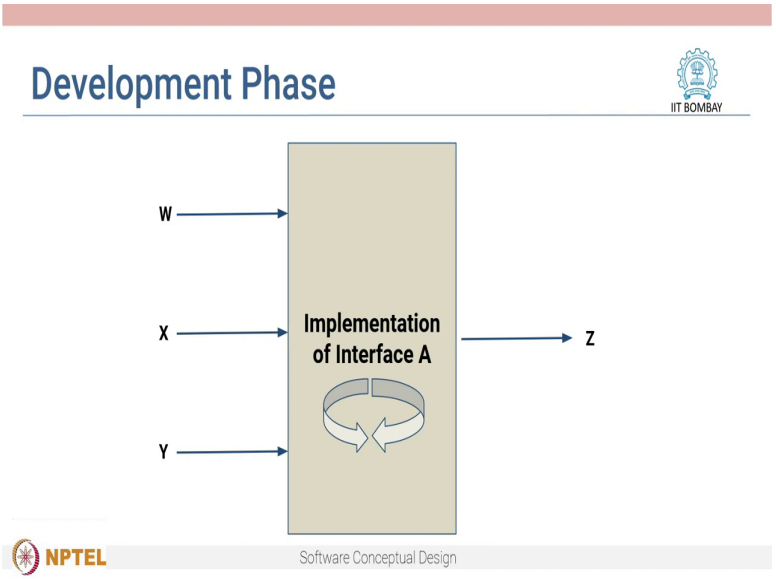
So, how do developers write code? When a feature is being implemented, multiple developers work together and write code for the feature. They use tools like GitHub to collaborate and write code.

(Refer Slide Time: 03:12)



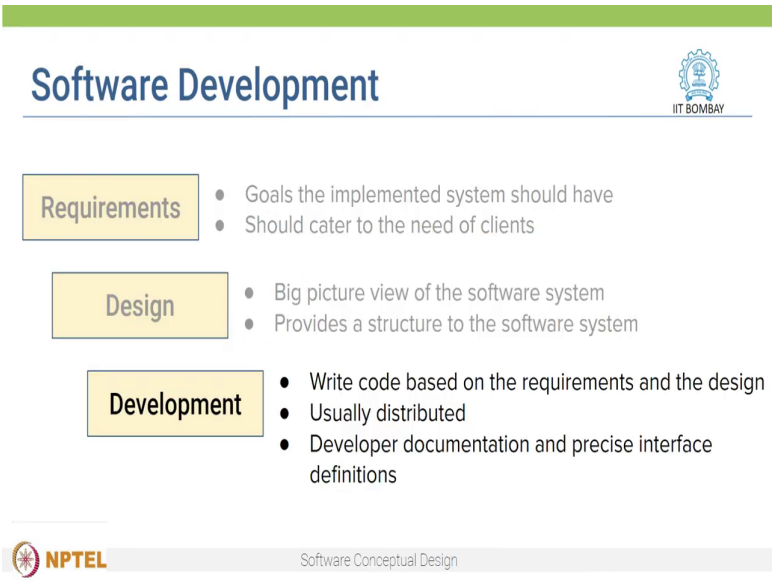
Hence, coding is often done in a distributed manner with developers working in different locations and even in different time zones. Hence, it is very important that everyone working on the code base has a consistent understanding about what the code does. That is the reason why developers write documentation for their code and write precise interface definitions. So, what is an interface?

(Refer Slide Time: 03:49)



An interface is nothing but the description of the actions that your functions can do without describing the implementation in detail. For example, in this case, interface A takes 3 inputs in the form of W, X and Y, and returns an output in the form of Z. Thus, the interface shows what requests are accepted in what format and the corresponding responses. The code being implemented by the interface can change as long as the interface remains the same.

(Refer Slide Time: 04:35)



In the development phase, software developers write code based on the requirements and the design of the software system. Development is usually distributed that is multiple people

write code which implement the required functionalities. To ensure that everyone in the development team is in agreement, developers document what their code does and write precise interface definitions.

Software Conceptual Design
Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering
Indian Institute of Technology, Bombay

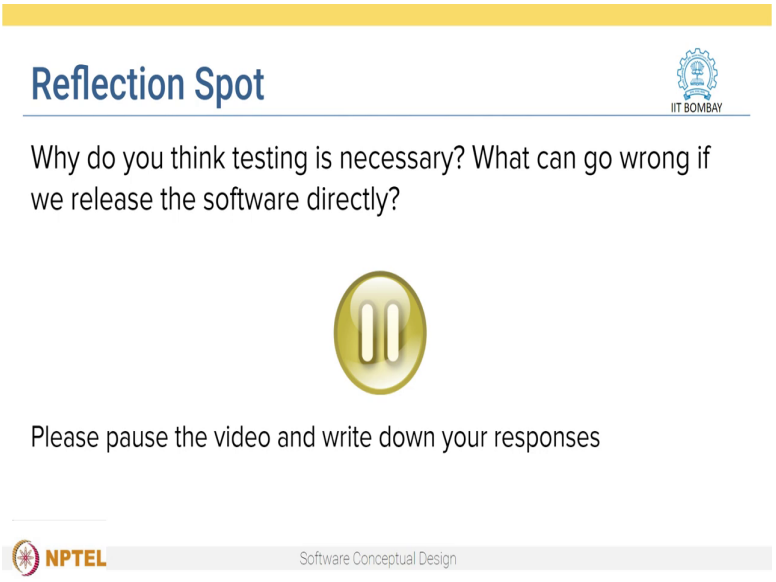
Lecture - 05
Software Testing and Maintenance

In the previous video, we talked about how the design and the development of our software systems are done. So, it looks like we are finally ready to release our software to our clients.

We might have an implementation ready, but we are not yet ready to release the software. In this video, we will talk about two other important phases in software development, testing and maintenance.

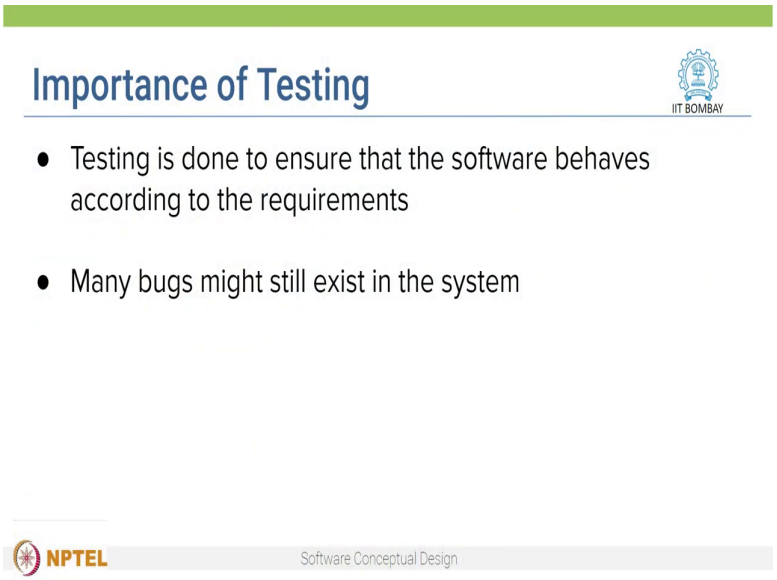
Testing: so we have to test our software before we release it to the clients. So, learners, here is a question for you.

(Refer Slide Time: 00:46)

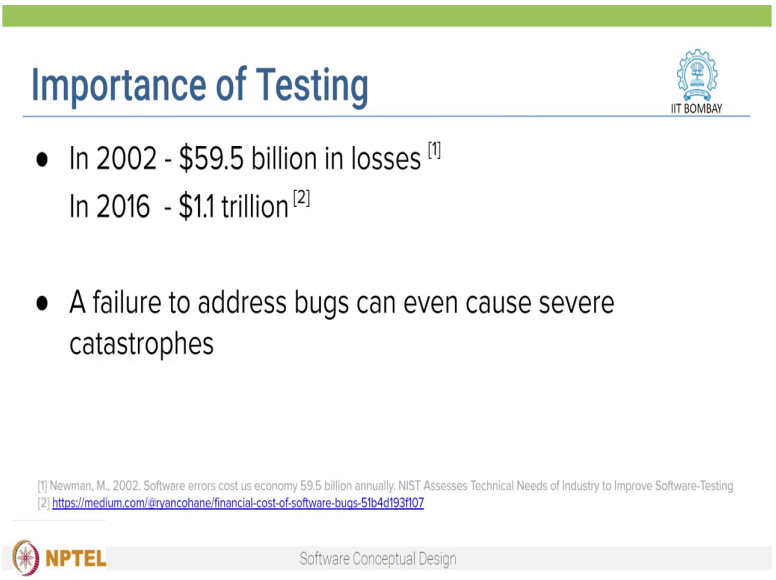


Why do you think testing is necessary? What can go wrong if we release the software directly? You can pause this video and write responses to this question in your notebook. After the software system is built, it's necessary that this software behaves according to the requirements; at this stage, testing of the system is done.

(Refer Slide Time: 01:23)



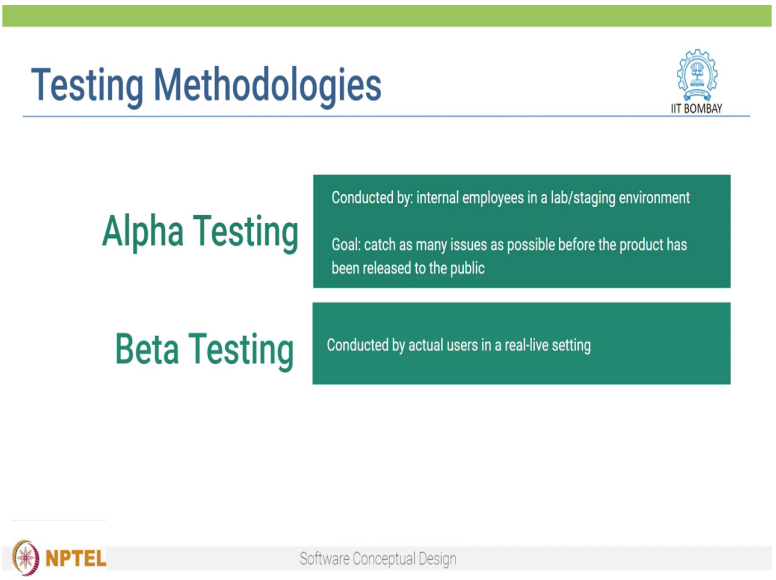
(Refer Slide Time: 01:30)



Testing is important because many bugs or defects can still exist in the system. In a study done almost 20 years back in 2002, it was found that software bugs caused the US economy around 60 billion in losses. In 2016, this number jumped to 1.1 trillion. A failure to address even simple bugs can cause severe catastrophes.

You can find more details about this in the additional resources, do check them out. So, how is testing actually done? There are several testing methodologies.

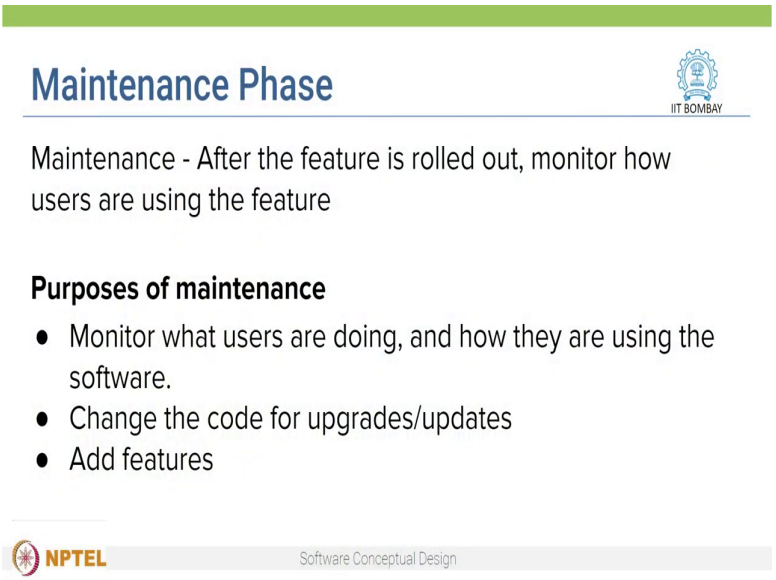
(Refer Slide Time: 02:10)



For example, Alpha Testing is done by internal employees of a company in a lab or a staging environment. The goal of the alpha test is to catch as many issues as possible before the product has been finally released to the public.

Then there is Beta Testing: beta tests are conducted by actual users in a real life setting. We have additional resources which give more information regarding alpha and beta testing. After adequate testing, the feature is rolled out, but it is necessary to actually monitor how users are using the feature and keep examining the difficulties and errors they encounter, this is done during the maintenance phase.

(Refer Slide Time: 03:01)



The purpose of maintenance phase can be to monitor what users are doing and how they are using the software. It can also be to change the code for any upgrades and updates. You may also want to add features to the existing software. Let me illustrate this with an example; let us go back to the Amazon Pay example. Even after releasing this feature what difficulties do you think the users might face?

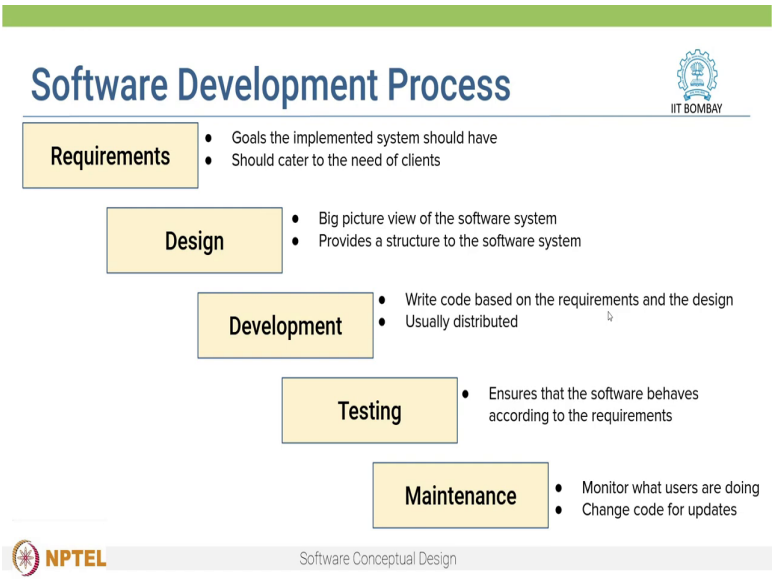
Ok, so there can be issues like some conditions might have been missed resulting in failures, or it can even be simple UI issues in a very specific browser.

Yes, all these issues can arise, the team identifies these issues in the maintenance phase and continues to fix such issues.

Software Conceptual Design
Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering Indian Institute of Technology, Bombay

Lecture - 06
Software Development Models

(Refer Slide Time: 00:05)

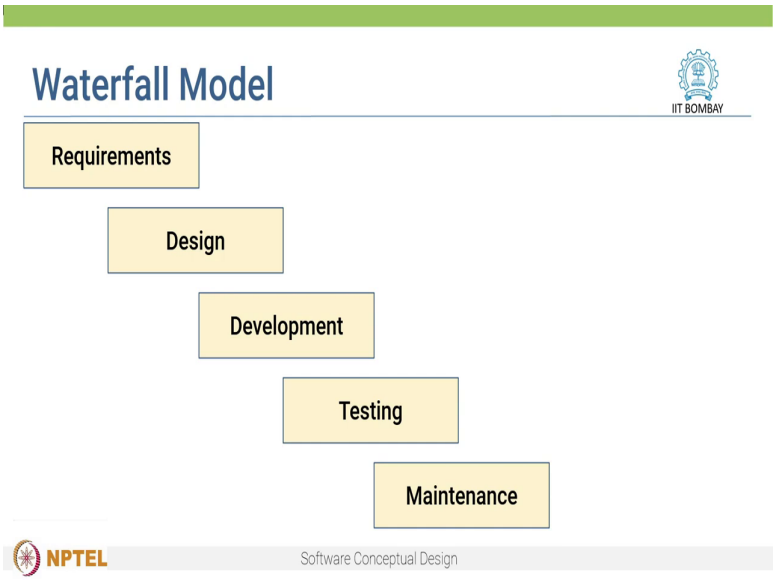


In the previous video lectures, we spoke about the Software Development process and the various phases in this process. We looked at requirements, which are the goals which the implemented system should have, and these requirements should cater to the need of the clients. Once we have gathered the requirements from the clients, we then come up with the design of the software system.

The design is the big-picture view of the software system and provides a structure to the entire software. Once the design is ready, we then write code in the development phase.

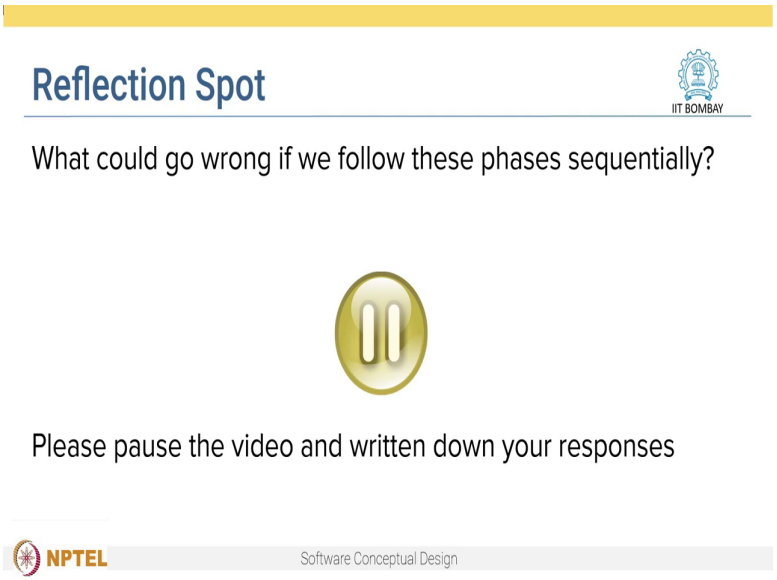
After we have written code, we rigorously test our system to ensure that it behaves according to the requirements. We then release the software to the clients, and in the maintenance phase, we monitor what the users are doing and change the code for updates and upgrades.

(Refer Slide Time: 01:24)



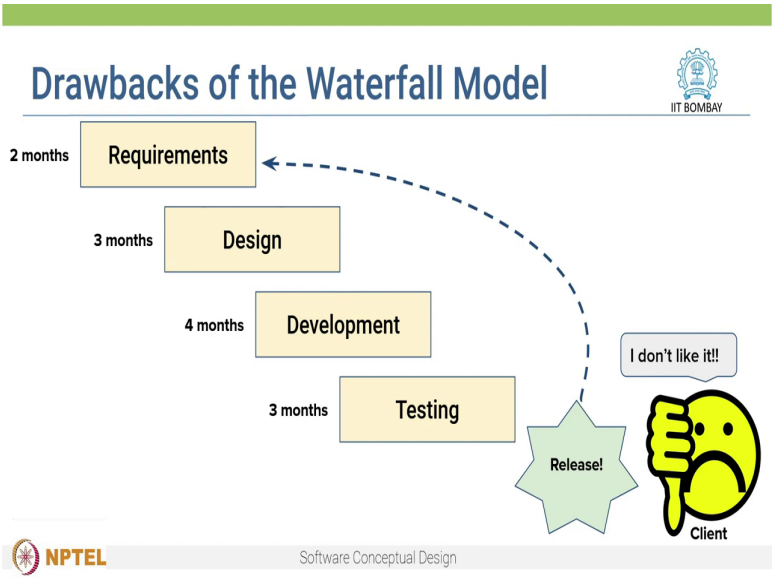
This process is commonly known as the waterfall model, where we start from the requirements, move to the design, the development, testing and finally, to the maintenance phase. We see that each of these phases occurs sequentially. Learners, I want you to now pause and reflect on this question. What can go wrong if we follow this process sequentially while creating a software system?

(Refer Slide Time: 02:04)



Please pause the video and write down your responses before you proceed.

(Refer Slide Time: 02:29)

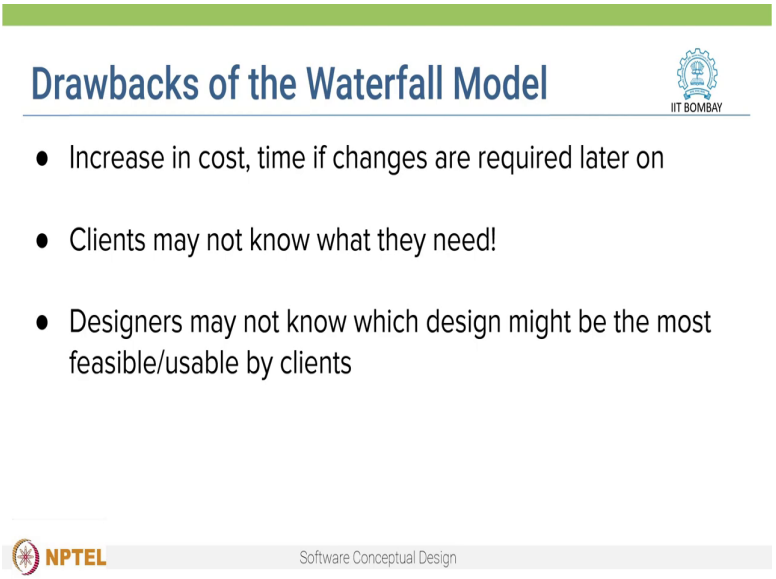


So, what are the drawbacks of this waterfall model? So, let us say, you are coming up with a software system, you have gathered the requirements and let us say it took you 2 months to do that. After that, you came up with a design which took around 3 months for your team. Now that you have the requirements and the design, you start developing the system. You write code for it, and this takes around 4 months.

Once the system is developed, then you rigorously test it identify bugs and defects and fix it. This can take, let us say, around 3 months. Now finally, your software is set for release, and you give it to your clients. Now, the clients are seeing your software for the first time, and unfortunately, they do not like it. It can be a particular feature which they do not like, it might be the entire user interface and so on.

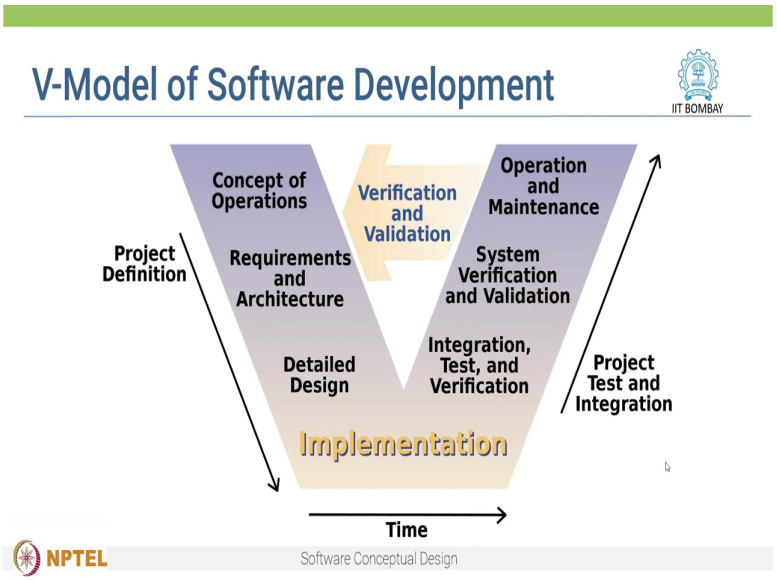
So, your software team has spent around 12 months building this system, and now you have to go back to the drawing board and refine your requirements based on the current needs of the clients.

(Refer Slide Time: 04:12)



So, what are the drawbacks of this waterfall model? As we saw, if changes are required later on, this leads to an increase in cost as well as time. We also saw that clients themselves may not know what they need in a software system, and their requirements can change over time. Third, the designers themselves may not know which design might be the most feasible or the usable by clients.

(Refer Slide Time: 04:55)

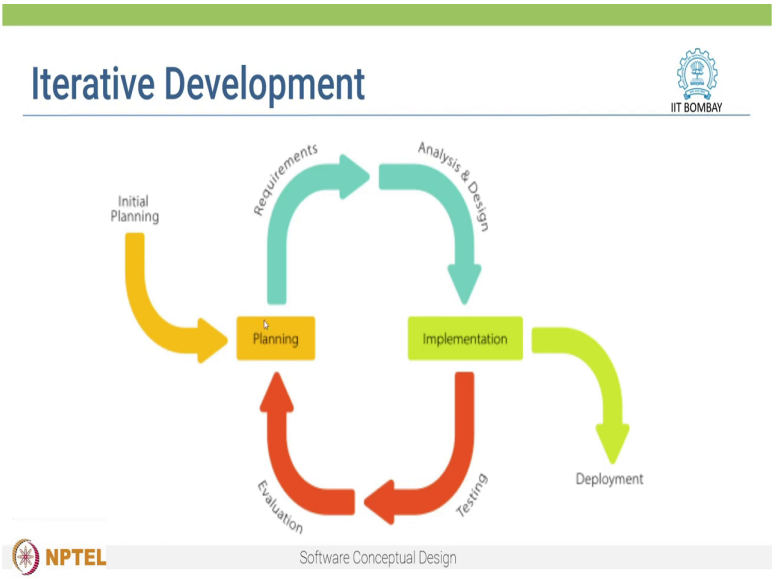


A variation of the waterfall model is the V-model of software development. As we can see in the figure, instead of a sequential process, the process steps are bent upwards after the

implementation phase to form the typical V shape. Like the waterfall model, we start from the requirements and come up with the design and implement the software system.

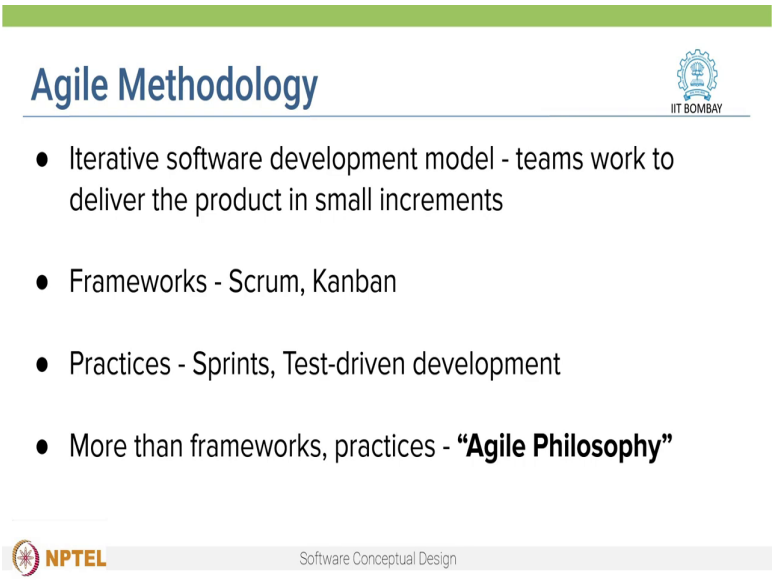
After that, we test and verify individual modules and validate it with the design. After validation of the individual modules, then we then verify and validate the entire system and compare it and validate it with the architecture.

(Refer Slide Time: 05:57)



To elevate the difficulties of the waterfall model and the V model, Iterative software development is a model which is being used widely today. In this model, software projects are built using multiple iterations. For example, in the first iteration, the development team plans and builds features for key requirements they deploy it and show it to the client and get their feedback. Even if the client wants changes, the team can quickly refine it in the next iteration. This process of development and refinement continues until the entire product is developed.

(Refer Slide Time: 07:00)



A popular iterative software development model is the Agile Methodology. In this model, teams work to deliver the product in small increments. Instead of waiting till the end to deliver the entire product, teams develop prototypes of key features and quickly release the prototype. There are several frameworks like Scrum and Kanban and practices such as sprints and test driven development.

The Agile software development is an umbrella term for a set of frameworks and practices based on the values and principles of the agile philosophy. You can learn more about the agile software development philosophy and the frameworks like Scrum and Kanban in the extra resources provided to you this week.

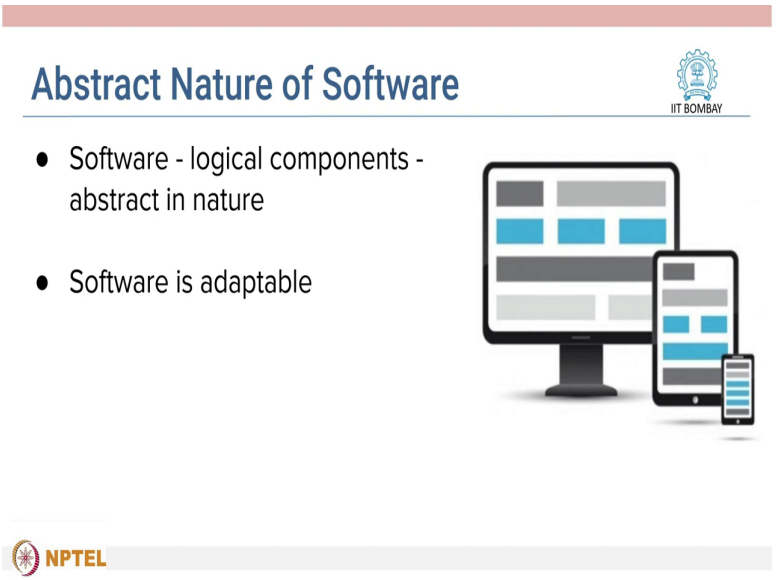
Software Conceptual Design
Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering
Indian Institute of Technology, Bombay

Lecture - 07
Comparing and Contrasting Software Development

In the previous videos of this week, we described the different phases of the software development process and various development models like V-Model and Agile. Now, let us compare and contrast the software development process with other common development processes. For example, let us consider the construction of a building. How similar or different is creating a software from construction of a building?

So, when we construct a building, a blueprint is created first, and then the construction of the actual building starts, also, the construction of the building involves a lot of tangible components. However, software, most of the time, is made up of logical components, which are very abstract in nature.

(Refer Slide Time: 01:00)



Software, most of the time, is made up of logical components which are abstract in nature; due to this abstract nature, it is possible to quickly adapt software based on the needs or

requirements of users. Hence, software is also adaptable and can be changed by anyone on the team.

(Refer Slide Time: 01:27)

That also brings us to the next difference that a building is static, whereas a software keeps changing. In the screen, you can see a 2000s music player Winamp; however, now we have applications like Saavn, Gaana.

That was interesting. So, we see that there are key differences in terms of the abstractness and the evolution of software. But there are also similarities in terms of the processes, such as requirements gathering, undertaking site visits or understanding the end users, building models of scale, etcetera. Before we end this week, here is a thought experiment for all of you, how is software design similar or different from building a car?

Software Conceptual Design

Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering
Indian Institute of Technology, Bombay

Lecture - 08
Creating a Software Conceptual Design

In this week, we will do a quick recap of what we learnt in the previous week. Namely, the components and modules in a software application this week. We will explore methods to begin Creating Software Conceptual Design ok. We will also have many activities for you to do.

Software Conceptual Design

Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering Indian Institute of Technology, Bombay

Lecture - 09
Recap of Week 1

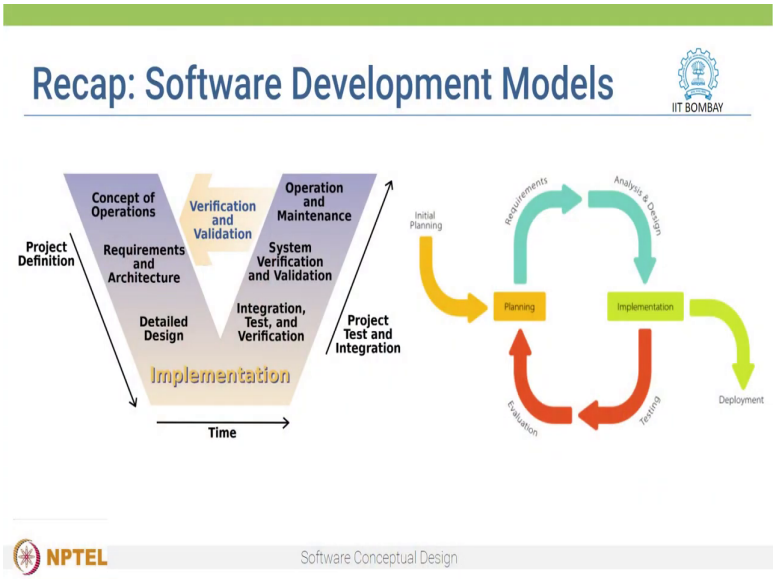
So, learners, last week we discussed how software is made up of several components and the process of creating a software. We looked at the example of the Amazon Pay feature. So, creation of such software systems begins with thinking about who is going to use the software? For what purpose and in what way?

Yes, after gathering the requirements from the end user, we need to create a design when the big picture of the software is developed. The actual creation of code happens after this, which is often referred as the development phase. Following this is the testing and maintenance phase.

But how do these activities happen? The nature of iteration in these activities are all captured in the software development process model. So, learners, could you recollect some of the software development process models that were discussed, pause this video and write your answers in the notebook.

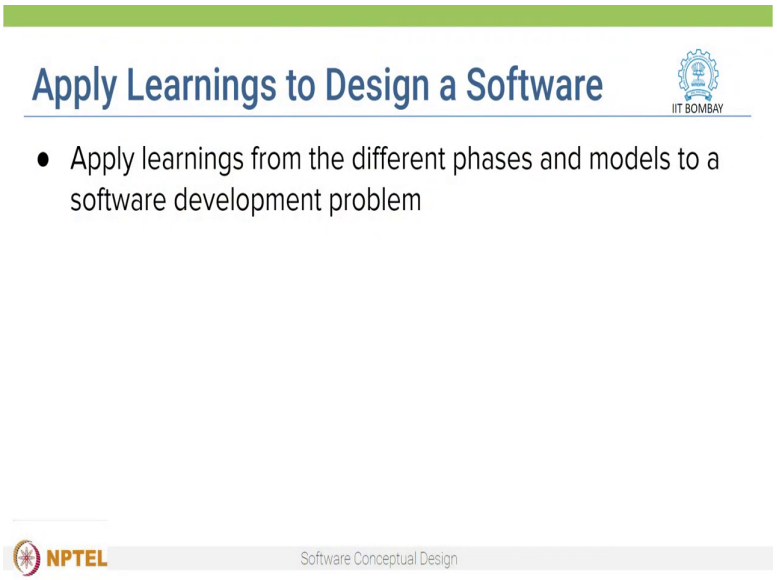
(Refer Slide Time: 01:14)

(Refer Slide Time: 01:28)



So, we saw that there are different software development models we saw the V model, then there is the waterfall model and the agile process models, to name a few. So, the development team chooses a model that is appropriate for them. However, most of the times they adapt the model based on their own processes and also on the nature of the software that the team is developing.

(Refer Slide Time: 02:00)

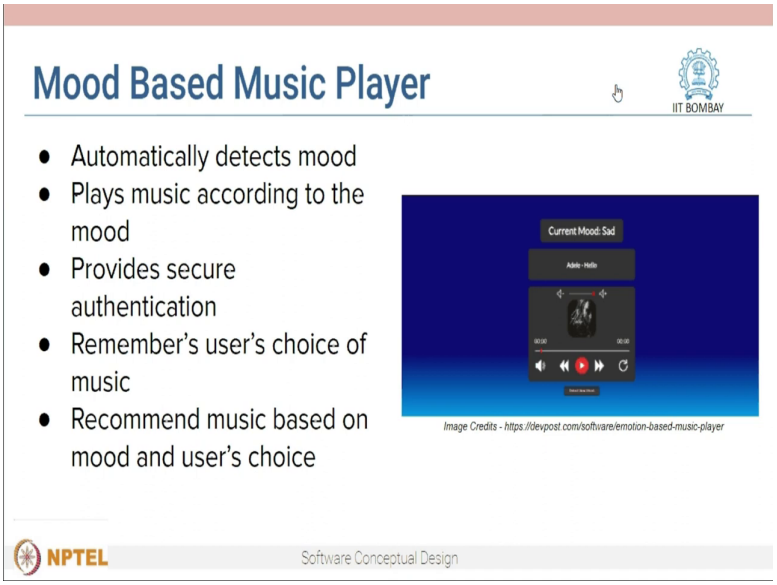


Ok, so let us try to apply the learnings from the different phases and models to an actual software design problem. The details of the problem provided in the LxI.

Software Conceptual Design
Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering Indian Institute of Technology, Bombay
Lecture - 10
How to Start Creating a Software Conceptual Design

In the previous LxI, we had asked you and how you would go about designing a mood based music player. Mood based music player that sounds really interesting, but I am wondering what does it do?

(Refer Slide Time: 00:20)

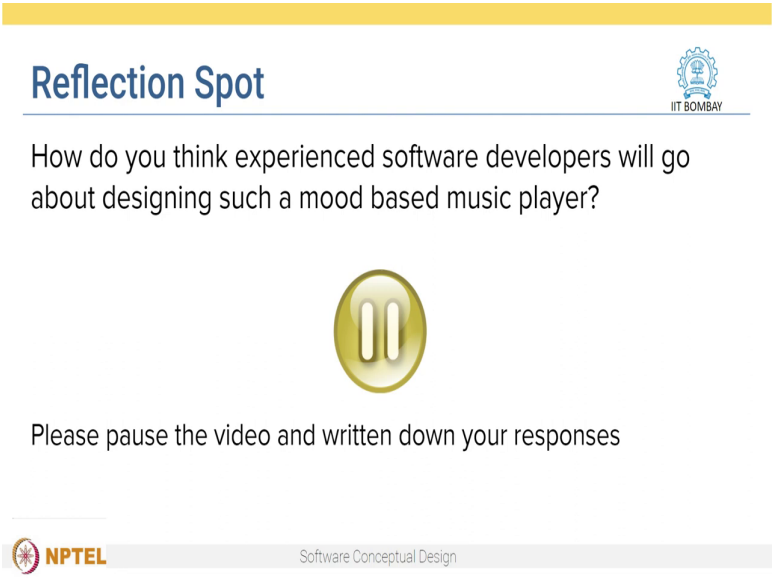


A mood based music player automatically detects mood, plays music according to the mood, provide secured authentication, remembers users choice of music, recommends music based on the mood as well as users' choice.

This sounds very cool, but I am still wondering how should I start designing such a mood based music player?

Yeah, it is an interesting question; last week we learned about the various phases and models in a typical software development; however, they only provide a framework, but do not really guide you on how to think while creating a software.

(Refer Slide Time: 01:29)



Right, so I was thinking, how will an experienced software designer go about designing such a mood based music player?

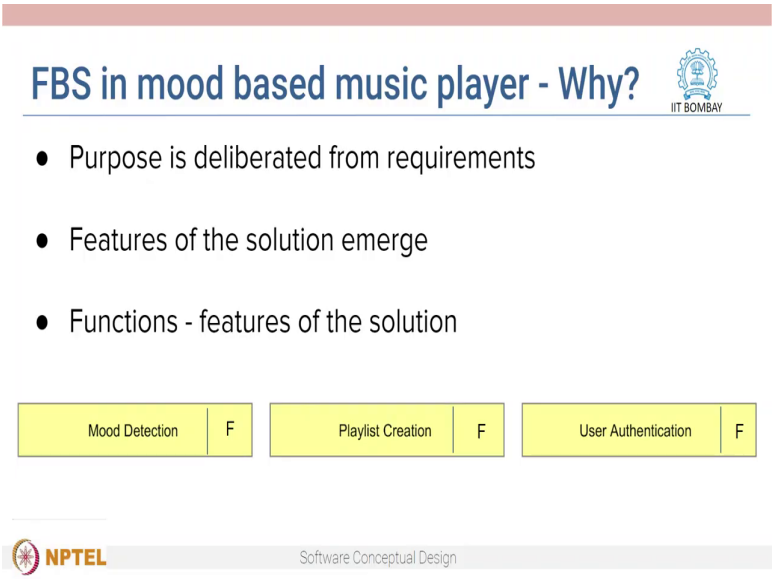
Yeah that is a good thought. So, learners, here is a reflection spot for you. How do you think experienced software developers will go about creating a mood based music player? Pause the video and write your answers in the notebook.

So I think that experienced developers would have worked on similar systems in the past, and I think that they will refer to these systems that they created as reference points.

Yes, that is true they also have an integrated view of the system and tend to continuously think about the why, what and how of the design.

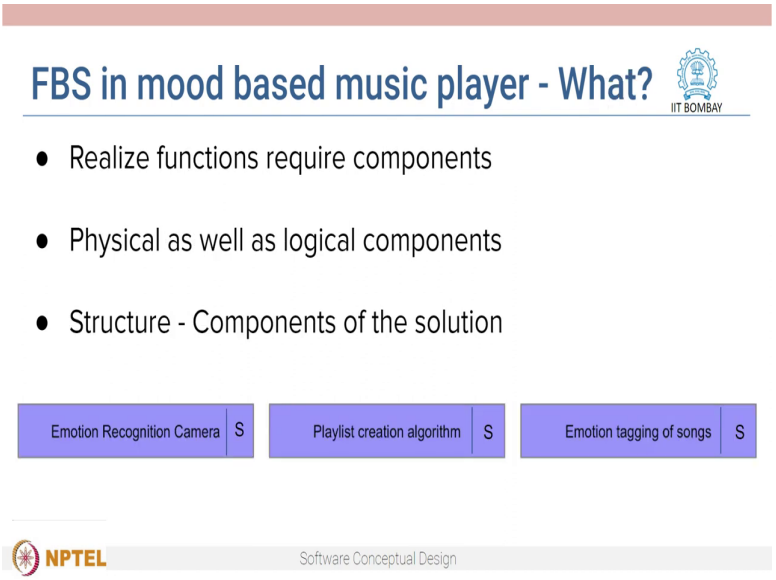
The why, the what and the how; can you explain it in some more detail?

(Refer Slide Time: 02:22)



The why; the purpose of software design is often deliberated from the requirements after this deliberation, the features of this solution are envisaged they often map to the requirements. The features of a design solution are called as functions. For the mood based music player, it could be more detection playlist creation user authentication, to name a few.

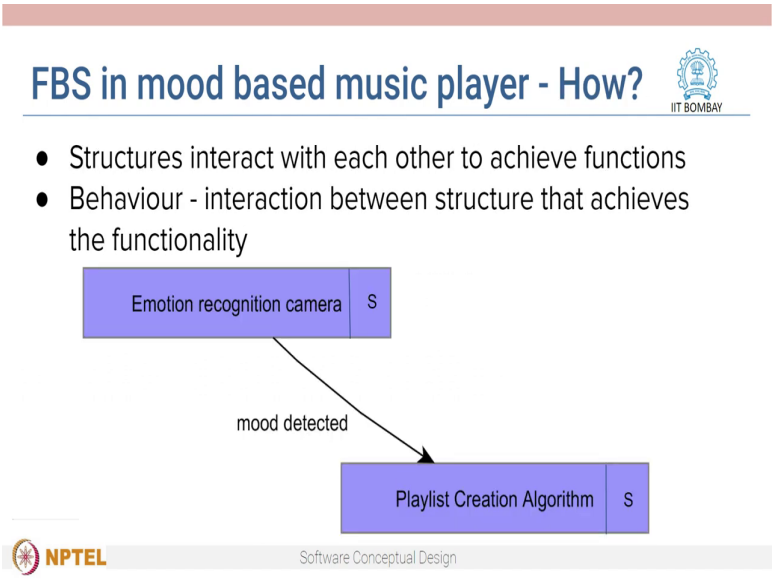
(Refer Slide Time: 02:53)



The what: to realize the functions in the solution, physical as well as logical components are required. These components are referred to as structures. In the mood based music player,

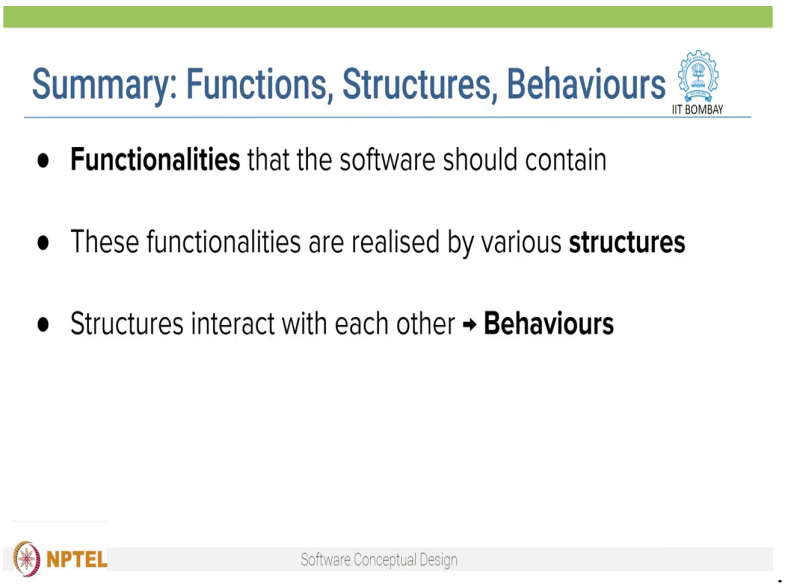
there could be an emotion recognition camera, some algorithms to automatically generate playlist, emotion tagging of songs to name of you.

(Refer Slide Time: 03:20)



The how: the structures interact with each other to achieve the function. For example, the emotion captured it recognizes the user emotion then provides it to the playlist generator which creates a playlist based on the mood. Behaviour refers to the interaction between the structures that result in achieving the purpose of the design.

(Refer Slide Time: 03:48)



So, now, I think I am understanding what you mean. So, to start creating a software design, we could think of the functionalities that this design will contain and these functionalities are realised by various structures that interact with each other and these interactions are known as behaviours.

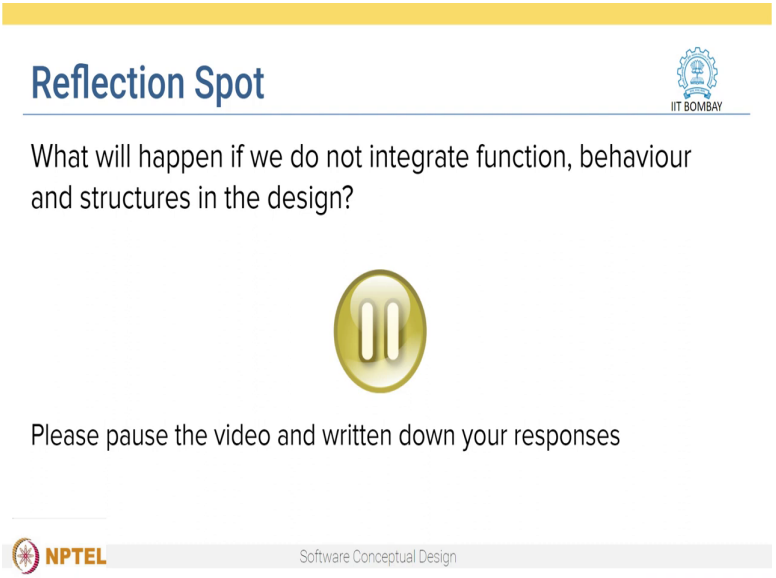
It is a good summary; indeed, we would we have only started thinking about function structures and behaviours individually we would need to integrate them together and let us see how to do this in the next LeD.

Software Conceptual Design
Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering
Indian Institute of Technology, Bombay

Lecture - 11
Using the Function-Behaviour-Structure Design Framework

So, last time we spoke about Function Behaviour and Structure. We discussed that functionalities are realized by various structures that interact with each other these interactions are termed as behaviours.

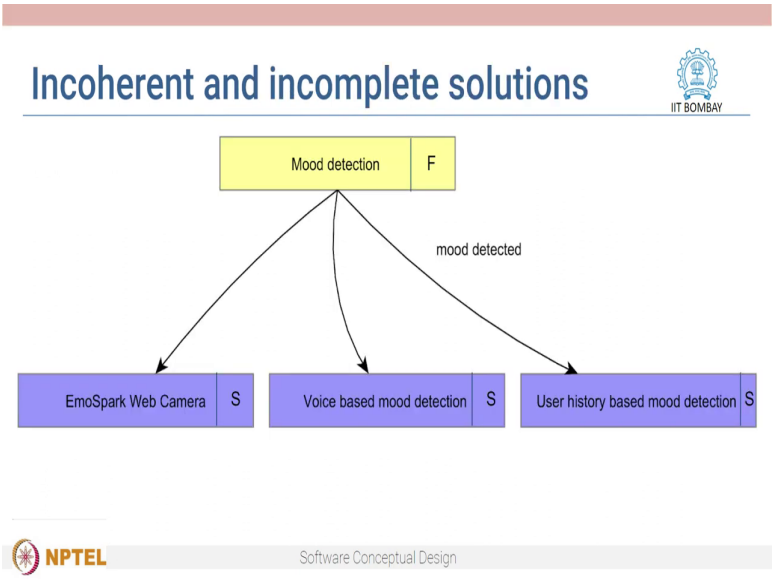
(Refer Slide Time: 00:38)



Yes, we also looked at the function, behaviour and structure elements of a mood based music player, but you also mentioned that we need to integrate the function, behaviour and structure. Why do we need to do that?

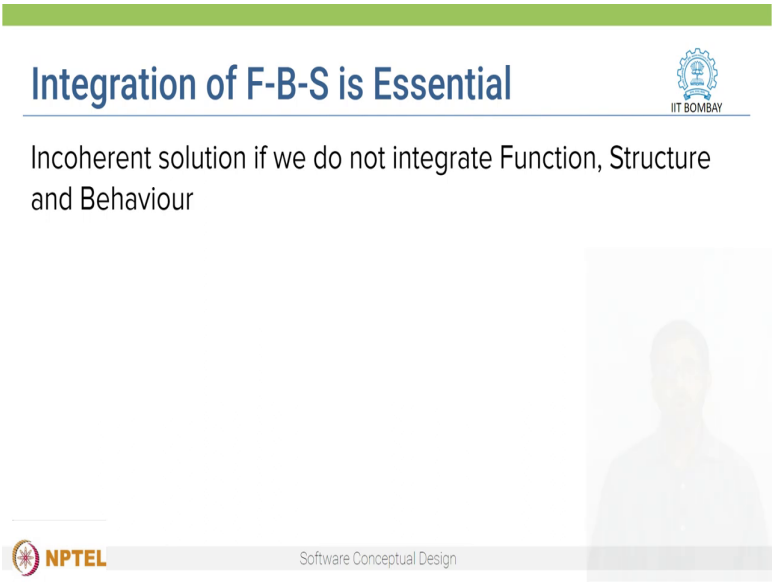
Yes, that is an important question. So, learners, what will happen if you do not integrate function, behaviour and structure? Pause the video write your answers in the notebook, and then continue.

(Refer Slide Time: 01:03)



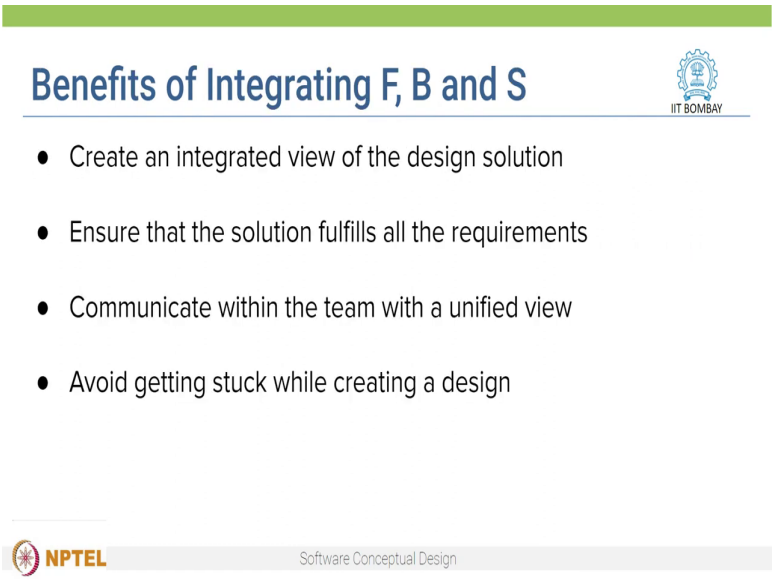
So, if you do not integrate function, behaviour and structure the design would be many solutions and incoherent. For example, mood based music player, if you had only focused on the functionality of mood detection, we would have created many solutions for achieving this functionality. This is not decidable.

(Refer Slide Time: 01:32)



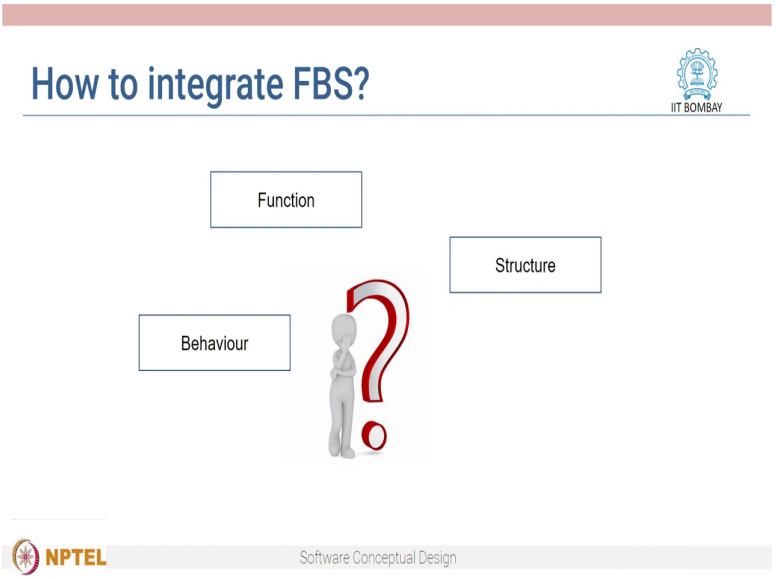
Ok. So we will have an incoherent solution if we do not integrate the function, structure and behaviour, but what are the benefits of integrating them?

(Refer Slide Time: 01:45)



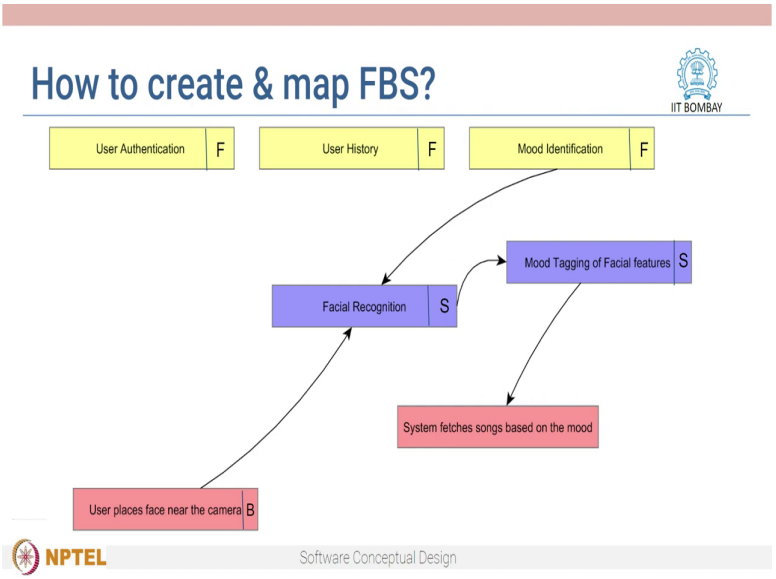
While creating conceptual design, it is useful to integrate function, structure and behaviour for the following reasons; it helps us create an integrated view of the design solution, we also can ensure that the solution fulfils all the requirements, this unified view can be used to communicate within the team and the various stakeholders, it is also a useful technique to avoid getting stuck while creating a design.

(Refer Slide Time: 02:17)



Ok. So, now, I have this individual FB and S. Now, how do I go about integrating the function structure and behavior?

(Refer Slide Time: 02:29)



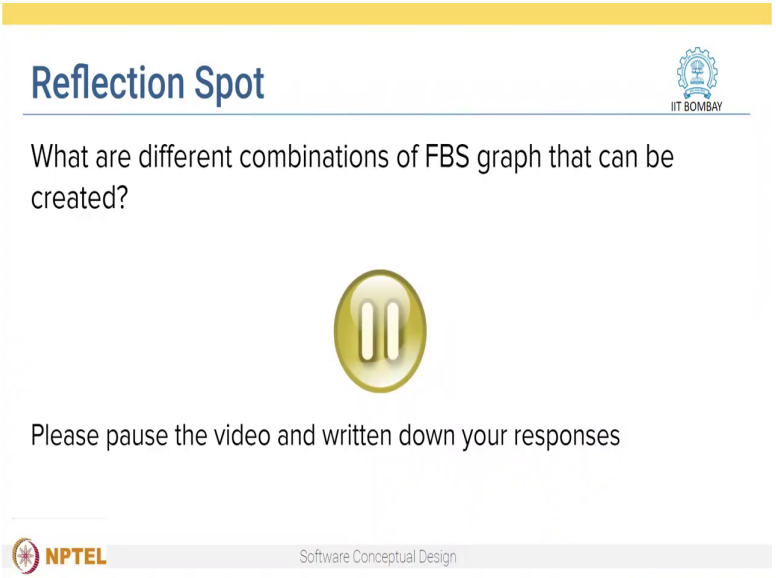
One way you could do is to start from known solution. For example, we know that there are cameras that can deduct mood automatically. These could be the structures in the designed solution. But we also need to evaluate if these structures are applicable in the current problem and adaptive. In the next step, we can start thinking in terms of end user as well as a system to generate behaviours.

For example, a user would place the camera to automatically start playing songs the system could fetch song lists based on a particular mood and so on and so forth. In the next step, we map all the structures that work together when a behaviour happens. In this way, we identify structures, behaviours and map them. The next step would be to identify how the structures and behaviours satisfy a particular requirement. So, now, we have identified structures, behaviours and functions and also mapped them.

Ok, so this looks something like a graph right?

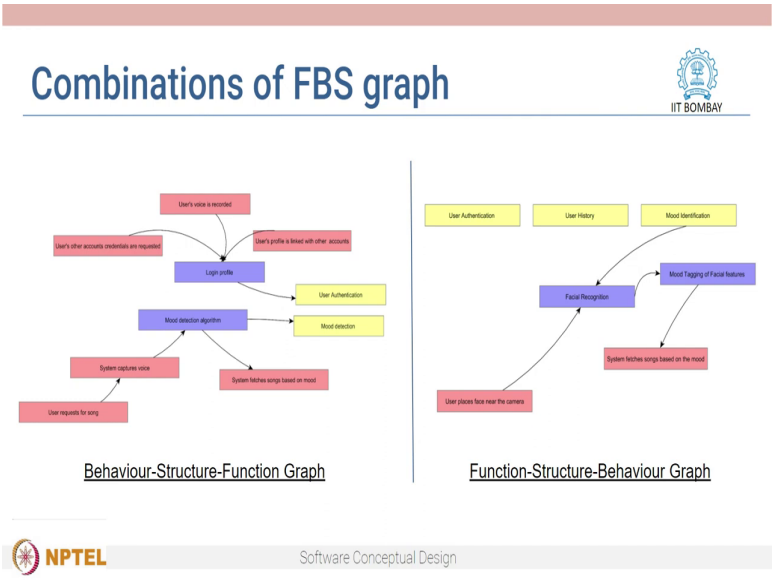
Yes, it is called the Function Behaviour Structure - FBS graph. The nodes are the FBS links connect the FBS just like a graph you could start from any of the nodes F, B or S and move on to the next as you identify and connect them.

(Refer Slide Time: 04:14)



So, learners, here is a question for you, can you think of combinations of FBS graph and the kinds of graph that can be created? Pause the video write your answers in the notebook and continue.

(Refer Slide Time: 04:40)



As you can see on the screen, there are combinations of the FBS graph, in this graph, the behaviours are mapped to the structures and then the function BSF. In other graph, functions are broken down and mapped to structures which are combined to different end-user behaviours FSB. So, there can be many types and kinds of FBS graphs.

Oh ok. So, there are many possible FBS graphs for a given problem, but how will I know if the graph that I have created is good enough?

That is a good question there is a checklist of things in to keep in mind to self evaluate a FBS graph.

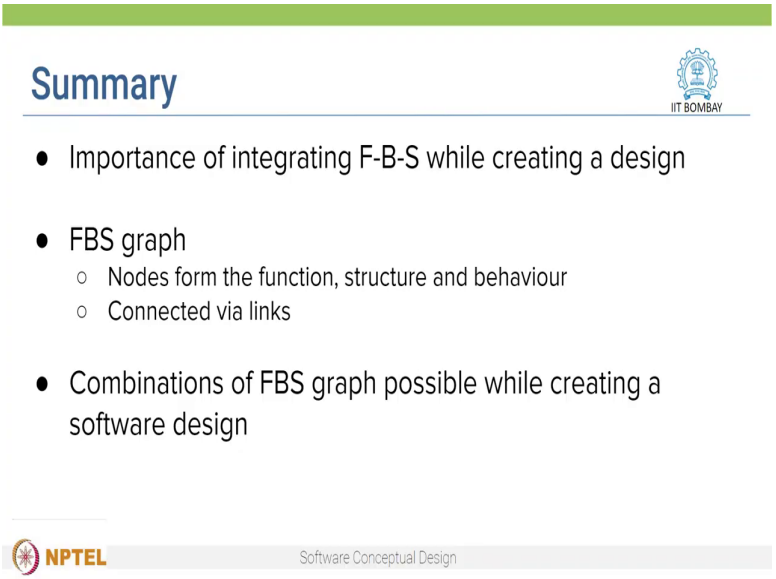
Ok. That sounds interesting.

You know there is a learning environment where you could do all this, create a FBS graph and self evaluate it for different software design problems.

Really! well, that sounds like a very cool solution.

Yes, think and link is a learning environment where the learners are taken through the steps of creating the FBS graph as well as evaluating it.

(Refer Slide Time: 05:59)



In this video, we saw the importance of integrating FB and S while creating a design. We also saw that one of the ways to integrate them is to create a FBS graph.

Yes, in an FBS graph, the nodes they form the function, structure and behaviour and these nodes are connected via links.

You also saw that there are many combinations and kinds of FBS graph possible for a software design problem. A checklist is available to self-evaluate the FBS graph. In the next

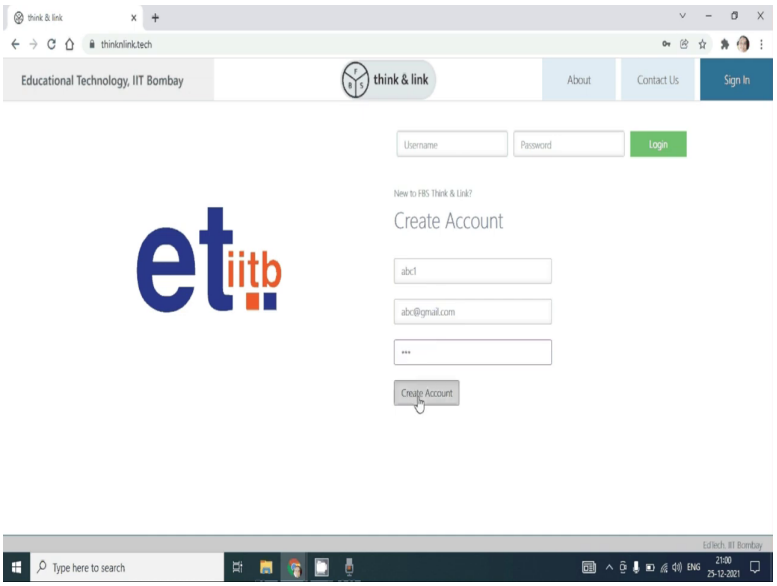
LeD, we will explore how to use think and link to create and self evaluate a FBS graph to various software design problems.

Software Conceptual Design

Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering Indian Institute of Technology, Bombay

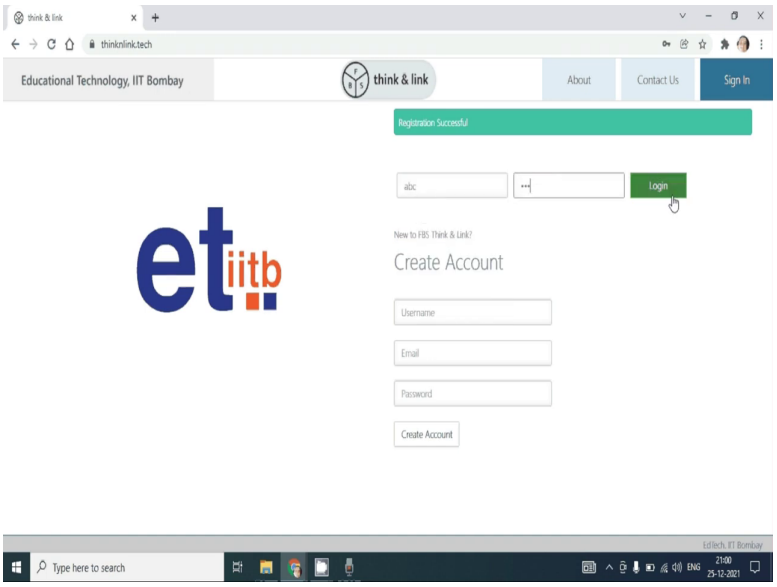
Lecture - 12
Creating Software Conceptual Designs using think and link

(Refer Slide Time: 00:05)



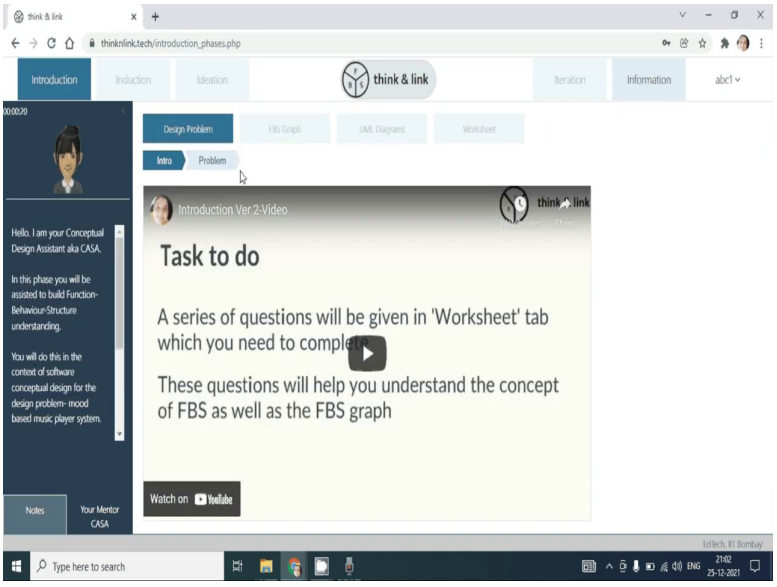
This is the URL for the think and link learning environment. Once you arrive at this page you will have to create a username for yourself; let us say for example, I create a username, please provide an email id as well as a password.

(Refer Slide Time: 00:38)



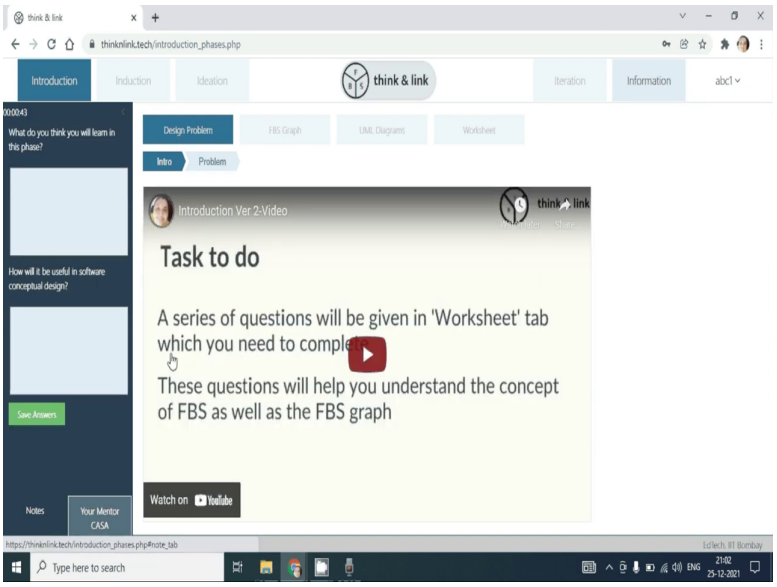
Once you create an account, you use the username and the password to log in.

(Refer Slide Time: 00:48)



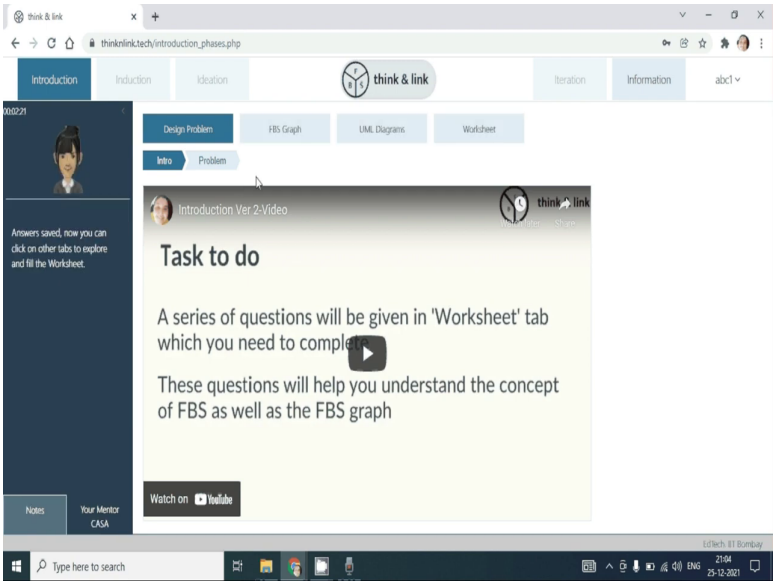
Once you log in you will arrive at this page, where you see the phase that you are in the introduction phase. You have the pedagogical agent who will give you prompts throughout the learning environment.

(Refer Slide Time: 01:09)



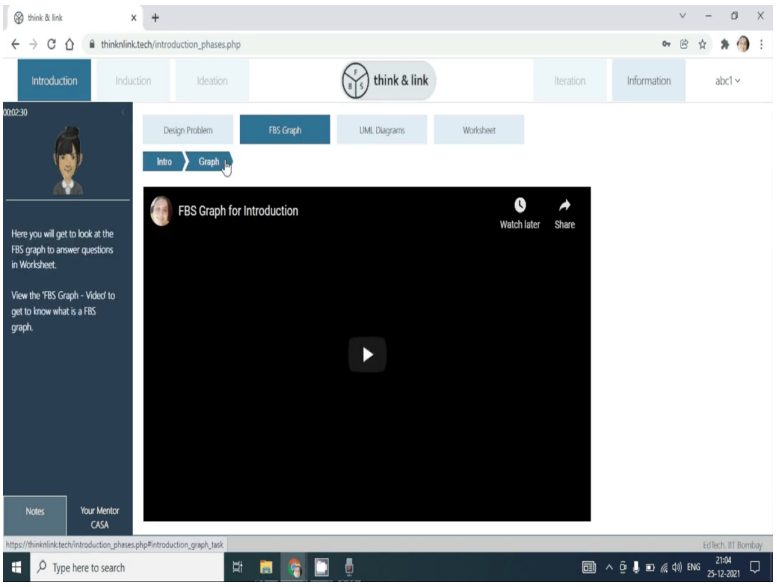
You also have a place where you could write your notes. In this video, you will be taken through the task which you have to be doing, once you complete the task the other tabs will automatically be enabled.

(Refer Slide Time: 01:29)

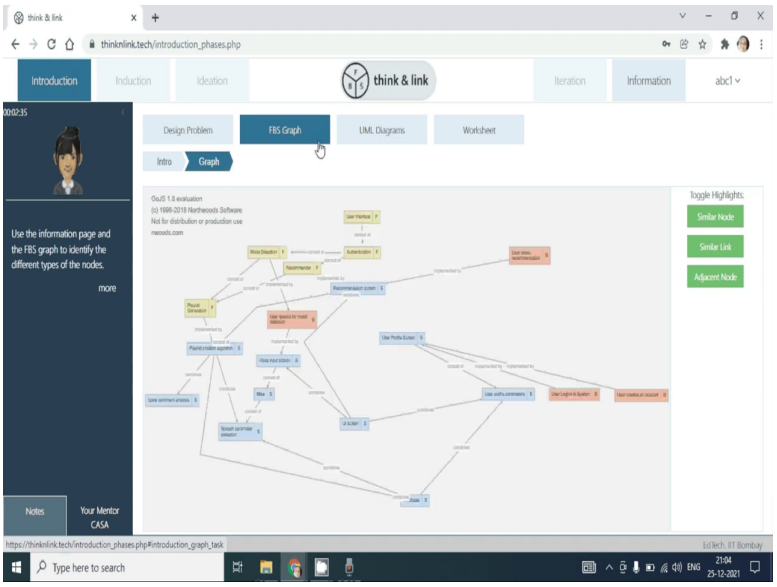


Once you save your answers in this phase, you will be provided with the FBS graph, as you can see.

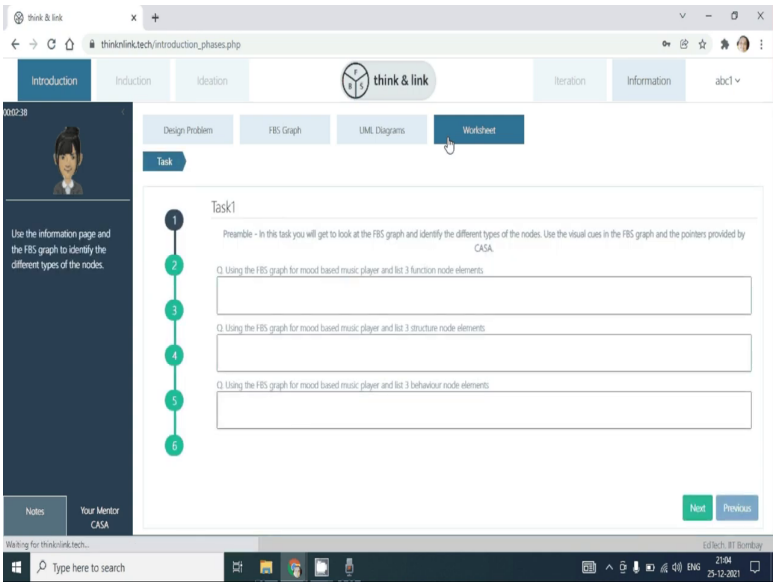
(Refer Slide Time: 01:37)



(Refer Slide Time: 01:45)

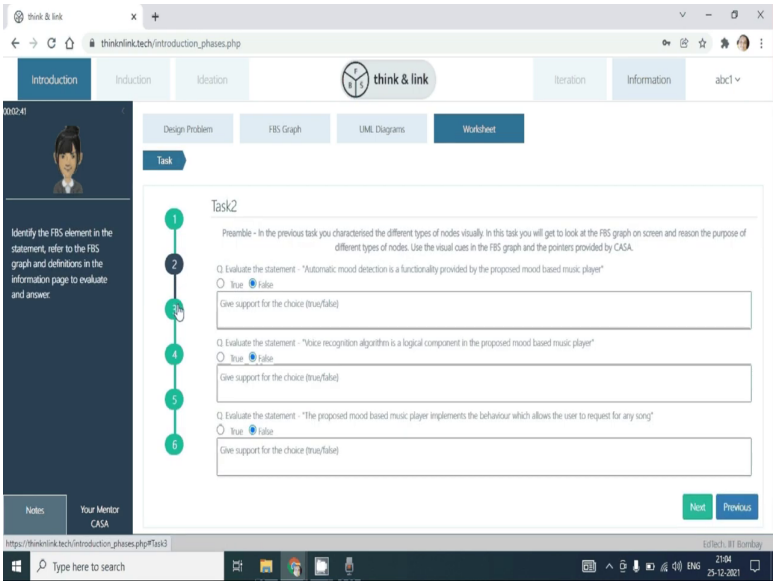


(Refer Slide Time: 01:48)

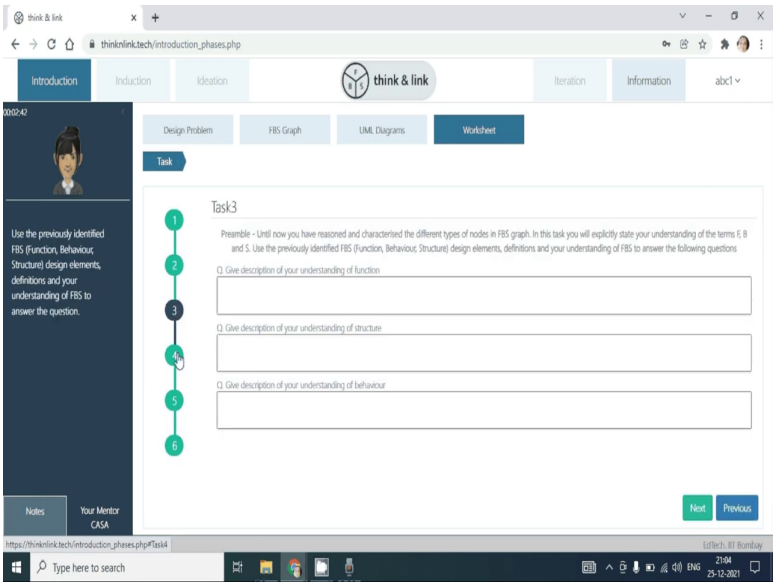


A worksheet where you will have to complete a set of tasks.

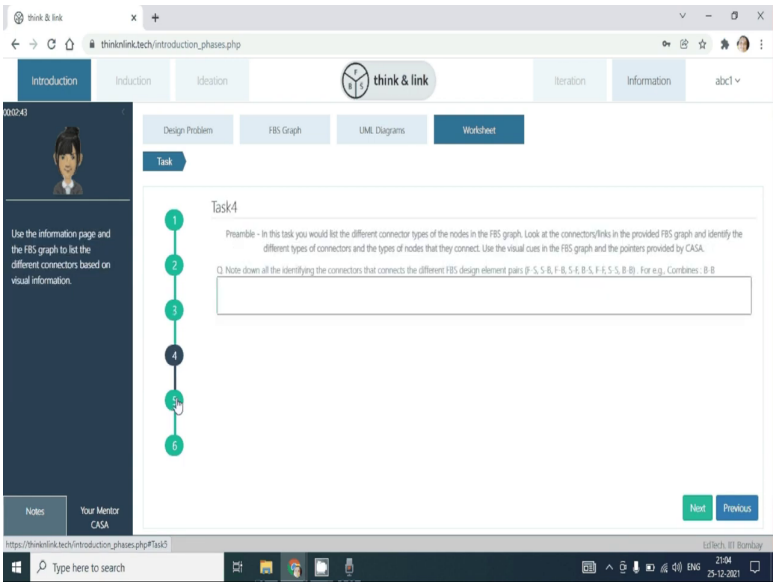
(Refer Slide Time: 01:50)



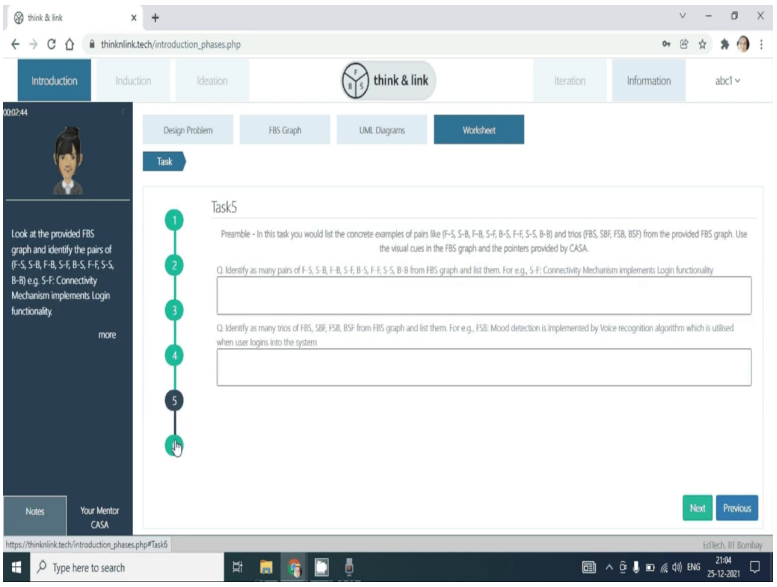
(Refer Slide Time: 01:51)



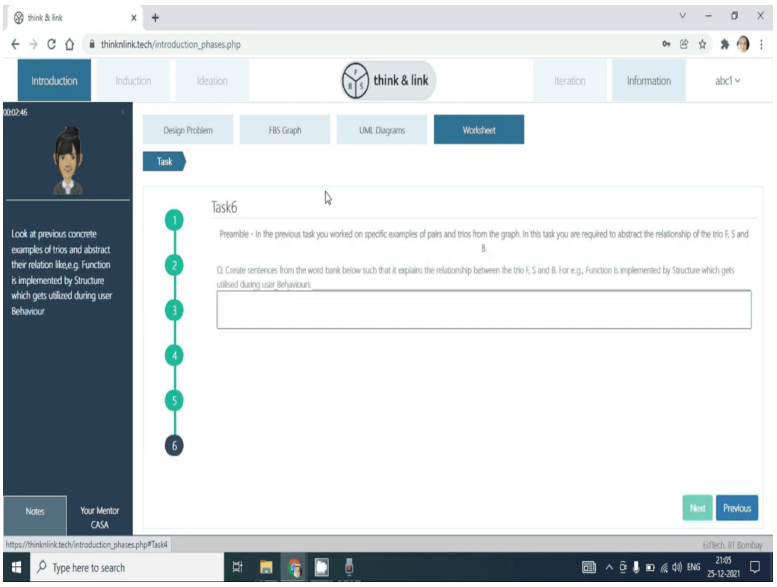
(Refer Slide Time: 01:53)



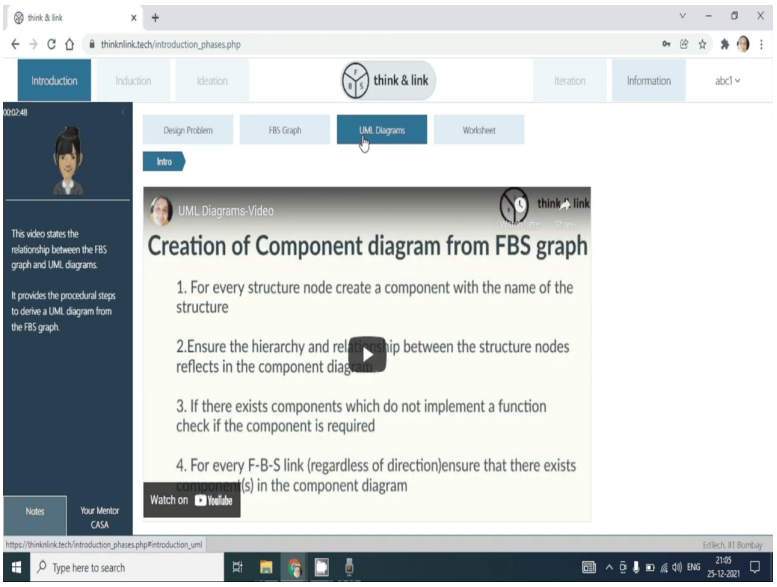
(Refer Slide Time: 01:54)



(Refer Slide Time: 01:56)

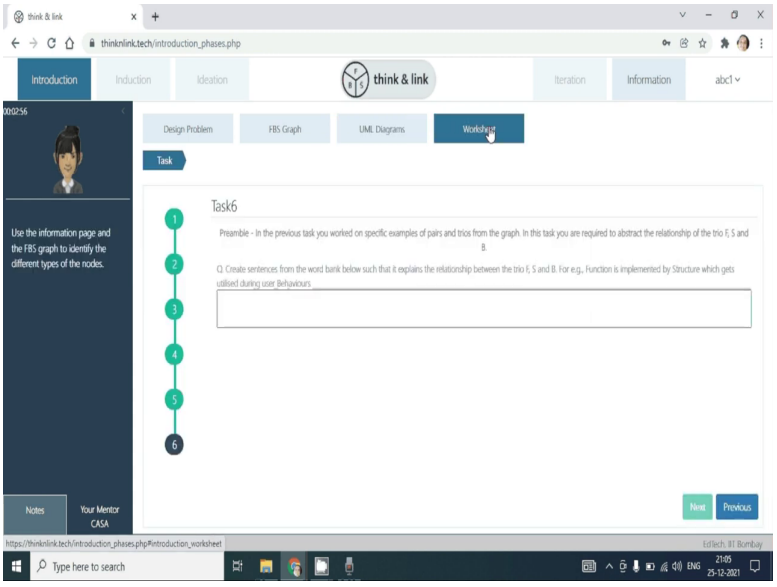


(Refer Slide Time: 01:58)



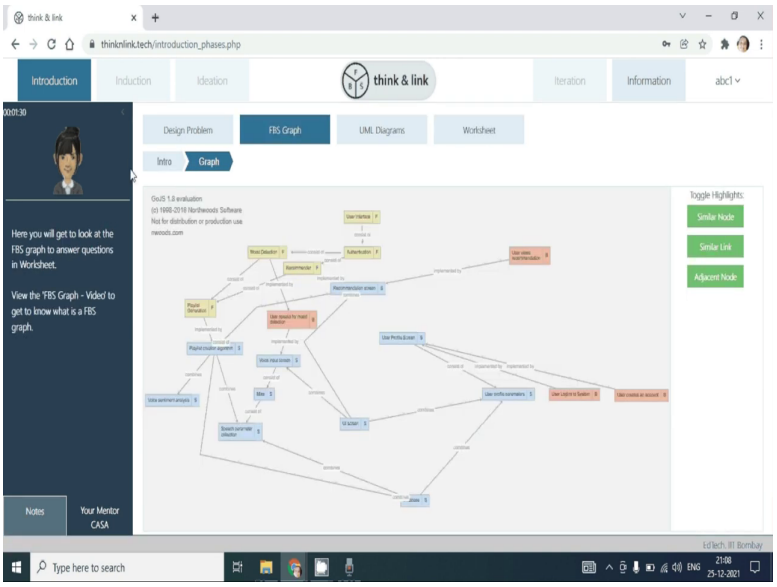
As well as a UML diagram corresponding for the FBS graph.

(Refer Slide Time: 02:06)



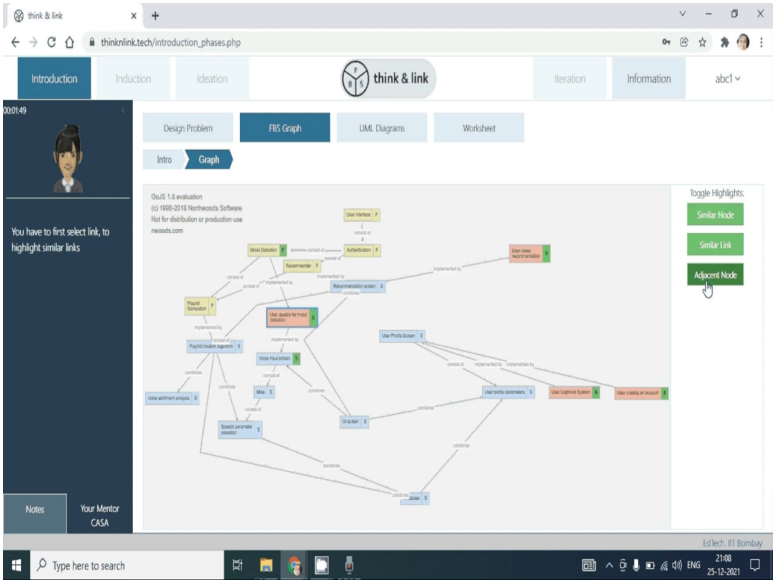
Once you complete the worksheet question and answers, this phase will end.

(Refer Slide Time: 02:10)



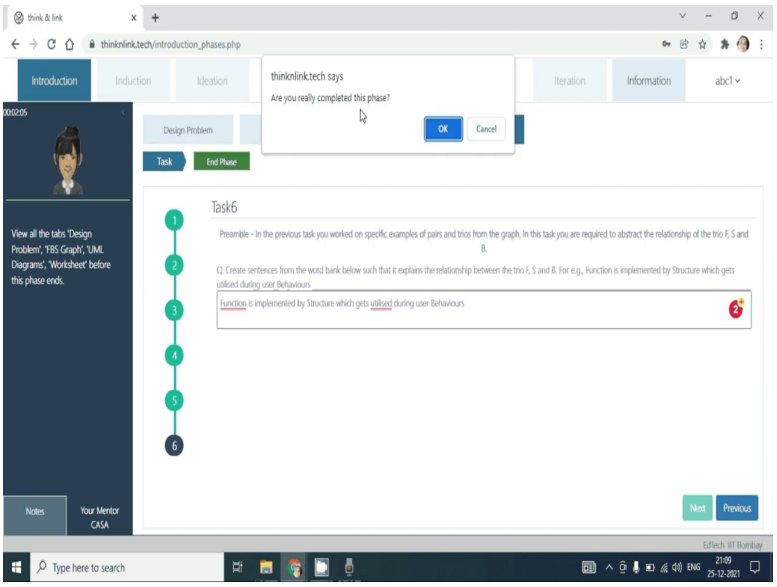
In the introduction phase, you are required to look at the FBS graph, explore the FBS graph via the various interactions possible.

(Refer Slide Time: 02:24)



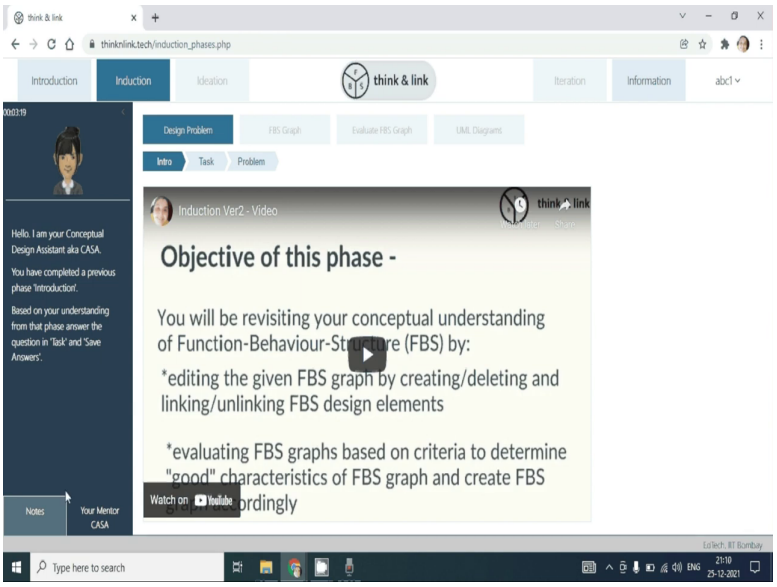
Complete the answers in the worksheet.

(Refer Slide Time: 02:34)

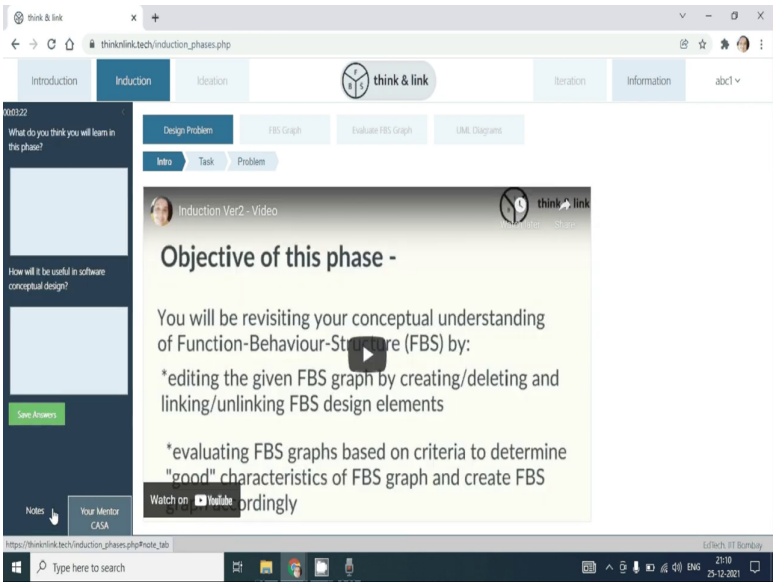


Once you complete all the answers and explore the FBS graph, the end phase will be visible. And as you end this phase, you will be moved to the next phase, which is the induction phase.

(Refer Slide Time: 02:50)

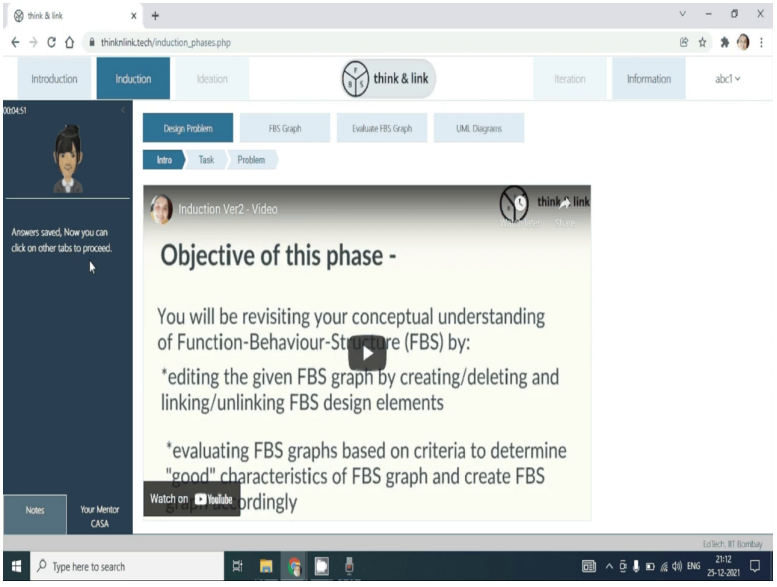


(Refer Slide Time: 03:02)



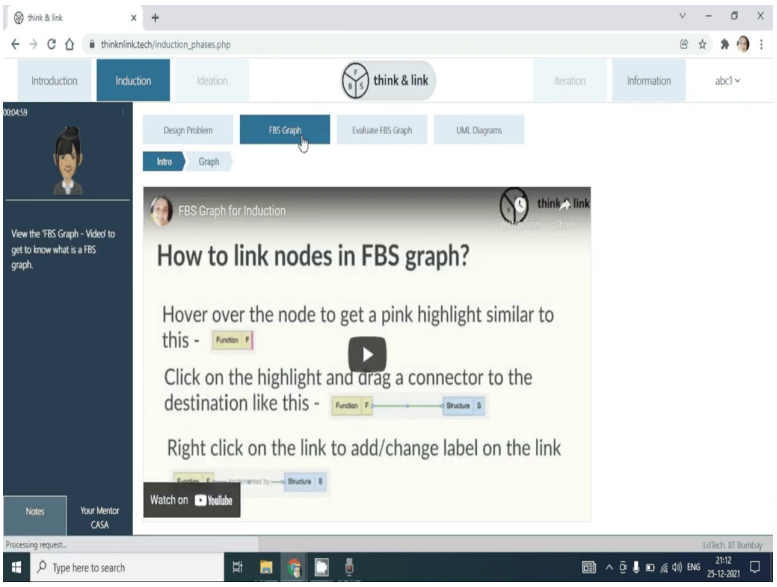
In the induction phase similar to the introduction phase you are required to look at the video, find the objectives in the phase as well as how would you incorporate the learnings in the software conceptual design. You will have to complete these answers, and then the next tabs will be enabled.

(Refer Slide Time: 03:23)

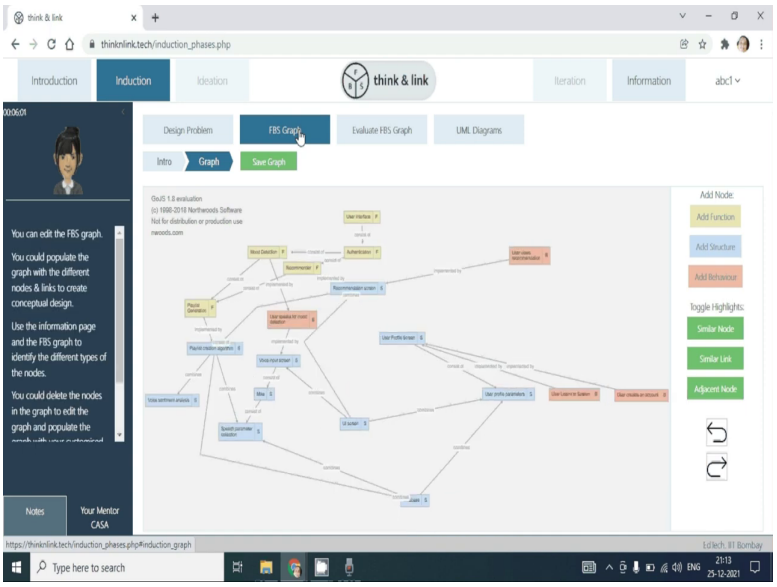


Once you save your answers you the other terms will be automatically enabled.

(Refer Slide Time: 03:33)

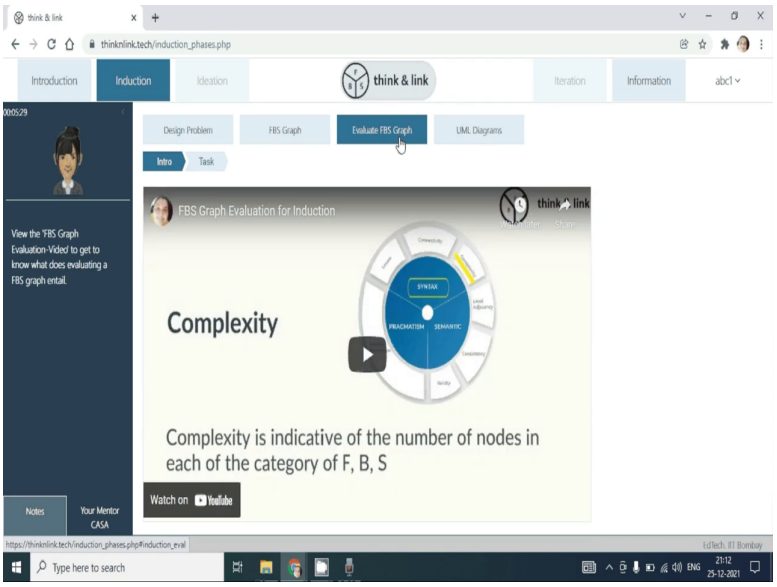


(Refer Slide Time: 03:36)

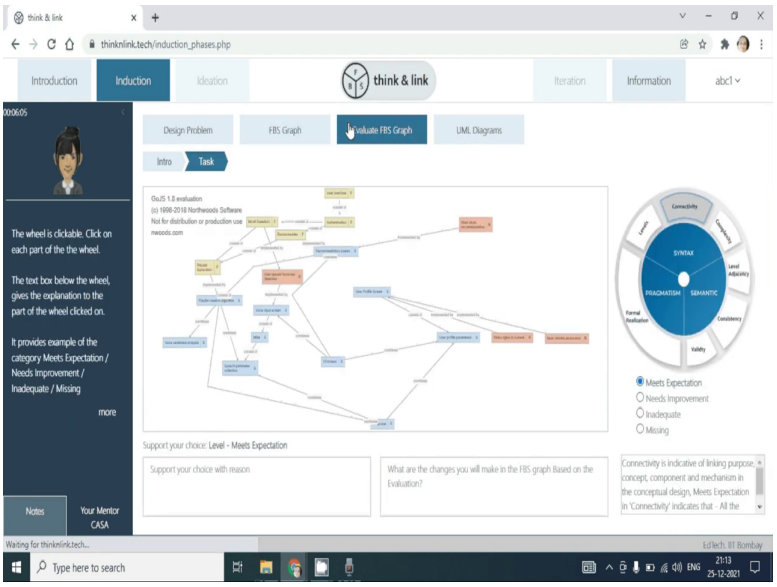


In this phase, you are required to not only explore the graph but edit the graph by adding Functional, Structure, Behaviour nodes and connecting them. The pedagogical agent will provide prompts at various phases as you edit the graph.

(Refer Slide Time: 04:00)



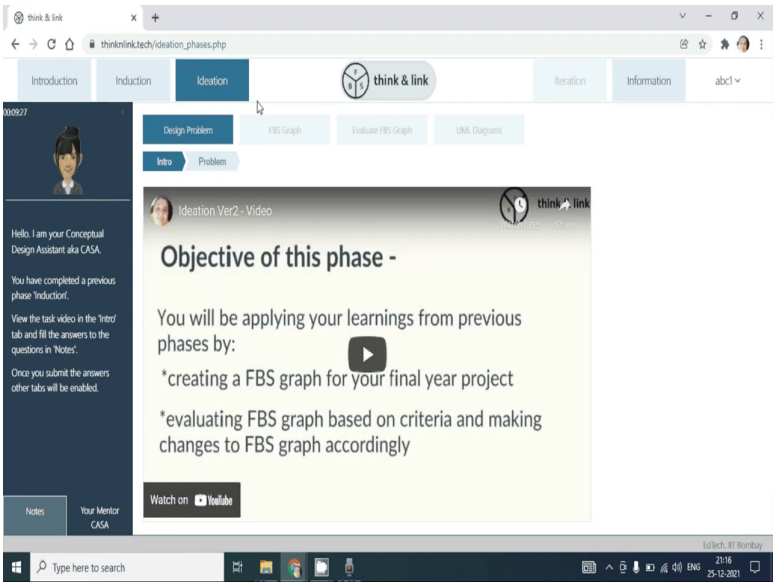
(Refer Slide Time: 04:04)



After having edited the graph, you can evaluate your FBS graph based on the evaluation parameters provided in this wheel. You will have to look at the evaluation parameters, assign corresponding categories and also, at the same time, reflect on why you are giving the particular rating. You can go back and edit the FBS graph and come back and evaluate the FBS graph.

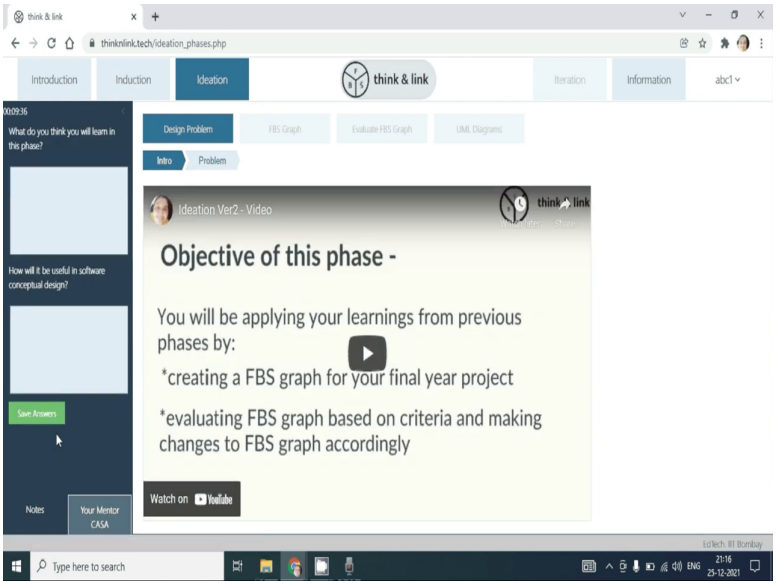
Once all the evaluation parameters have been completed, this phase will end. After you have edited the FBS graph and evaluated, this phase will end.

(Refer Slide Time: 04:58)



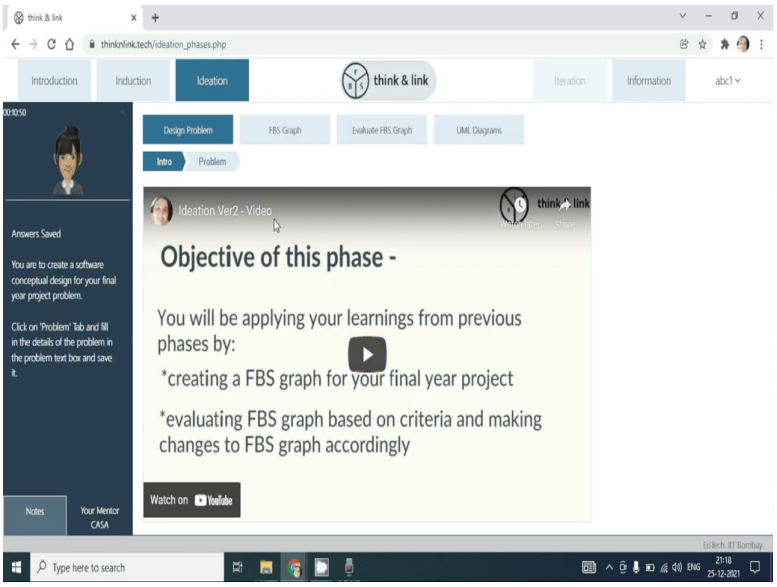
You are then taken to the third phase, which is the ideation phase.

(Refer Slide Time: 05:11)

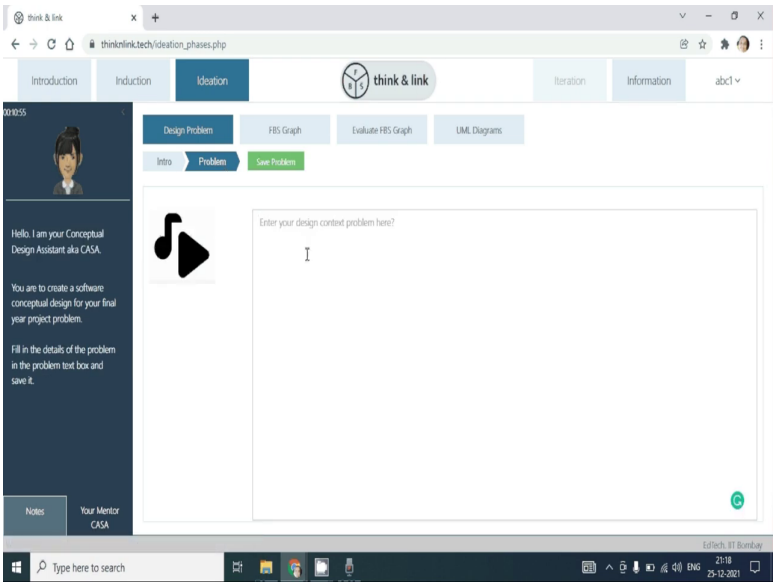


In this phase, similar to the earlier two phases, you will have to complete the planning questions that are provided.

(Refer Slide Time: 05:15)

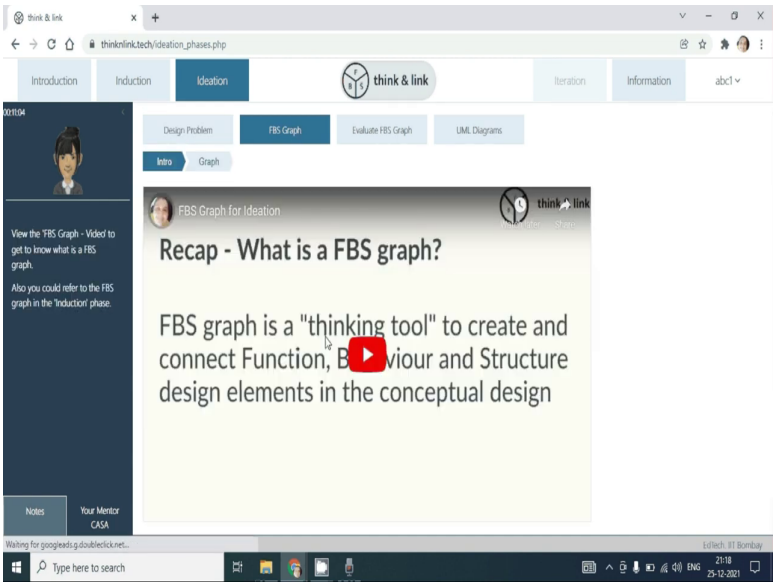


(Refer Slide Time: 05:19)

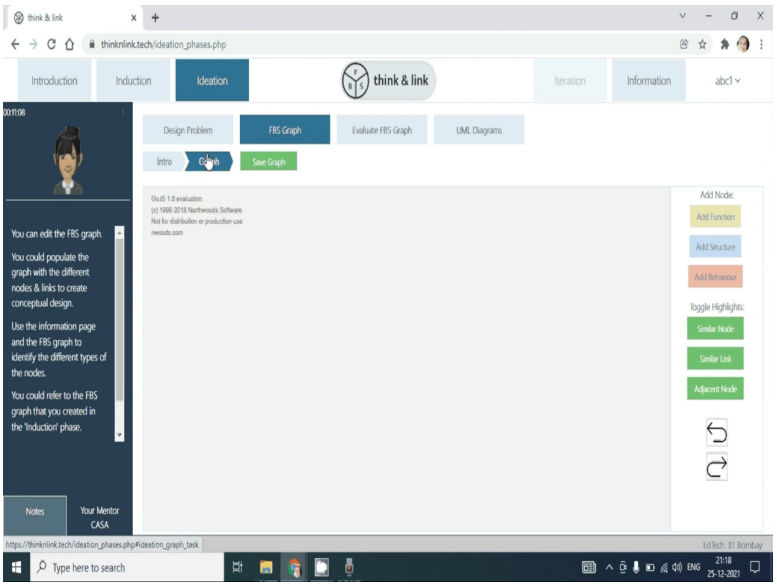


In the last phase of think and link, you are required to write the design problem that you are interested to create a software conceptual design.

(Refer Slide Time: 05:29)

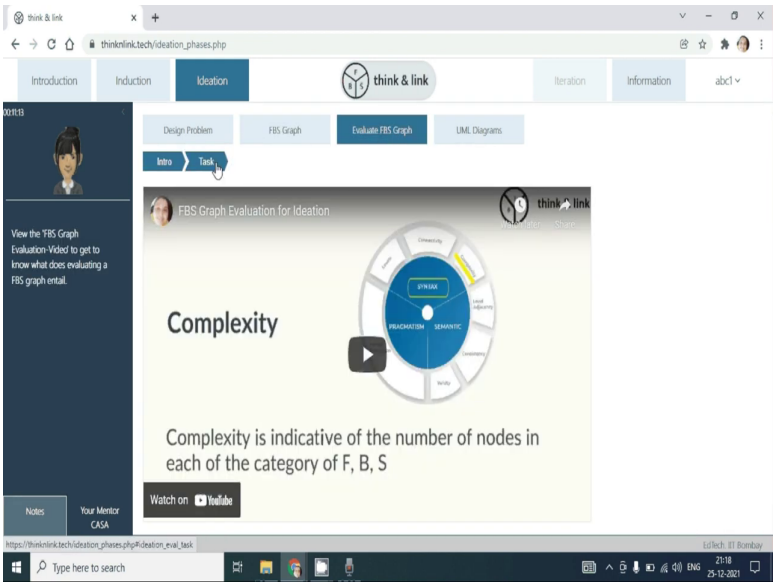


(Refer Slide Time: 05:32)

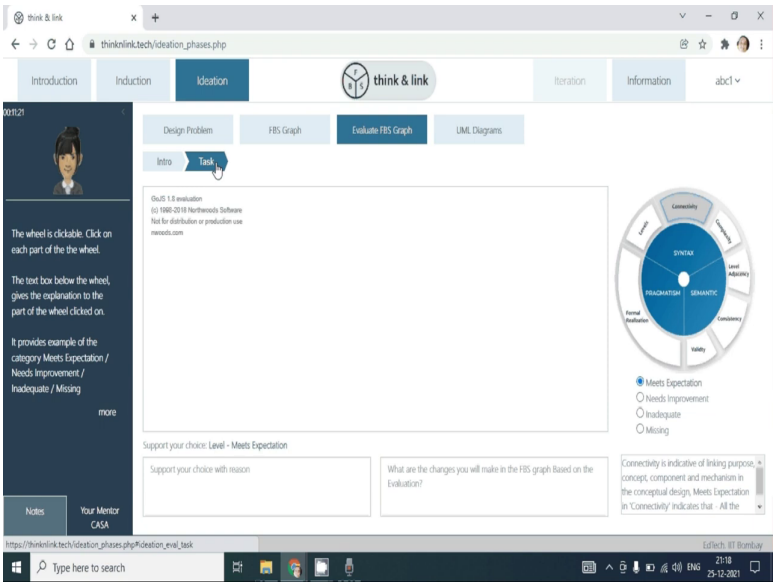


Create a FBS graph from the scratch.

(Refer Slide Time: 05:37)



(Refer Slide Time: 05:39)



Evaluate the FBS graph based on the evaluation parameters in the wheel and then complete this task of creating FBS graph for two problems.

Thank you.

Software Conceptual Design

Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering
Indian Institute of Technology, Bombay

Week - 02
Lecture - 13
Summary

In this week, we explored the Software Conceptual Design process. We saw that it involves the creation of a design description which is represented graphically or numerically, or textually. We explored a technique for software conceptual design called the Function Behaviour Structure framework.

And we have used the tools called think and link in order to help us create the software conceptual designs. Keep exploring the tool Think and Link, for creating software conceptual design and many techniques using the Function - Structure - Behaviour framework.

Software Conceptual Design

Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering
Indian Institute of Technology, Bombay

Lecture - 14
Week 3: Comprehending and Evaluating Software Designs

Last week, we saw how to create a software conceptual design using the Function Behavior Structure framework. This week, we will be looking at another software design representation called UML or the Unified Modelling Language. You will learn different types of UML diagrams and how they can be used to create detailed designs.

You will also be interacting with the learning environment called VeriSIM, which will help you develop an integrated understanding of different UML diagrams. Finally, we will be looking at how you can evaluate a given design by considering several quality attributes.

Software Conceptual Design
Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering
Indian Institute of Technology, Bombay

Lecture - 15
Software Modelling

In the previous week, we learnt about the Function Behaviour Structure framework and used the FBS framework to create a software conceptual design.

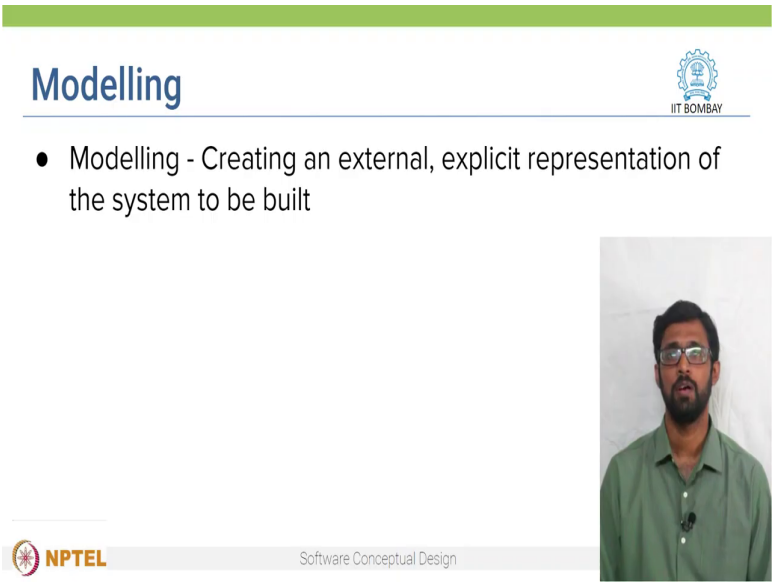
Yes. The FBS framework helped me create the conceptual design for the mood based music player. So, in a way, it helped me plan for what to code during the development phase right?

Yes, that is true. Another benefit of creating such a representation is that it helps in communicating the design with a larger team. Representations that we saw in the system think and link, like the FBS graph and UML diagram, represent the design communicated in different ways and methods.

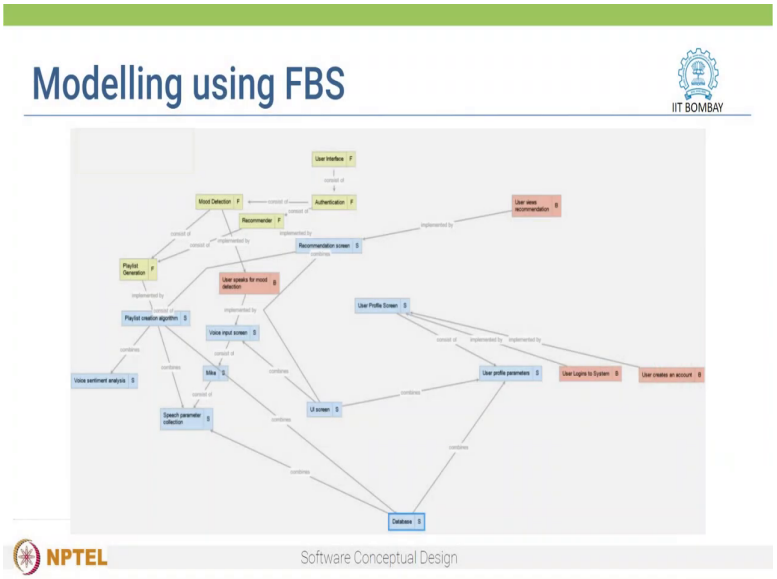
Yes, and this process of representing the solution design is referred to as software modelling.

Modelling, what is that mean? Can you please elaborate in more detail?

(Refer Slide Time: 01:18)

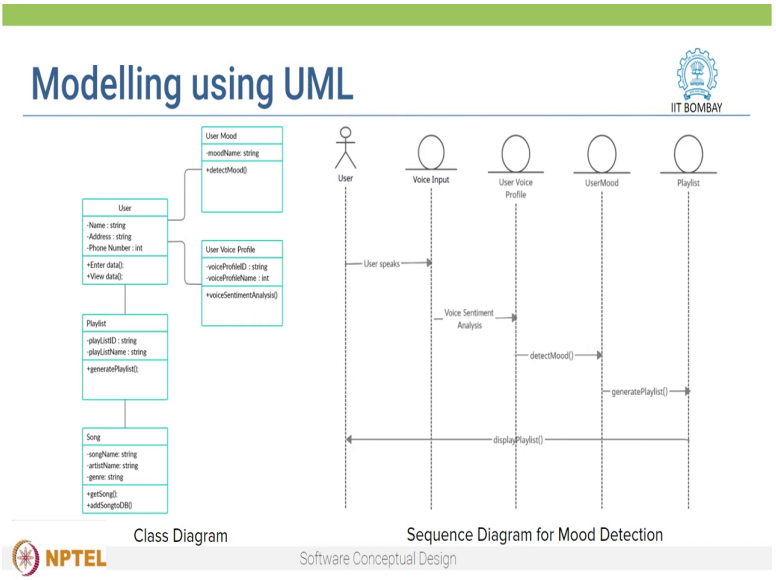


(Refer Slide Time: 01:29)



So, modelling is a way of creating an external, explicit representation of the system to be built. For example, the FBS graph is an external representation which describes the Functions, Structures and Behaviours of the system to be built. In a way, it provides an overview of the entire software solution.

(Refer Slide Time: 01:46)

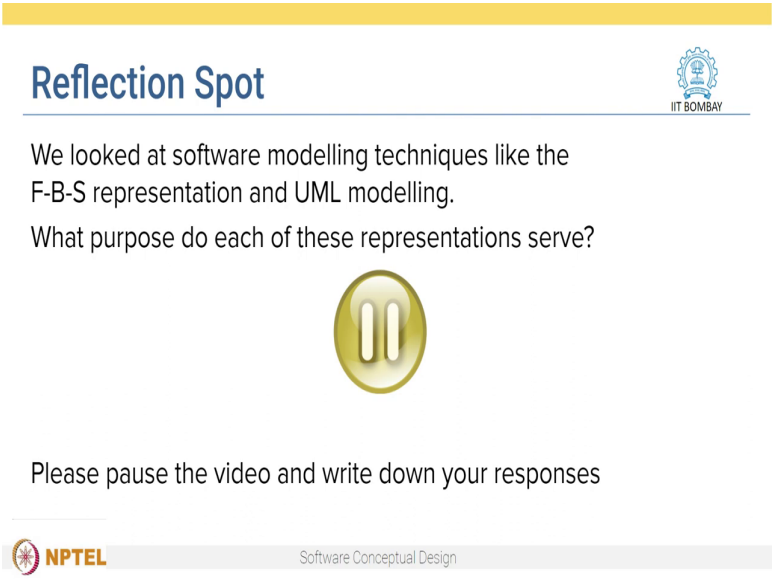


There are other models like the unified modelling language diagrams or UML diagrams, and these can help us represent the software design via multiple views and at a greater level of detail. We will learn more about modelling using these UML diagrams in the next video.

So the FBS representation is a form of software modelling. UML is another form of software modelling, why are both required?

Yes, this is an important question. Let us reflect about this for a moment.

(Refer Slide Time: 02:34)

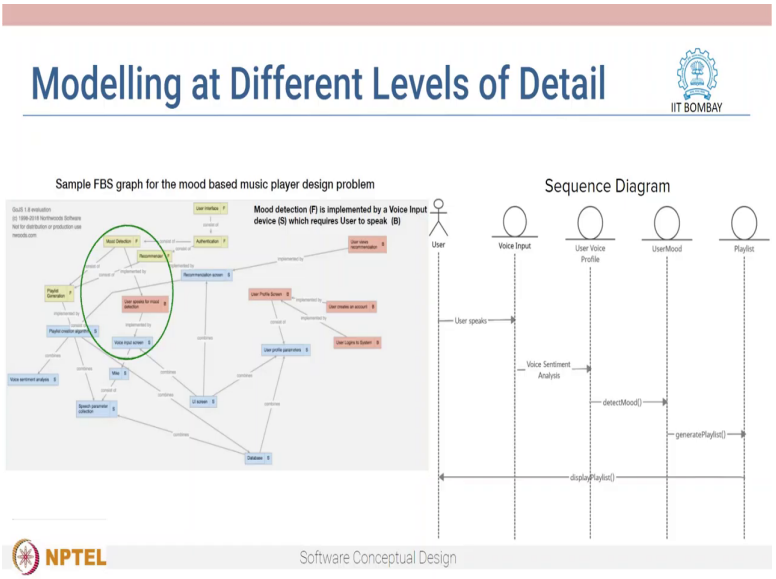


So, here is a reflection spot for you. We looked at software modelling techniques like the FBS representation and UML modelling. What purpose do each of these representation serve? You can pause your video and write your answers before proceeding.

So, both these modelling techniques they serve different purposes, based on the requirements, we can create an FBS representation as a way to create an idea or a concept of the system which we want to build. Once we have this representation, we can go into more details and create UML diagrams for the software system.

Ok, so these models are at different levels of detail.

(Refer Slide Time: 03:42)



Yes, that is true. So, on the left, you can see an FBS graph of the mood based music player. A part of the functionality, which is the mood detection is detailed out in the sequence diagram shown on the right.

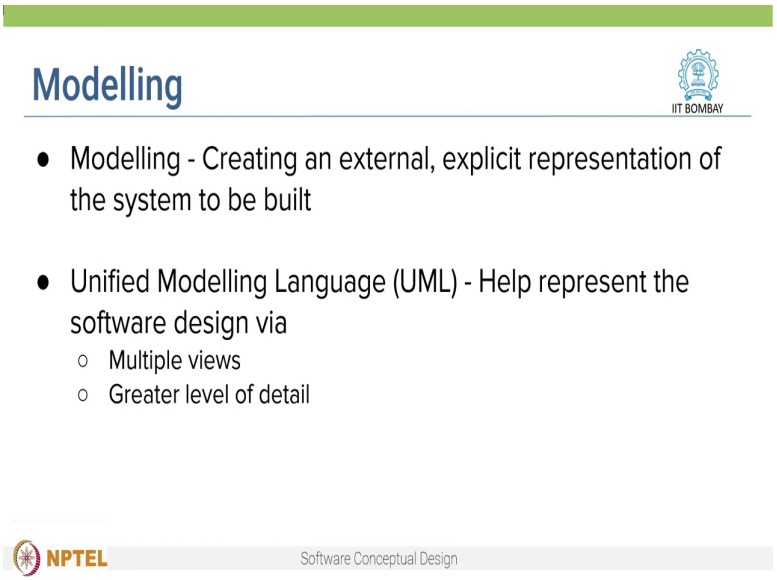
So, last week we have already explored a form of modelling using the FBS design framework then we understood the advantages of modelling. We were also introduced to this idea that modelling can be done through different representations. So, what is next?

Yes. So, another representation which is used is UML diagrams. We will explore modelling using UML diagrams in the next video.

Software Conceptual Design
Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering
Indian Institute of Technology, Bombay

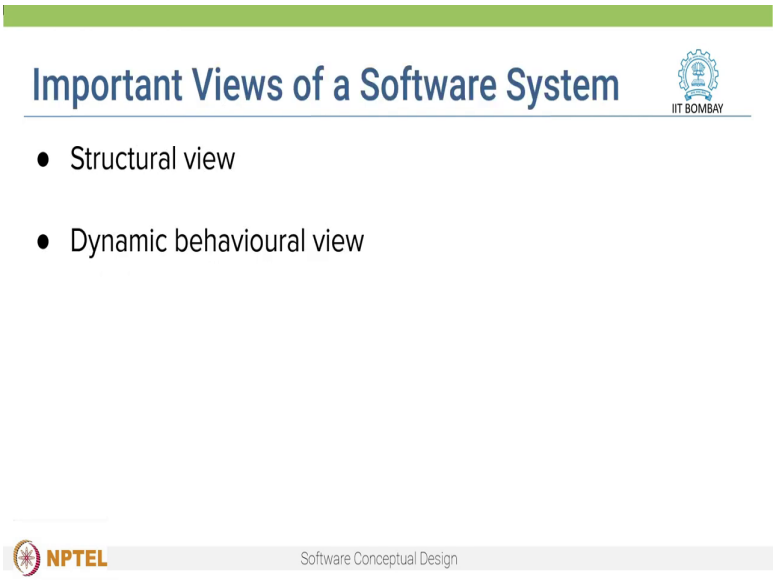
Lecture - 16
Unified Modelling Language (UML)

(Refer Slide Time: 00:05)



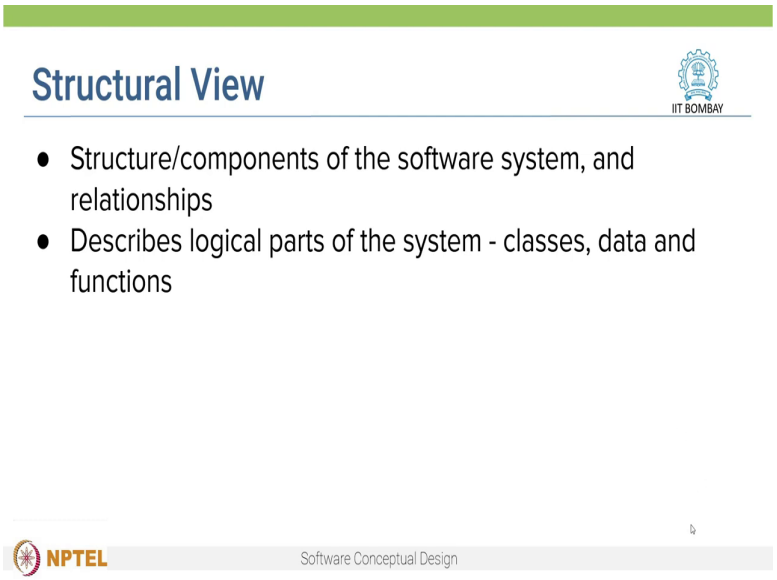
In the previous video, we talked about modelling a software design. Modelling is a way of creating an external explicit representation of the system to be built. In this video, we will look at the Unified Modelling Language or the UML diagrams. These diagrams can help us represent the software design via multiple views and at a greater level of detail.

(Refer Slide Time: 00:41)



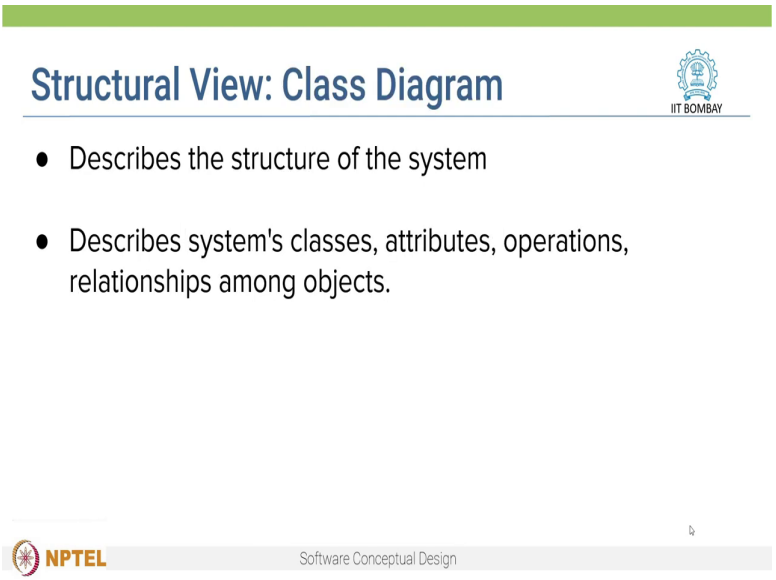
Let us look at what these views are. Two important views of a software system are the Structural view and the Dynamic behavioural view.

(Refer Slide Time: 00:52)



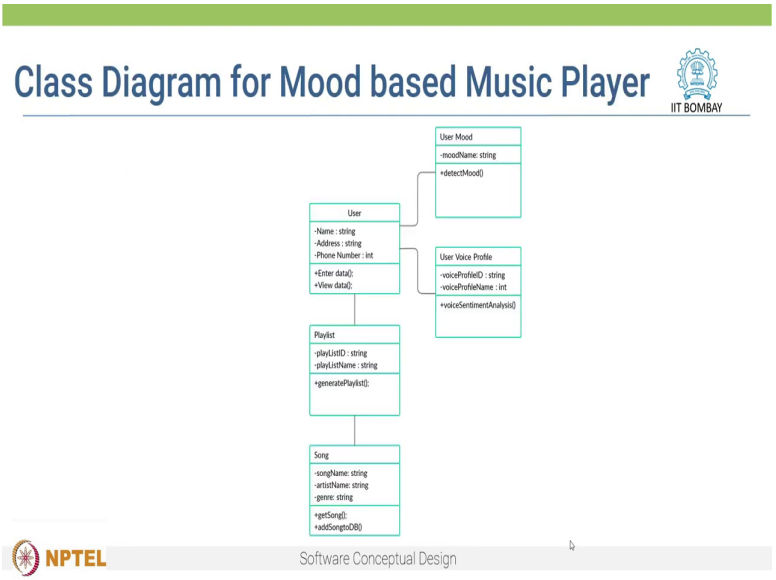
So, what is the structural view? The structural view defines the structure of the software system. They describe the things in the system and their relationships to other things. The structural view describes the logical parts of the system, including the classes and relevant data and functions in these classes. They do not describe the time-dependent behaviour of the system.

(Refer Slide Time: 01:26)



One such diagram which represents the structural view is the Class Diagram. The class diagram is a type of a static structure diagram that describes the structure of a system by showing the systems classes, the attributes, the operations or methods and the relationship among these objects.

(Refer Slide Time: 01:53)

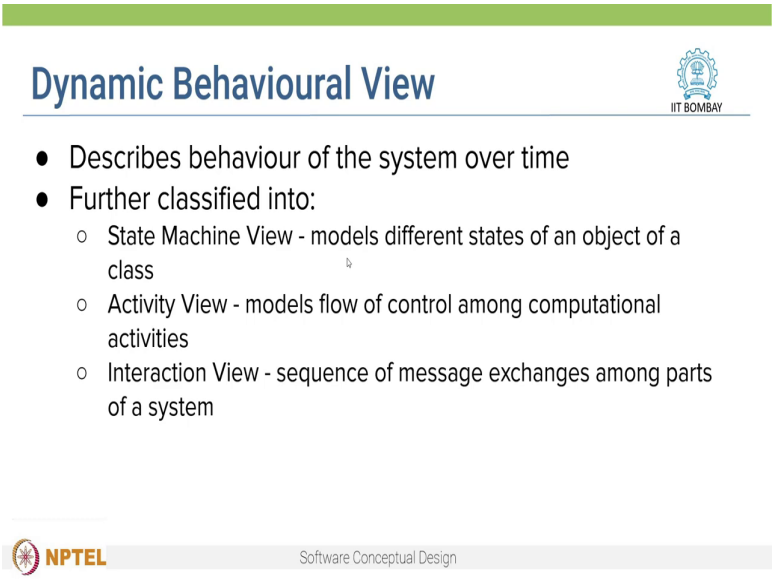


The diagram shown on this slide describes the Class Diagram of the mood based Music Player. The class diagram is the main building block of object-oriented modelling. In class diagrams, the classes are presented with boxes that contain 3 compartments. The top

compartment contains the name of the class, the middle compartment contains the attributes of the class, and the bottom compartment contains the operations of the class, which can be executed.

We see that the class diagram models relevant classes like the User, the User Voice Profile, the Playlists, the Songs and so on. Each of these classes contain data members and functions necessary to realise certain behaviours of the mood-based music player. For example, the user mood class has a data attribute moodName, which stores the current mood of the user. The detectMood function, in the user mood class updates the moodName variable based on the User and the User Voice Profile Data.

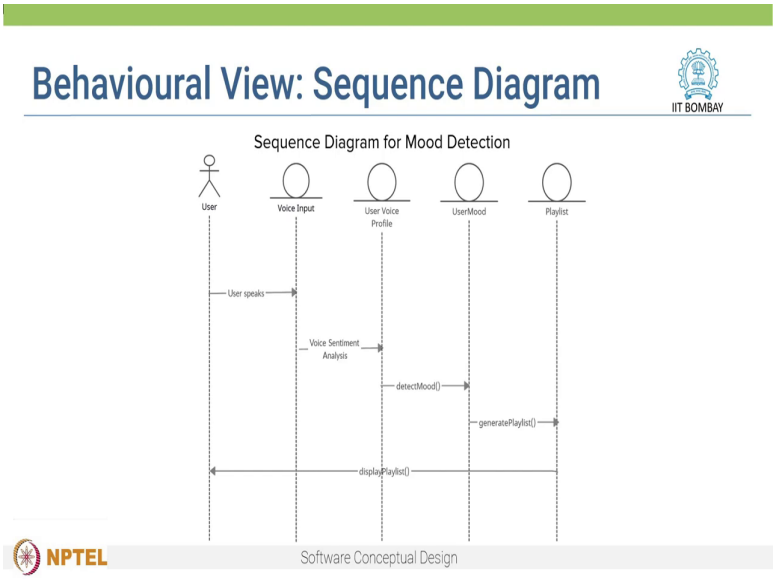
(Refer Slide Time: 03:29)



The Dynamic Behaviour View describes the behaviour of the system over time. Various views include the state machine view, the activity view and the interaction view. The state machine view models the possible life histories of an object of a class. The activity view shows the flow of control among computational activities involved in performing a calculation or a workflow.

The interaction view describes sequence of messages exchanged among different parts of a system. It gives a holistic view of the behaviour in a system by showing the flow of control across many objects.

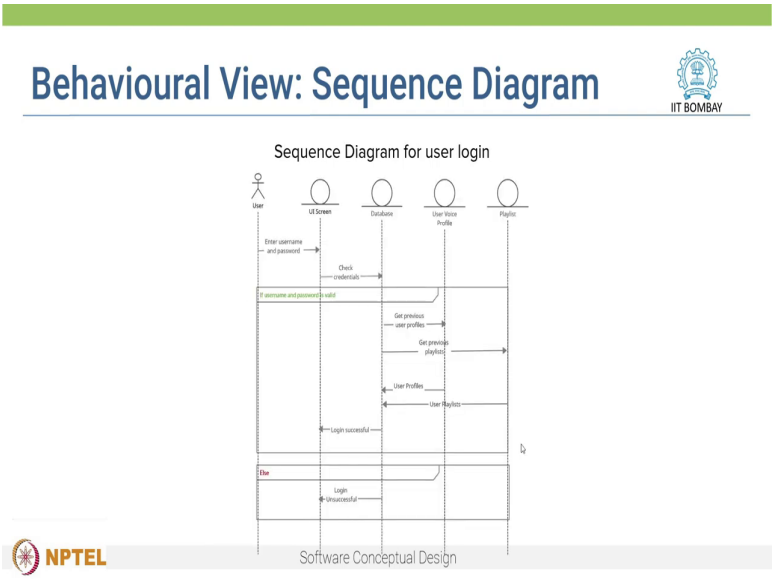
(Refer Slide Time: 04:28)



The behaviours corresponding to the requirements of a system can be implemented using Sequence Diagrams. The diagram on this slide represents the sequence diagram which describes the behaviour of the mood detection requirement for the mood based music player system. The classes from the class diagram are represented in the sequence diagram on the top. And they have parallel vertical lines called the lifelines.

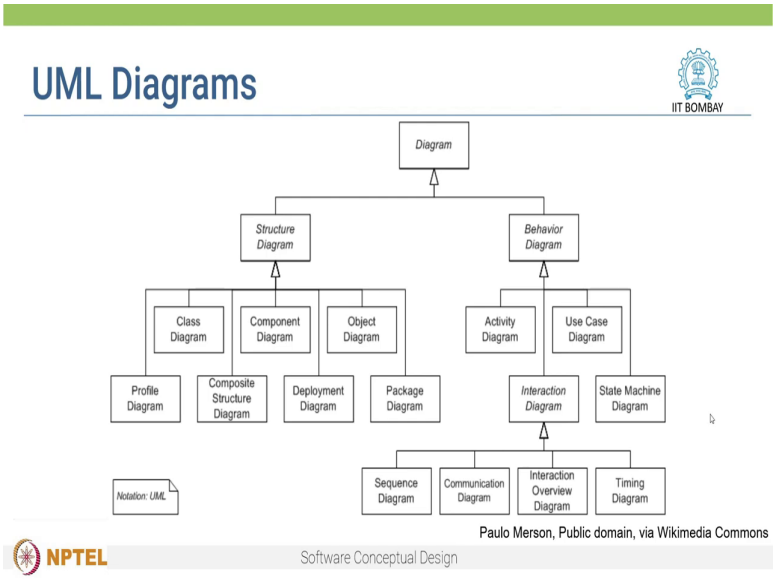
The horizontal arrows which you see are the messages exchanged between them in the order that they occur. These messages can be functions from the class diagram, such as detectMood, generatePlaylist and so on. This sequence of messages correspond to the implementation of a given requirement or sub-requirement. In a similar fashion, sequence diagrams can be constructed to model other requirements as well.

(Refer Slide Time: 05:52)



For example, the sequence diagram in this slide models the login feature for the mood based music player.

(Refer Slide Time: 06:12)



There are several other UML diagrams, such as state charts, communication diagrams, activity diagrams, etcetera which can be used to adequately model a given software design. Details of various UML diagrams can be found in the extra resources provided in this week. During the software design process, a subset of these UML diagrams are used to create the design model.

Software Conceptual Design
Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering
Indian Institute of Technology, Bombay

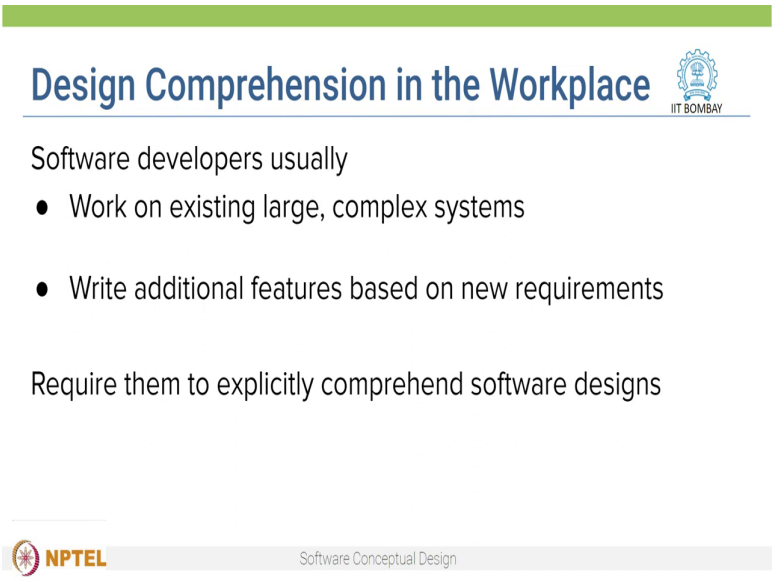
Lecture - 17
Software Design Comprehension

In the previous video, we looked at different UML diagrams used to model a given design. We saw that a given design can have various diagrams which describe multiple views of the system at different levels of granularity.

So, we can understand the software system through these detailed design diagrams, is not it?

Yes, this process of trying to understand a given design is known as software design comprehension. Design comprehension is an important skill required in the workplace.

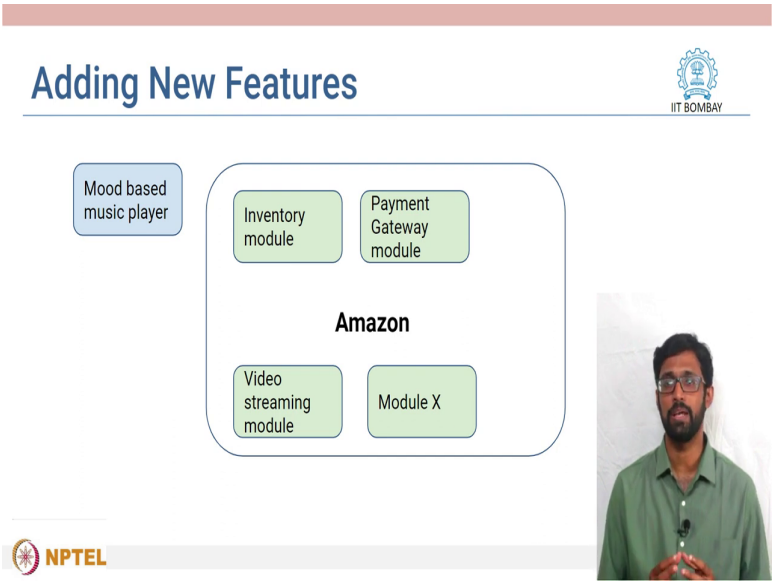
(Refer Slide Time: 00:51)



Software developers usually work on large, existing complex systems, and they write additional features based on new requirements given to them. These tasks require them to explicitly comprehend software designs.

Let us think of a scenario, where design comprehension is needed.

(Refer Slide Time: 01:18)

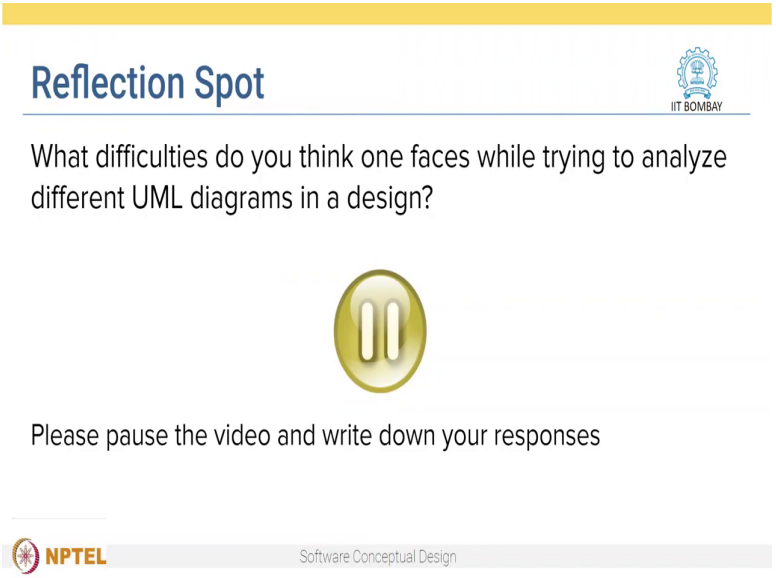


For example, let us say we need to integrate the mood based music player with an existing Amazon music application. How do we go about doing this? So, developers, before they actually start coding, they need to comprehend existing subsystems in the music player and design the new mood based music feature. So that it can be seamlessly integrated to the existing design.

I understood that design comprehension is essential, but let us say I am a student was just learnt about these UML diagrams. It can be difficult to make sense of all these diagrams right?

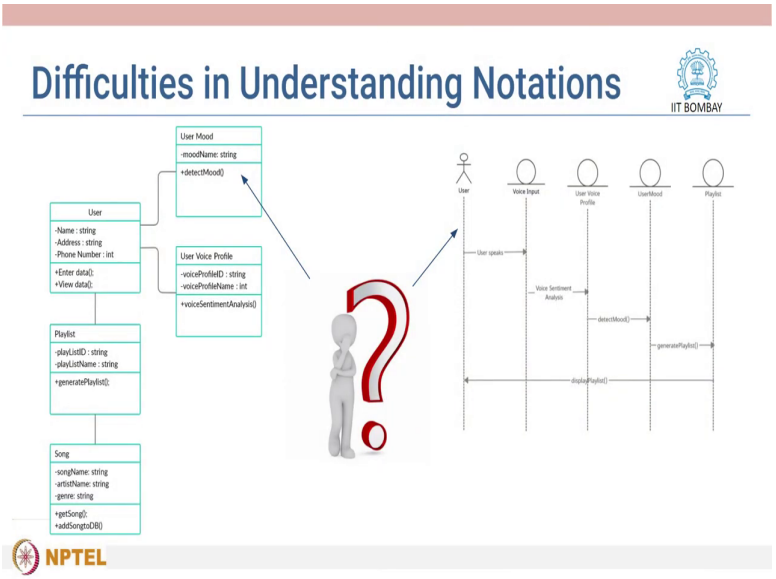
Yes, that is true. Let us reflect on these difficulties. So, students, here is a reflection spot for you.

(Refer Slide Time: 02:24)



What difficulties do you think one faces while trying to analyze different UML diagrams? You can pause this video and write down your responses before proceeding.

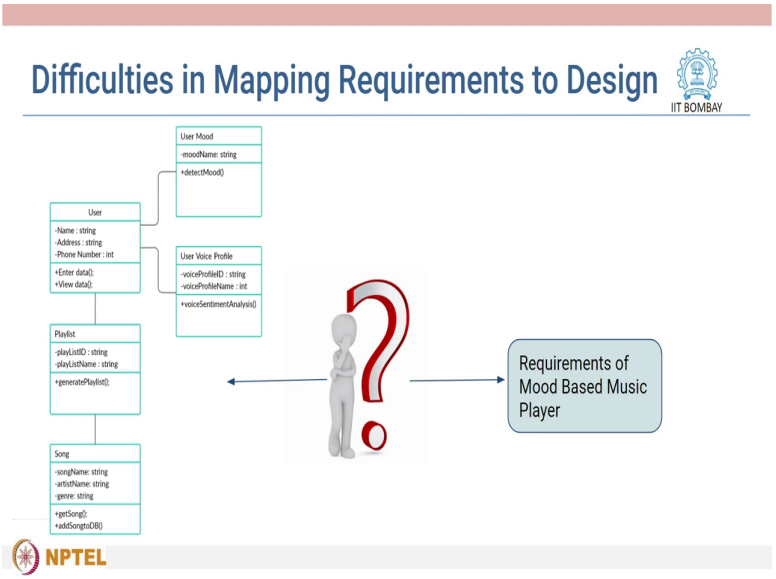
(Refer Slide Time: 02:45)



One might not understand what these diagrams mean. For example, there are various syntax and notations for the different design diagrams.

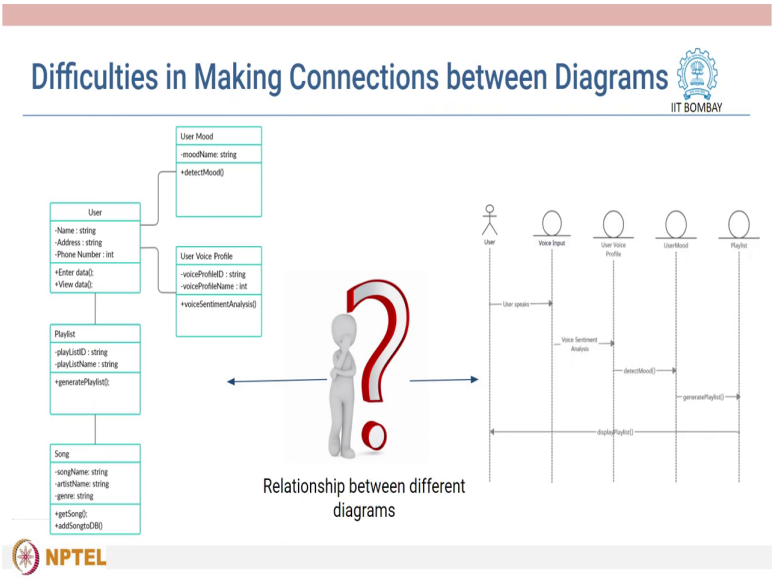
Yes, that is true. But this can also be easily learned and practised.

(Refer Slide Time: 03:07)



Others might find it difficult to understand how certain requirements and parts of the solution design map. For example, how do classes in the class diagram, like song similarity, etcetera relate to the requirement of the mood based music player?

(Refer Slide Time: 03:30)



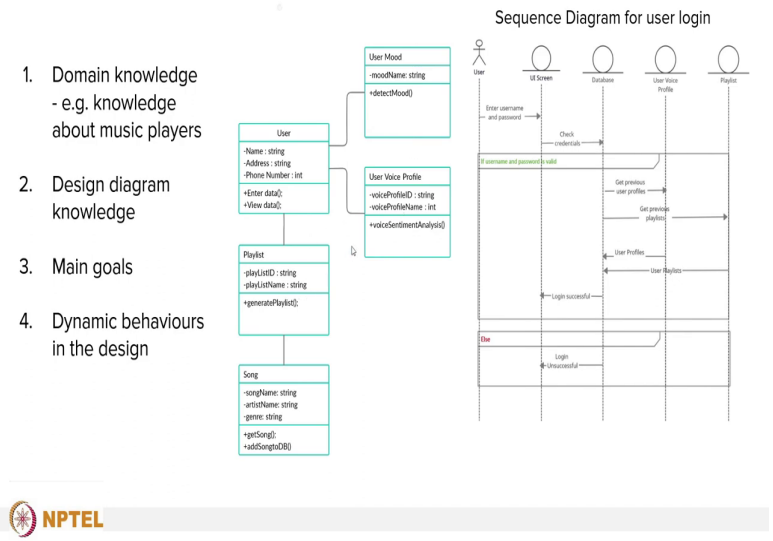
Another difficulty that people could face is making connections between these diagrams, mainly because each of the diagram has huge amount of details.

Yes, that is true. For example, the sequence diagram takes objects from the class diagram and describes its behaviour. However, students find it difficult to make this connection between the class diagram and the sequence diagram.

But it is important to make connections between the different UML diagrams to comprehend the design, but how can we develop design comprehension?

Yes, this is a very good question. To answer this let us see how an experienced software designer tries to comprehend different diagrams in the design. So, how does an experienced software designer comprehend a given design?

(Refer Slide Time: 04:37)



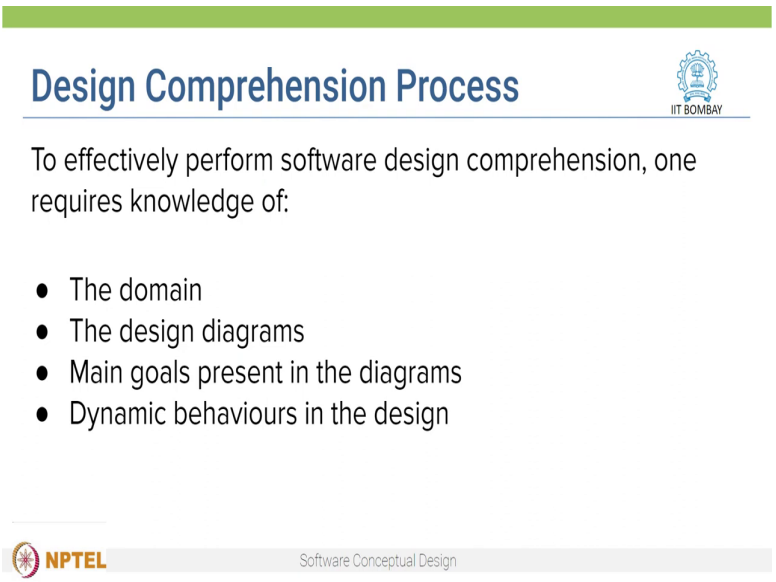
Let us take the example of the design of the mood based music player, while trying to comprehend this design, one needs, first of all, an understanding of the domain. Problem domain knowledge refers to the knowledge about the context for which the design has been developed.

For example, in the case of the mood based music player, students should be familiar with how a music player works. One also needs knowledge about the design diagrams itself. The design diagram knowledge is based on the syntax and semantics of different design diagrams. For example, students should understand various symbols and representations in the class and the sequence diagrams, and what they mean in the context of software design.

One should also understand the main goals of these design diagrams. The main goals refer to the purpose of each diagram and its role in the design. For example, a sequence diagram for a particular functionality can be composed of a sequence of sub goals that realize that functionality. For example, a sequence diagram for login into the mood-based music player will include sub-goals such as user input, functionality is to do when the login is successful and what to do when the login is unsuccessful.

One also requires knowledge about the dynamic behaviours in the design. When expert designers see a set of design diagrams, they think about the dynamic behaviours in the design. This involves knowledge about how data members in the class diagram change based on the execution of the sequence diagram. For example, when a login is successful, a user's profile and playlists are fetched from the database, and corresponding values of the user profile and the playlist should be updated.

(Refer Slide Time: 07:11)



To effectively perform software design comprehension, one requires knowledge of : the domain, the design diagrams, the main goals present in the diagrams and the dynamic behaviours in the design, which is nothing but how the system works in real time.

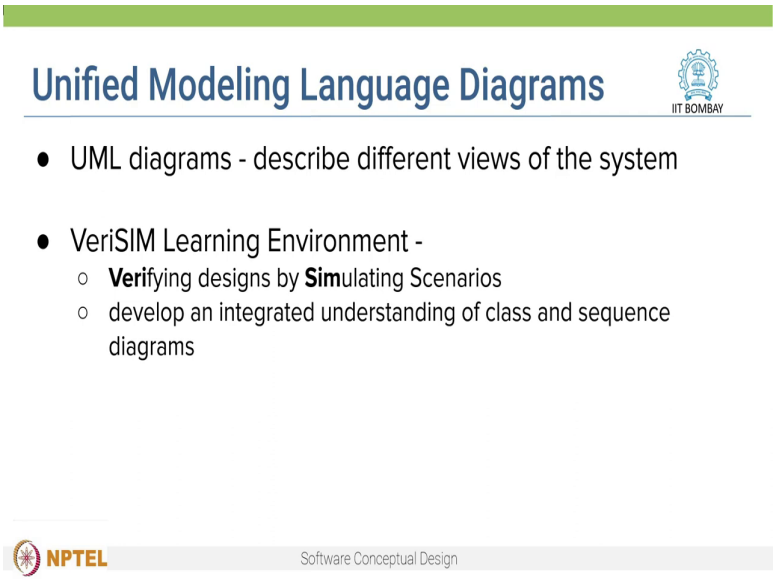
Ok. So, that is how experienced software designers make sense of each of these diagrams, but I wonder if there is a way to help students comprehend such diagrams.

Yes, the Verisim learning environment helps in just that. Verisim is a learning environment which helps students effectively comprehend various design diagrams. We look at the activities and features of Verisim in more detail in the next video.

Software Conceptual Design
Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering
Indian Institute of Technology, Bombay

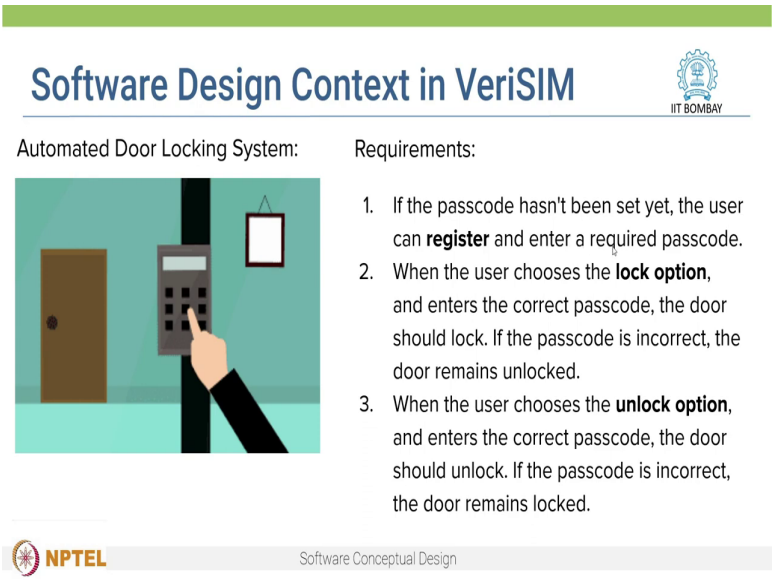
Lecture - 18
VeriSIM : A Learning environment for Comprehending Software Designs

(Refer Slide Time: 00:03)



In the previous video, we looked at the unified modelling language or the UML diagrams. We saw that UML diagrams describe different views of the system. In this video, we will be looking at the VeriSIM Learning environment. VeriSIM stands for verifying designs by simulating scenarios. Using this learning environment, you will be able to develop an integrated understanding of the class and the sequence diagrams.

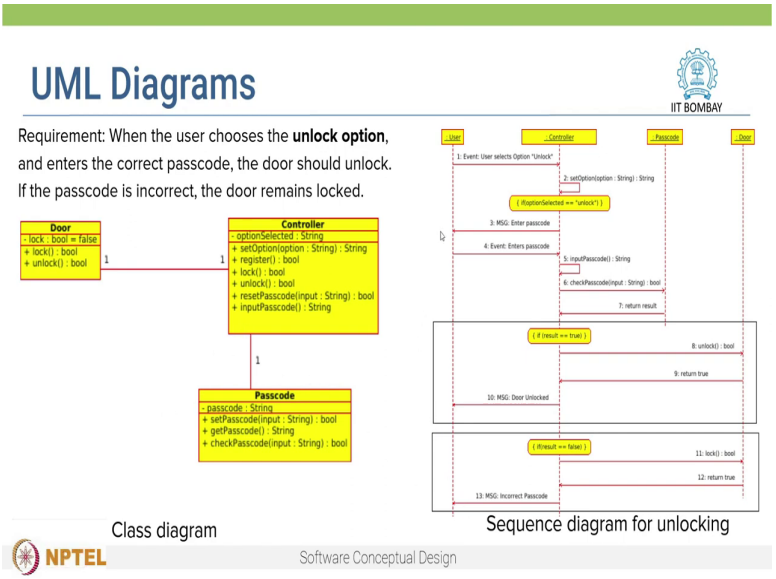
(Refer Slide Time: 00:42)



So, let us look at the software design context in VeriSIM. So, in VeriSIM, you will be introduced to an automated door locking system you will be introduced to the requirements of the system. For example, one requirement can be that the user can register himself or herself by entering a passcode, then the user can choose a lock or an unlock option. So, when a user chooses a lock option and enters the correct passcode, the door should lock.

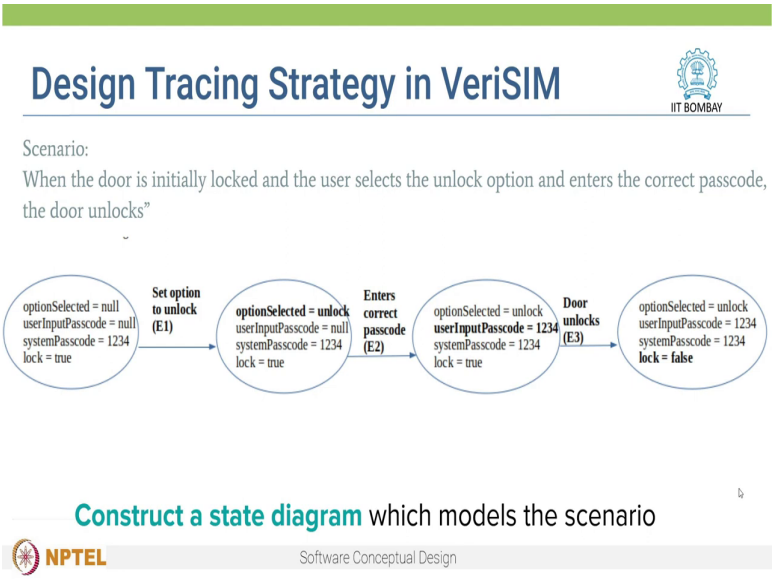
However, if the passcode is incorrect, the door should remain unlocked; a similar requirement is there for the unlock option as well.

(Refer Slide Time: 01:37)



Now, these requirements are modelled using various UML diagrams, such as the class diagram, which is shown here on the left and one of the sequence diagrams for unlocking is shown on the right. The automated door locking system has a class diagram and several sequence diagrams which model the requirements of the system.

(Refer Slide Time: 02:08)



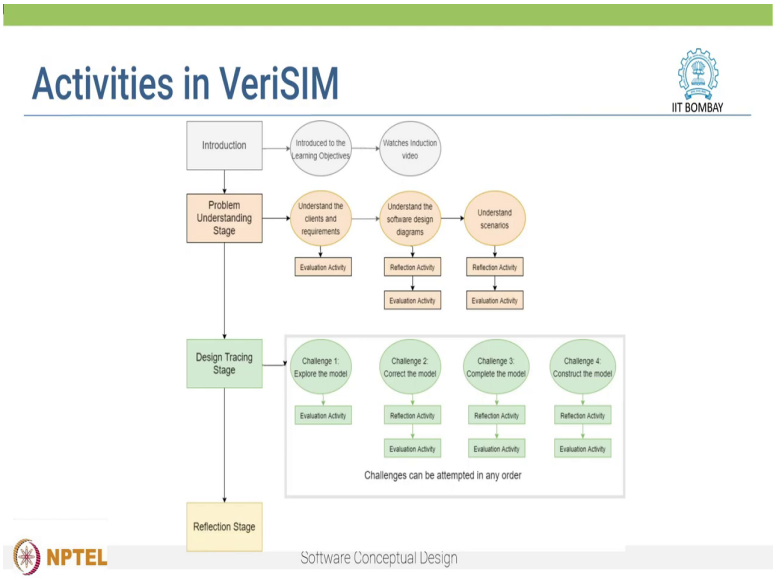
So, in VeriSIM, you will be introduced to a strategy known as the design tracing strategy. So, consider a scenario of the automated door locking system. So, let us say that the door is initially locked, and the user selects the unlock option and enters the correct passcode on doing this, the door should unlock. So, what is design tracing? In design tracing, you construct a state diagram which models a given scenario. So, in this case, this state diagram models this given scenario.

We can see that the transitions in the state diagram correspond to parts of the scenario. So, we see initially that the user selects the unlock option and then they enter the correct passcode and finally, is the door unlocks. So, these transitions correspond to messages in the sequence diagram, and based on these transitions, we see that the states which contain data variables and their values these keep on changing.

So, for example, when the user selects the unlock option, the value for the variable option selected changes to unlock. These variables are taken from the class diagram, and the transitions are messages which are taken from the sequence diagram. So, as you construct

these state diagrams, this will help you model different scenarios in the system and thereby help you develop an integrated understanding of the class and the sequence diagrams.

(Refer Slide Time: 04:14)



So, what are the activities in VeriSIM? You will be first introduced to what VeriSIM is what the learning objectives are, then you will move on to the problem understanding stage, where the requirements and the design diagrams for an automated door locking system will be given to you. The next stage is the design tracing stage, where you will attempt different challenges which will help you construct the state diagram which you saw in the previous slide and these challenges can be attempted in any order.

And finally, you will go through the reflection stage, which will help you reflect on what you have learnt in VeriSIM. After each of these activities, there are evaluation as well as reflection activities which will help you reflect on what you have learnt in that activity. So, this information is enough for you to now go and attempt VeriSIM. Happy learning.

Software Conceptual Design
Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering
Indian Institute of Technology, Bombay

Lecture - 19
Software Design Quality Perspectives

In the previous video, we understood the importance of design comprehension and how experienced designers comprehend a given design. We also interacted with the VeriSIM learning environment. Verisim helped us understand how to comprehend a given design by tracing various scenarios in the design.

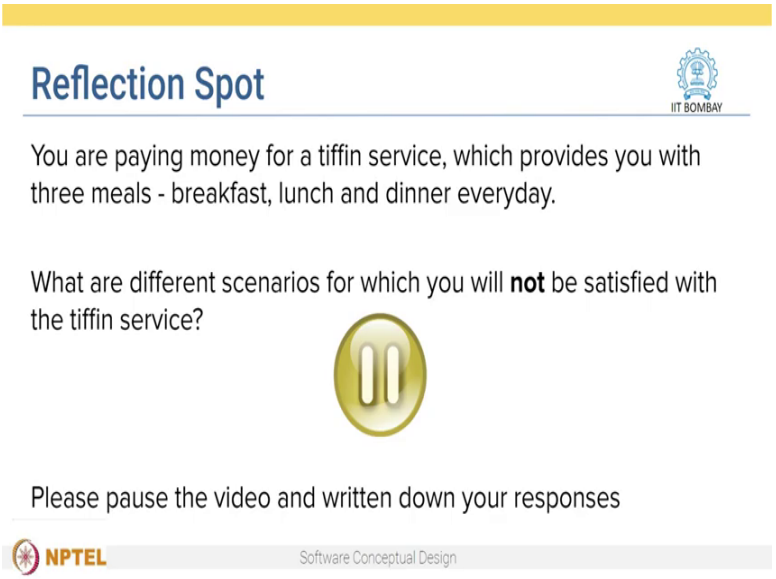
Ok. So, we have modelled created a design comprehended, and now we also have an integrated understanding of the design.

Yes, along with comprehending a design, we also have to evaluate whether the design is correct.

Design is correct in what sense? Can you help me understand this with an example?

Yes, let us take an example from a non-software context it might help us understand the concept better.

(Refer Slide Time: 01:15)



Let us say that you are paying money for a tiffin service which provides you with three meals, breakfast, lunch and dinner every day. So, what are different scenarios for which you will not be satisfied with the tiffin service? You can pause this video, write down your responses and then proceed.

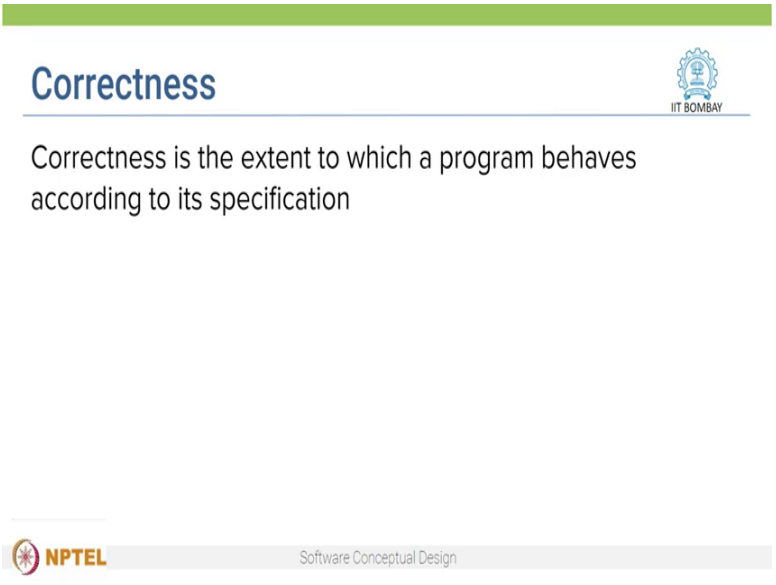
So, the first thing is that I want my food to be delivered on time, and I do not want any mix-ups happening. For example, the service should not be giving me breakfast for lunch.

Yes, exactly, so what you actually mentioned now these are the basic requirements right? The tiffin service should make sure that these basic requirements are satisfied this is known as correctness.

Ok. So, going back to the software design context, we need to ensure the correctness of the design we make sure of the design satisfies all the requirements we can say that the design is correct in some sense.

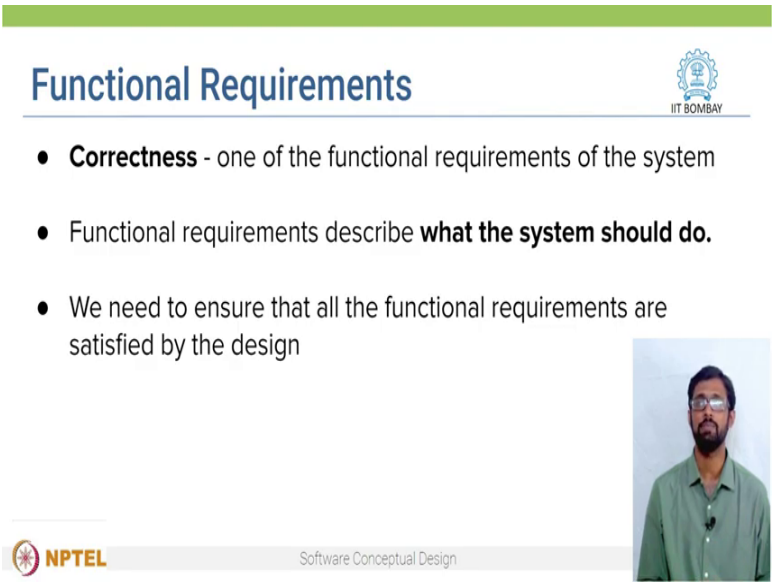
Yes, you are right. When you check for correctness, you ensure that the design satisfies all that your system should do.

(Refer Slide Time: 02:49)



Correctness is the extent to which a program behaves according to its specification. Learners, this is what you did in VeriSIM as well, using design tracing, you checked whether various scenarios in the design correctly satisfied all the requirements of the automated door-locking system.

(Refer Slide Time: 03:20)



Correctness is one of the functional requirements of the system. Functional requirements describe what the system should do. We need to ensure that all the functional requirements are satisfied by the design.

So, checking if the functional requirements are satisfied is one way to evaluate the given design, but is it the only way are there other perspectives we need to consider while evaluating a given design?

Yes, there are several other ways to evaluate a software design, it is necessary to evaluate a design against various quality attributes.

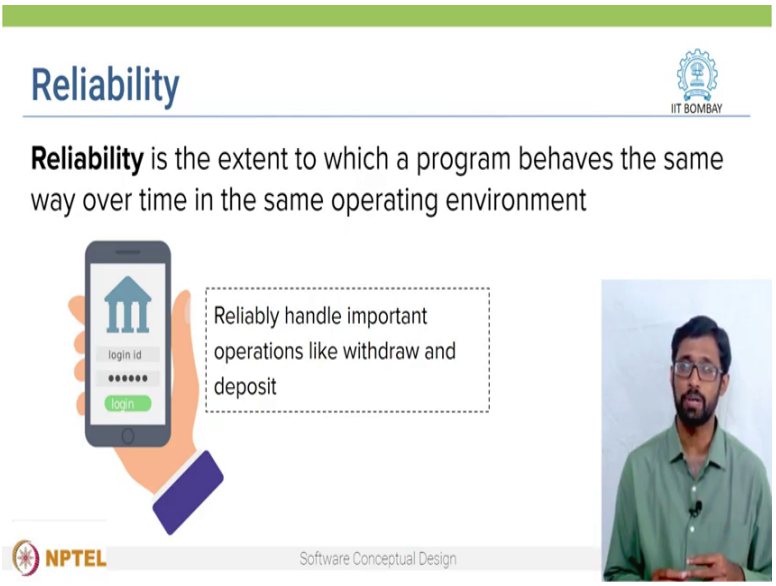
Quality attributes? can you help me understand this with an example?

Yes, let us go back to the tiffin service example. What else do you expect from the tiffin service?

Ok. Next, I want the service to be reliable for example, if I miss more than three meals a week, I would not be happy.

Exactly this is termed as reliability.

(Refer Slide Time: 04:46)

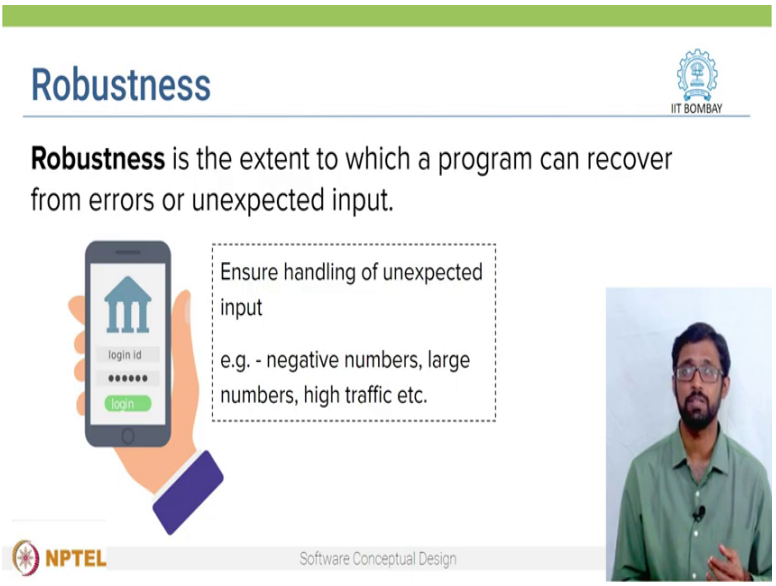


Reliability is the extent to which a program behaves the same way over time in the same operating environment. In case of software, we need to ensure that it does not crash often; otherwise, users will get frustrated. For example, in a mobile wallet like Amazon Pay, reliability is even more important. We must ensure that the wallet reliably handles all operations especially important operations like withdrawal and deposit.

I also want the service to handle unexpected situations reasonably well. For example, if the main chef falls sick, is there a replacement? Is there any other option to serve the customers?

Ok. So this is similar to the quality attribute of being robust even in unexpected situations.

(Refer Slide Time: 05:59)

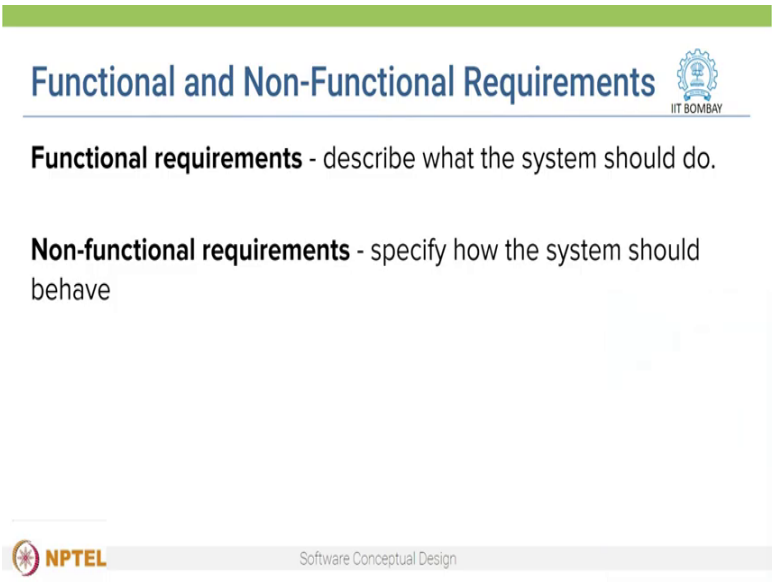


Robustness is the extent to which a program can recover from errors or unexpected input. For example, in a mobile wallet system like Amazon Pay we have to ensure that the service handles unexpected input like negative numbers and large numbers and high traffic as well.

So, while checking for quality attributes like reliability and robustness, we are, in a way checking how the system is behaving and not exactly checking the system requirements. So, are these quality attributes different from the functional requirements of the system?

Yes, you are right attributes like reliability and robustness are known as the non-functional requirements of the system.

(Refer Slide Time: 06:58)



Non-functional requirements essentially specify how the system should behave. For example, we looked at two non-functional requirements reliability and robustness.

Ok. So, functional and non-functional requirements have to be considered while designing, creating and evaluating the system.

Yes, you are right to summarize in this video, we discussed functional and non-functional requirements. There are other non-functional requirements such as performance portability, security, and so on. Do check out the additional resources for more information.

Software Conceptual Design

Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Department of Computer Science and Engineering
Indian Institute of Technology, Bombay

Lecture - 20
Week 3: Summary

To summarize, this week, we looked at another form of modeling called the UML representation. The activities in VeriSim would have helped you to develop an integrated understanding of different UML diagrams. We also saw that there are functional and non-functional requirements.

And we saw examples of quality attributes such as reliability and robustness. It is always good to explicitly note down these quality attributes and design decisions based upon them, since they are often implicit and not usually represented in the diagrams.

Software Conceptual Design
Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Indian Institute of Technology, Bombay

Week - 04
Lecture - 21
Tying it all Together

In this week, we will be recapping what we have learnt in the previous weeks. In week 1 we learnt about the different phases of software development process in week 2 we learnt about the FBS framework. In this week, we will give an overview of how FBS relates to the phases in software development process.

In week 3 we learnt about another form of modelling using UML diagrams in this week now, we will learn about how these forms of modelling describe the software design at different levels of abstraction. We will also provide you with various videos on experiences of how software developers work in the industry today, and we will have discussions and questions based on these videos.

Software Conceptual Design
Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Indian Institute of Technology, Bombay

Lecture - 22
Software Development Process Recap

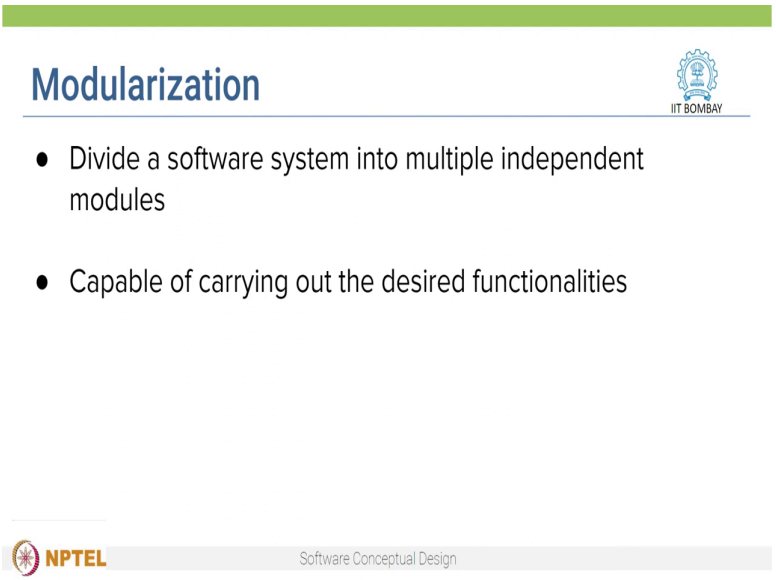
Since we are at the last week of the course it is important to reflect on the learnings and concepts we learnt in the previous weeks so that we can make connections between them.

Yes, I agree also making connections between the newly learned concepts and the earlier known concepts is an important exercise.

So, in week 1 we looked at how a relatively large software like Amazon is built, it is made up of several interworking modules.

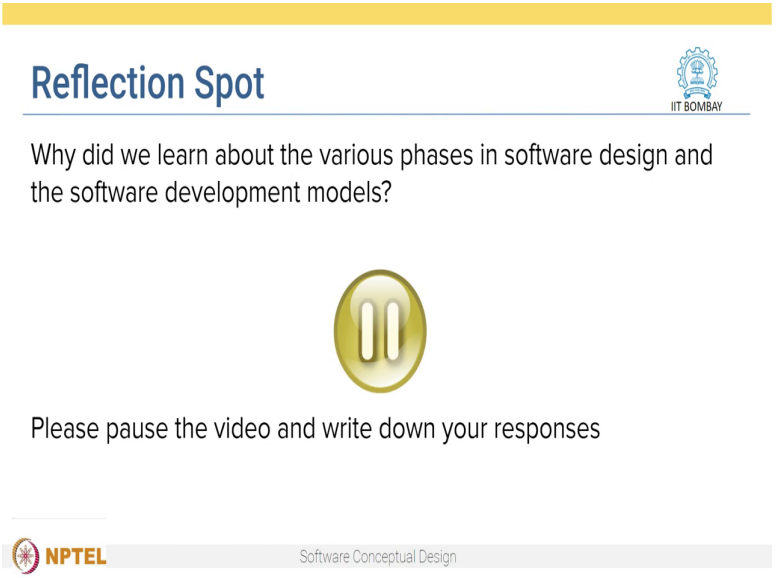
You are right, and this concept is known as modularization.

(Refer Slide Time: 00:48)



Modularization is used to divide a software system into multiple independent and co working modules. These modules are capable of carrying out the desired functionalities, and they form the basic building blocks for the entire software.

(Refer Slide Time: 01:12)



Also, in week 1 we learnt about the different phases and the software development models. So, learners here is a question for you why did not we learn about the various phases in software design and the development models? Pause the video write your answer in the notebook, and resume.

Software development does not start immediately with programming and creating a software it begins with understanding the needs of the end user, creating a plan, verifying it and then implementing it using a programming language. After creating the software we test it and then deploy it at the customer's end.

So, software development is a systematic process of problem-solving.

Yes, you are right, the software development phases are broad and could be done iteratively and in many different ways.

So that is why it is important for software developers to know and understand these phases and processes. We also compared and contrasted software design we design in other domains, specifically architecture.

Yes, the abstract and intangible nature of software adds to the complexity in software design and development. However, there are similarities in the design processes across the domains, much like the steps to problem-solving.

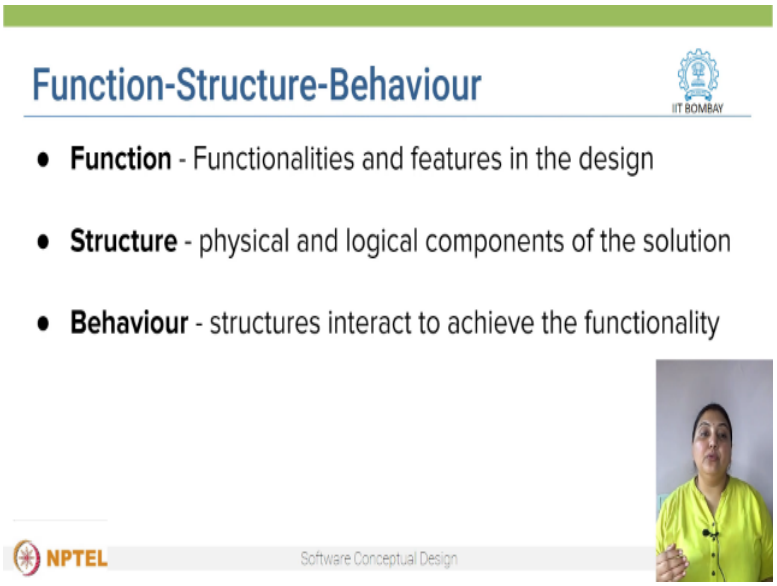
Well, in week 2 and 3 we created software designed models using the FBS design framework and the UMI diagrams. How do these models map to the software development phases and the processes?

Yes, that is an important question we will discuss this in detail in the next video.

Software Conceptual Design
Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Indian Institute of Technology, Bombay

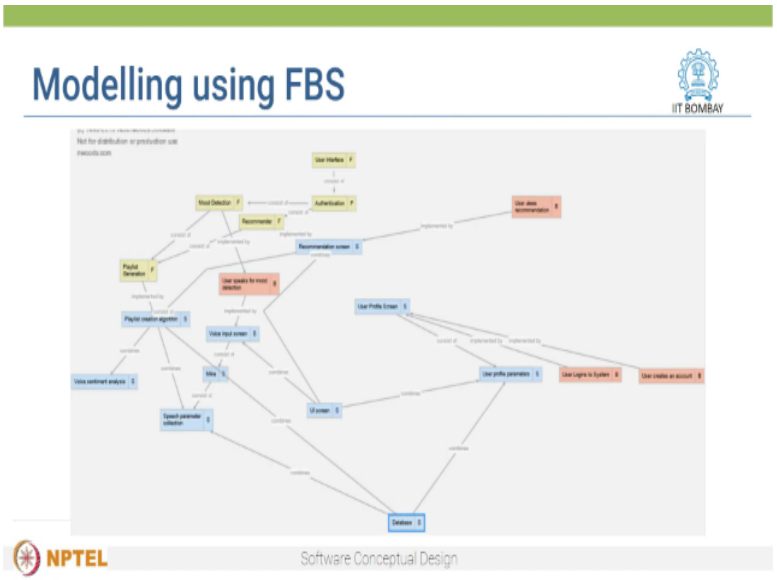
Lecture - 23
Mapping
Function-Behaviour-Structure to the Software Development Process

(Refer Slide Time: 00:06)



In week 2, we introduce the Function Behaviour Structure design framework. The function corresponds to the functionalities and features of the solution designed. Its structures correspond to the physical and logical components of the solution. The mechanism in which the structures interact to achieve the functionality is termed as behaviours.

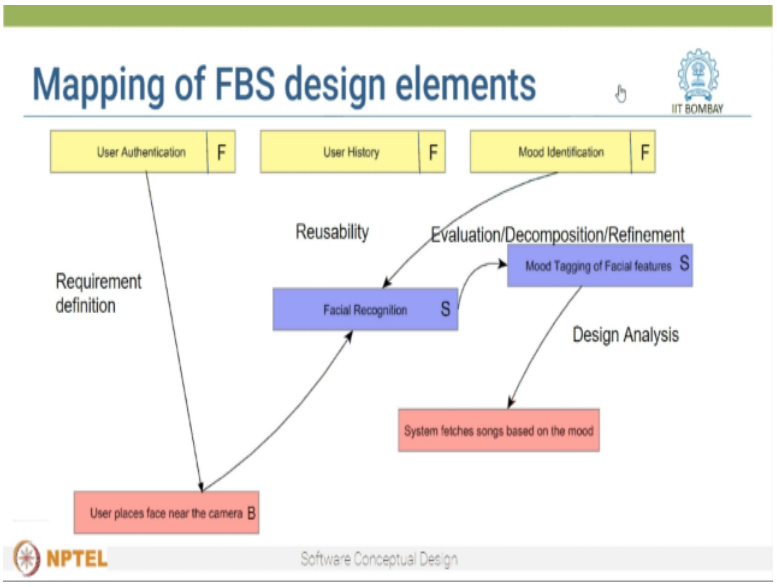
(Refer Slide Time: 00:32)



We then created an FBS graph which was an integrated model of the solution design. We use the think and link learning environment to create and evaluate FBS graphs for different software design problems. In week 2 we created FBS to the software design solutions, but I am just wondering how does this map to the software processes and the development models that we just discussed in week 1?

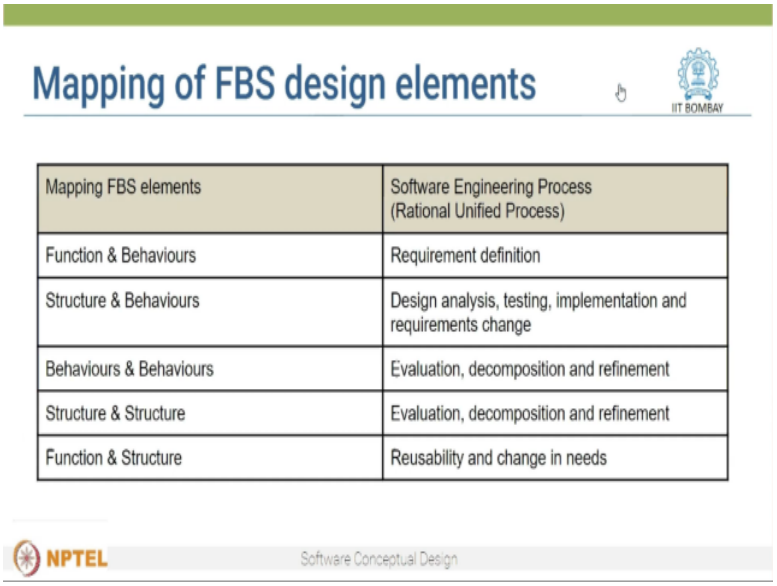
That is a good question, actually, there is a mapping of the various processes like requirement definition, design, implementation, testing, deployment, do the function structure behaviour framework? So how is this mapping actually done, can you please explain?

(Refer Slide Time: 01:32)



While mapping the FBS design elements, certain software processes get initiated. For example, when the function is mapped to a behaviour requirement definition process gets initiated when a structure is mapped to a behaviour, the design analysis process gets initiated. So on and so forth, certain doors while mapping the software process gets initiated.

(Refer Slide Time: 02:07)



Mapping FBS elements	Software Engineering Process (Rational Unified Process)
Function & Behaviours	Requirement definition
Structure & Behaviours	Design analysis, testing, implementation and requirements change
Behaviours & Behaviours	Evaluation, decomposition and refinement
Structure & Structure	Evaluation, decomposition and refinement
Function & Structure	Reusability and change in needs

In this table, we see the different software engineering processes getting initiated while the FBS design elements are mapped to each other.

Software Conceptual Design
Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Indian Institute of Technology, Bombay

Lecture - 24
Abstractions in Modelling

In the previous video, we saw how the FBS framework mapped to the software development process.

Yes, in week 3 we saw another set of diagrams, the unified modelling language diagrams. The process of representing the solution designed using representations like FBS and UML is referred to as software modelling.

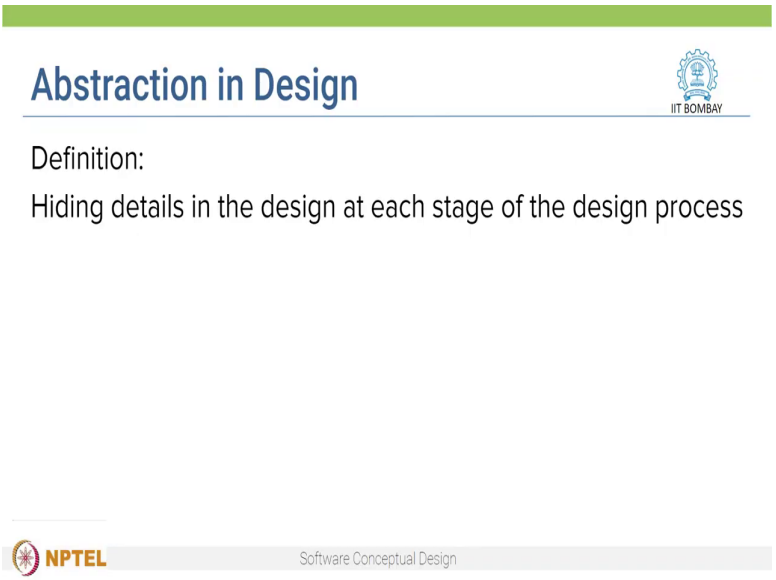
Modelling is a way of creating an external, explicit representation of the system to be built based on the requirements we create an FBS representation to expand an idea or concept based on the system which we want to build.

So, once we have this representation, we can go into some more detail and create UML diagrams for the software system to be built.

So, one of the key differences between these modelling representation is a level of detail in them, right?

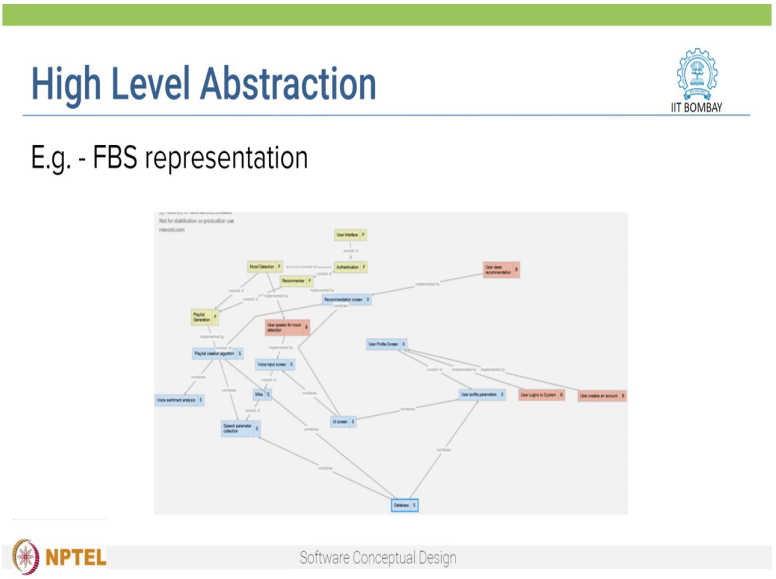
Yes, you are right. So, these levels of detail are known as levels of abstraction.

(Refer Slide Time: 01:19)



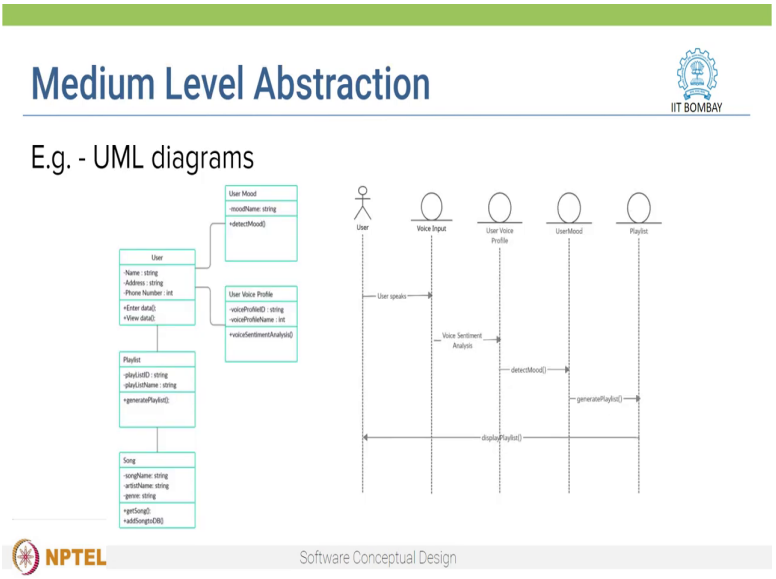
Abstraction is a way of hiding details in the design at each stage of the design process.

(Refer Slide Time: 01:33)



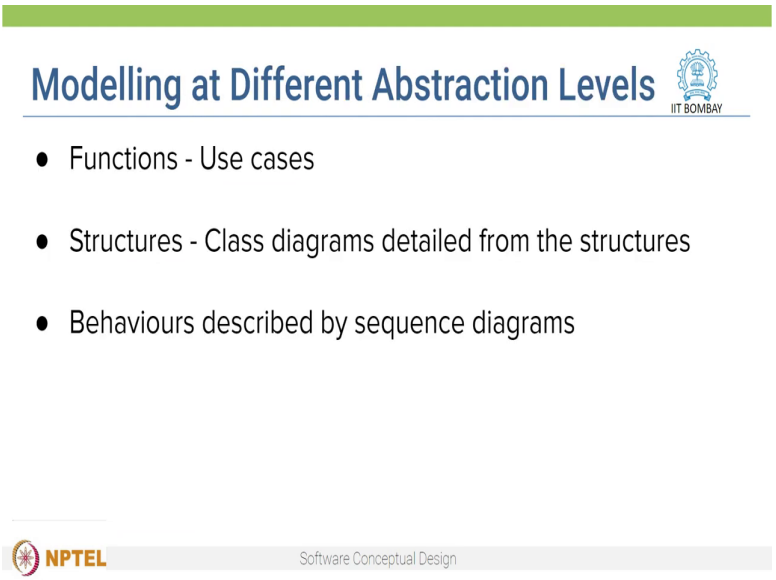
In week 2, we create a phase graph which expands on the function structure in behaviour of the design solution, it is an example of a high level extraction of the system.

(Refer Slide Time: 01:46)



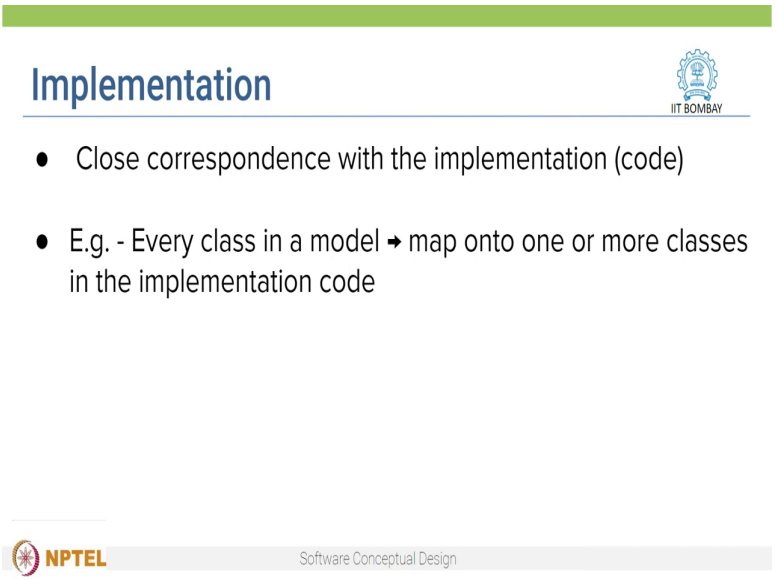
We then add more and more details to the solution design. In week 3 we were introduced to the UML diagrams, they correspond to the medium level abstraction of the system.

(Refer Slide Time: 02:00)



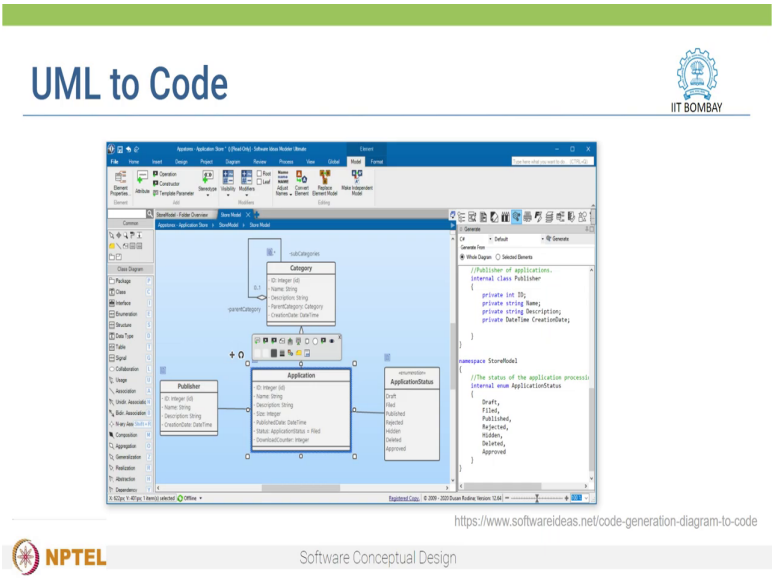
In think and link we saw how we can expand from the FBS model to the UML model. The functions in the FBS graph can be expanded to create use cases, similarly, the class diagrams can be detailed from the structures. The sequence diagram actually describe the different behaviours in the system.

(Refer Slide Time: 02:30)



Then these UML diagrams can be further detailed out to create models such that there is a close correspondence with the implementation code. For example, every class in a model could map on to one or more classes in the implementation code.

(Refer Slide Time: 02:56)

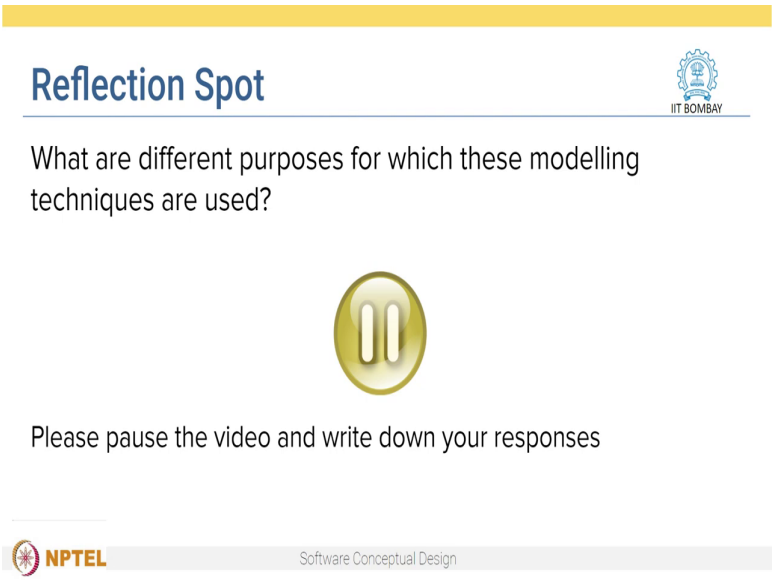


There are also various tools which automatically generate code from such detailed UML diagrams. So, what we are doing is that we are adding more and more details as we progress across the various levels of abstraction.

Yes, this is an important takeaway. One way to create a software design is to continue adding implementation details as we progress.

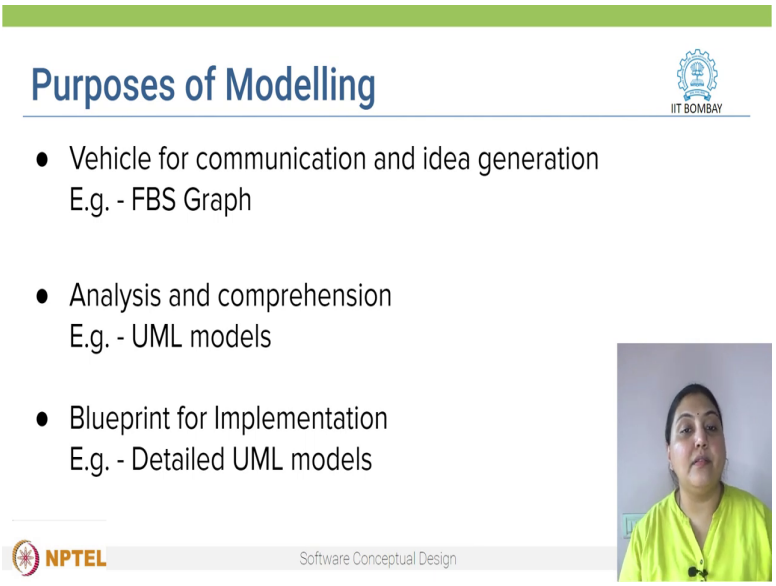
Ok. So, another point is that these modelling techniques they are not the same right? So, do they serve different purposes?

(Refer Slide Time: 03:43)



That is a good question. Learners, let us reflect on this question what are the different purposes for which these modelling techniques can be used? Pause the video write your answer in the notebook, and then resume.

(Refer Slide Time: 04:10)



For example, the FBS graph is a vehicle for communication and idea generation when we want to discuss key ideas of the software with other members of team a modelling technique like FBS can be used.

These ideas are detailed out using modelling representations like the UML models these UML models can be used for analysis and comprehension, like what we saw in VeriSIM.

In the latest example, we saw that when we generate UML diagrams with greater level of detail, the automatically code gets generated. These detailed UML diagrams can be given to developers who can start implementation in a specific programming language based on these models.

Software Conceptual Design
Prof. Sridhar Iyer Prof. Prajish Prasad Prof. T. G. Lakshmi
Indian Institute of Technology, Bombay

Week - 04
Lecture - 25
Summary

In this week, we have not only done a recap of what we have learnt in the previous weeks, but we have also seen some expert practices. We have seen how experts tend to spend a lot of time in the initial phases of design, such as understanding the problem, creating models, checking whether these models make sense in different scenarios before actually going on to create the solution.

Similarly, through this course, we have presented you with the various steps and processes that you could do before you actually implement a software. This is the process of software conceptual design.

THIS BOOK
IS NOT FOR
SALE
NOR COMMERCIAL USE

(044) 2257 5905/08
nptel.ac.in
swayam.gov.in